Relation Algebra

§1. Notation

Traditionaly Adjunction F -| G is defined as $F(x) \leq y$ iff $x \leq G(y)$, and you derive laws like this:

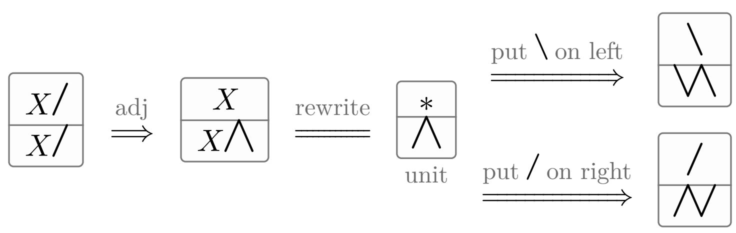
$$\begin{aligned} F(x) \leq F(x) \quad &\Longrightarrow \quad x \leq G(F(x)) \quad \Longrightarrow \quad F(x) \leq F(G(F(x))) \\ G(y) \leq G(y) \quad &\Longrightarrow \quad F(G(y)) \leq y \quad \Longrightarrow \quad G(F(G(y))) \leq G(y) \end{aligned}$$

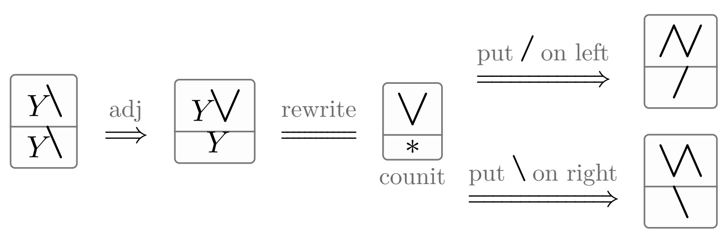
By define $X : * \mapsto x$, $Y : * \mapsto y$ and use diagram order, application can be replaced by composition: $XF \leq Y$ iff $X \leq YG$

(1a)

- write F as $/$, G as $\backslash$
- render $a \leq b$ as $\boxed{\frac{a}{b}}$
- **rewrite** $X \leq XF$ as $1 \leq F$

$$\boxed{\frac{X/}{Y}} = \boxed{\frac{X}{Y\backslash}} \tag{adj}$$





now we have $/\backslash/ = /$ and $\backslash/\backslash = \backslash$, and you just discovered string diagrams.

§1.1. Adjunctions

(1.1a)

F⊣G	F's type	monad FG	𝟙⊑FG	GF⊑𝟙	F preserves ∪	G preserves ∩	FGF=F	GFG=G
◁⊣▷	A→ A⊗A	◁▷=𝟙	𝟙⊑◁▷	▷◁⊑𝟙	(R ∪ S)◁= R◁ ∪ S◁	(R∩S)▷= R▷∩S▷	◁▷◁=◁	▷◁▷=▷
⊸⊣⟜	A→𝕀	⊸⟜=⊤	𝟙⊑⊸⟜	⟜⊸⊑𝟙	(R ∪ S)⊸= R⊸ ∪ S⊸	(R∩S)⟜= R⟜∩S⟜	⊸⟜⊸=⊸	⟜⊸⟜=⟜
°⊣°	(A→B)→ (B→A)	°°=𝟙	R=R°°	R=R°°	(R ∪ S)°= R° ∪ S°	(R∩S)°= R°∩S°	R°°°=R°	R°°°=R°
⟜◁⊣▷⊸	(X⊗A→Y)→ (X→Y⊗A)	𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	=	=	=	=
▷⊸⊣⟜◁	(X→Y⊗A)→ (X⊗A→Y)	𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	=	=	=	=
Δ⊣∩	(A→B)→ (A→B)²	R↦R∩R=R	R⊑R∩R	R∩S⊑R	Δ(R ∪ S)= ΔR ∪ ΔS	(R∩T)∩(S∩U)= (R∩S)∩(T∩U)	R∩R=R	R∩R=R
∪⊣Δ	(A→B)²→ (A→B)	(R,S)↦ (R ∪ S,R ∪ S)	R⊑R ∪ S	R ∪ R⊑R	(R ∪ T) ∪ (S ∪ U)= (R ∪ S) ∪ (T ∪ U)	Δ(R∩S)= ΔR∩ΔS	R ∪ R=R	R ∪ R=R
⊥⊣!	{*}→ (A→B)			⊥⊑R				
·S⊣/S	(A→B)→ (A→C)	S/S	R⊑(RS)/S	(T/S)S⊑T	(R ∪ T)S= RS ∪ TS	(R∩T)/S= R/S∩T/S	((RS)/S)S =RS	((T/S)S)/S =T/S
S·⊣S\	(B→C)→ (A→C)	S\S	R⊑S\(SR)	S(S\T)⊑T	S(R ∪ T)= SR ∪ ST	S\(R∩T)= S\R∩S\T	S(S\(SR)) =SR	S\(S(S\T)) =S\T
R∩⊣R⇒	(A→B)→ (A→B)	X↦R⇒(X∩R)	X⊑R⇒(X∩R)	R∩(R⇒Y)⊑Y	R∩(X ∪ Y)= (R∩X) ∪ (R∩Y)	R⇒(X∩Y)= (R⇒X)∩(R⇒Y)	R∩(R⇒(X∩R)) =X∩R	R⇒(R∩(R⇒Y)) =R⇒Y
𝓓⊣·⊤	(A→B)→ Cor A	R↦(𝓓R)⊤	R⊑(𝓓R)⊤	𝓓(A⊤)⊑A	𝓓(R ∪ S)= 𝓓R ∪ 𝓓S	(A∩B)⊤= A⊤∩B⊤	𝓓((𝓓R)⊤) =𝓓R	(𝓓(A⊤))⊤ =A⊤
𝓡⊣⊤·	(A→B)→ Cor B	R↦⊤(𝓡R)	R⊑⊤(𝓡R)	𝓡(⊤A)⊑A	𝓡(R ∪ S)= 𝓡R ∪ 𝓡S	⊤(A∩B)= ⊤A∩⊤B	𝓡(⊤(𝓡R)) =𝓡R	⊤(𝓡(⊤A)) =⊤A
·f⊣·f°	(A→B)→ (A→C)	ff°	𝟙⊑ff°	f°f⊑𝟙	(R ∪ S)f= Rf ∪ Sf	(R∩S)f°= Rf°∩Sf°	ff°f=f	f°ff°=f°
f°·⊣f·	(A→C)→ (B→C)	ff°	𝟙⊑ff°	f°f⊑𝟙	f°(X ∪ Y)= f°X ∪ f°Y	f(X∩Y)= fX∩fY	ff°f=f	f°ff°=f°
i⊣E	Map↪Rel	E	$\frac{1}{\ni}$:A→EA	∋:EB→B	—	—	$\frac{\ni}{\ni}$=𝟙	$\frac{R}{\ni}$∋=R
·∋⊣ $\frac{}{\ni}$	Map(A,EB)→ (A→B)	𝟙	$\frac{f\ni}{\ni}$=f	$\frac{R}{\ni}$∋=R	—	—	$\frac{f\ni}{\ni}\ni$ =f∋	$\frac{\frac{R}{\ni}\ni}{\ni}$ =$\frac{R}{\ni}$
$\frac{}{\ni}$ ⊣·∋	(A→B)→ Map(A,EB)	𝟙	$\frac{R}{\ni}$∋=R	$\frac{f\ni}{\ni}$=f	—	—	$\frac{\frac{R}{\ni}\ni}{\ni}$ =$\frac{R}{\ni}$	$\frac{f\ni}{\ni}\ni$ =f∋

§1.2. Composing adjunctions

(1.2a)

$$\underset{\text{f a map}}{\frac{X}{f(R/T)}} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \frac{f^\circ X}{R/T} \underset{\cdot T\dashv /T}{\Longleftrightarrow} \frac{(f^\circ X)T}{R}$$

$$\underset{\text{associativity}}{\Longleftrightarrow} \frac{f^\circ(XT)}{R} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \frac{XT}{fR} \underset{\cdot T\dashv /T}{\Longleftrightarrow} \frac{X}{(fR)/T}$$

$$\underset{\text{indirect equality}}{\Longrightarrow} \frac{f(R/T)}{(fR)/T}$$

	·T	T·	·g	g°·	T∩	°	⟜◁	Δ
·S	R/(ST)= (R/T)/S	T\(R/S)= (T\R)/S	R/(Sg)= (Rg°)/S	g(R/S)= (gR)/S	—	(Y/S)°= S°\(Y°)	(RS)^= R^(S⊗𝟙)	$(T_1 \cap T_2)/S= T_1/S \cap T_2/S$
S·	S\(R/T)= (S\R)/T	(TS)\R= S\(T\R)	S\(Rg°)= (S\R)g°	(g°S)\R= S\(gR)	—	(S\Y)°= Y°/S°	((S⊗𝟙)R)^= SR^	$S\backslash(T_1 \cap T_2)= S\backslash T_1 \cap S\backslash T_2$
·f	R/(fT)= (R/T)f°	T\(Rf°)= (T\R)f°	(fg)°=g°f°			(fY)°=Y°f°		$(T_1 \cap T_2)f°= T_1 f° \cap T_2 f°$
f°·	f(R/T)= (fR)/T	(Tf°)\R= f(T\R)	—	(fg)°=g°f°		(fY)°=Y°f°		$f(T_1 \cap T_2)= fT_1 \cap fT_2$
R∩	—	—	—	—	(R∩T)⇒Y= R⇒(T⇒Y)	(R⇒Y)°= R°⇒(Y°)	—	$R \Rightarrow (T_1 \cap T_2)= (R \Rightarrow T_1) \cap (R \Rightarrow T_2)$
°	(Y/T)°= T°\(Y°)	(T\Y)°= Y°/T°	(gY)°=Y°g°	(gY)°=Y°g°	(T⇒Y)°= T°⇒(Y°)	Y°°=Y	(R^)°= (R°⊗𝟙) (𝟙⊗▷⊸) both bends: the ° section's display	$(T_1 \cap T_2)°= T_1° \cap T_2°$
⟜◁	(RT)^= R^(T⊗𝟙)	((T⊗𝟙)R)^= TR^	—	—	—	(R^)°= (R°⊗𝟙) (𝟙⊗▷⊸) both bends: the ° section's display	—	—
∪	$(X_1 \cup X_2)T= X_1 T \cup X_2 T$	$T(X_1 \cup X_2)= TX_1 \cup TX_2$	$(X_1 \cup X_2)g= X_1 g \cup X_2 g$	$g°(X_1 \cup X_2)= g°X_1 \cup g°X_2$	$T \cap (X_1 \cup X_2)= (T \cap X_1) \cup (T \cap X_2)$	$(X_1 \cup X_2)°= X_1° \cup X_2°$		
⊥	⊥T=⊥	T⊥=⊥	⊥g=⊥	g°⊥=⊥	T∩⊥=⊥	⊥°=⊥		

(1.2b)

§1.3. Adjoint triples

Beyond ∪⊣Δ⊣∩, which §1.1 already carries as two rows, the table holds one adjoint triple with content: ∃_f⊣f*⊣∀_f along a map f, once on each side of the composite. f : A→B throughout this subsection.

$$\cdot f \dashv \cdot f° \dashv /f° \qquad (1.3a)$$
$$f°\cdot \dashv f\cdot \dashv f\backslash$$

The first two links of each chain are the map shunting rules, and the third is §1.1′s division row read at a chosen divisor: ·S⊣/S at S:=f°, and S·⊣S\ at S:=f.

The third link is not the first over again. For a map f, /f collapses to ·f°; being a map is exactly what that collapse spends R/(fS)=(R/S)f°. — f° is not a map, so /f° does not collapse in turn, and the three operators stay distinct. On Rel(𝕀,A)=EA they are the image triple:

operator	acts by	name
·f	S↦f[S]	direct image, ∃_f
·f°	$T \mapsto f^{-1}[T]$	inverse image, f*
/f°	$S \mapsto \{b : f^{-1}(b) \subseteq S\}$	∀_f

(1.3b)

§1.1′s 𝓓⊣·T is this same chain along the projection A⊗B→A, read through Rel(A,B)=E(A⊗B): 𝓓R={(a,a) : ∃b. aRb}, A⊤ is the relation that ignores b altogether, and the third link is R↦𝟙∩R/⊤={(a,a) : ∀b. aRb}.

$$\mathcal{D} \dashv \cdot\top \dashv \mathbb{1} \cap \cdot/\top \qquad (1.3c)$$

Three is where it stops, and one map breaks both ends. Take $A=\{a_1,a_2\}$ and B={b}: $f[\{a_1\}] \cap f[\{a_2\}]=\{b\}$ while $f[\{a_1\} \cap \{a_2\}]=\varnothing$, so ·f does not preserve meets and has no left adjoint, and the same two subsets give $\forall\_f(\{a_1\} \cup \{a_2\})=\{b\}$ against $\forall\_f\{a_1\} \cup \forall\_f\{a_2\}=\varnothing$, so /f° does not preserve joins and has no right adjoint. A monic f restores the binary case and no more — ⊤f still reaches only im f, and ∀_f⊥=B\im f is still not ⊥. The chain extends only when f° is a map as well, and then /f°=·f°°=·f and it repeats forever. For an S with neither S nor S° a map there is no triple at all: ·S⊣/S is two links and stops.

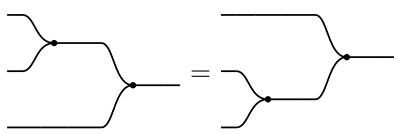
The rows that chain forever say nothing by it: ° is an order-isomorphism of hom-posets, so it is adjoint to itself on both sides and °⊣°⊣° has no end, and §1.1′s other row of that kind, ⟜◁⊣▷⊸, goes the same way.

§2. Relations

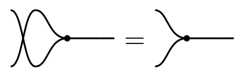
> Rel is a poset-enriched category with (⊗, °) where ⊗ is commutative cartisian product, and converse °⊣°, and
>
> C: (▷ : A⊗A→A, ⟜ : 𝕀→A) a commutative monoid with C°⊣C. We write C° as (◁ : A→A⊗A, ⊸ : A→𝕀)

(2a)

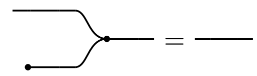
(2b)



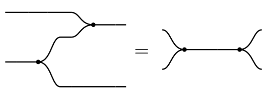
▷ associative



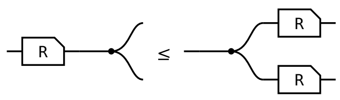
▷ commutative



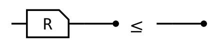
⟜ is its unit



Frobenius, one half — the other is its °



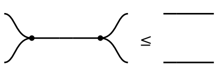
R◁≤◁(R⊗R)



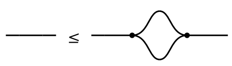
R⊸≤⊸

§2.1. ∀ object A, (A,◁,⊸)⊣(A,▷,⟜)

(2.1a)



▷◁≤𝟙



𝟙≤◁▷

⟜⊸≤𝟙

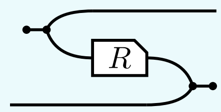
𝟙≤⊸⟜

The last of these is the only one that makes a picture **bigger**, and it is worth a name: **a wire is below the cut wire**. In `Rel` it reads `{(a,a)}⊆A×A` — cut a wire and its two ends stop having to agree, so cutting can only add pairs. It is the one weakening this calculus gives away for free.

§3. ° : $\mathcal{C}^{op} \longrightarrow \mathcal{C}$ is a 2 functor

(3a)

° is primitive, part of the data the first section lists. It turns both of R's wires round, and the Frobenius structure DRAWS that — the picture and the formula below are R°, not its definition:



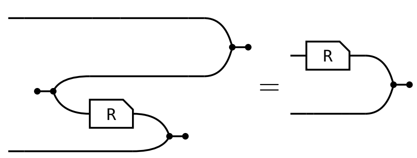
R°=(⊶◁⊗𝟙)(𝟙⊗R⊗𝟙)(𝟙⊗▷⊸)

where ⊶◁ : 𝕀→a⊗a opens a pair of wires out of nothing and ▷⊸ : b⊗b→𝕀 closes one, so the input of R° is where the output of R was. Taking that formula as the definition would now be circular: ◁ and ⊸ are ▷° and ⊶°. The laws of ° are that it is a contravariant 2-functor ° : $\mathcal{C}^{op}$→$\mathcal{C}$:

(i) 𝟙°=𝟙 (ii) (RS)°=S°R° (iii) (R⊗S)°=R°⊗S° (iv) R≤S implies R°≤S°

§3.1. The slide

The one rule (ii) and (iii) use, and each of them uses it twice. A converse facing a merge on the lower strand is the box itself, upright, on the upper one:

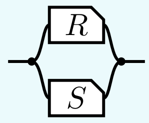


(3.1a)

R° is DEFINED as the bending of (R⊗𝟙)▷⊸, so the slide claims only that unbending it gives that back — and unbending undoes bending for every arrow. That is the snake above with a passenger: the ⊶◁ bends the a strand down and the ▷⊸ brings it back up, while the b strand rides through untouched. Nothing else is spent below.

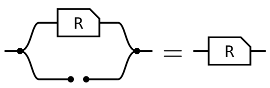
§4. ∩ is a commutative idempotent monoid on every hom-set

The **meet** of R,S : a→b, the paper's **convolution**, is R∩S:=◁(R⊗S)▷ — copy the input, run R and S on the two copies, merge the results — so what comes out is what both of them do. (4a)

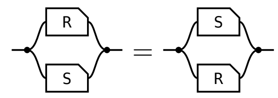


transcribed: a definition has no statement to export

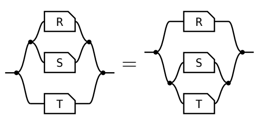
On every hom-set it is associative, commutative and idempotent , , with unit the maximal arrow ⊤=−∘∘− (the paper's Lemma 4.11).



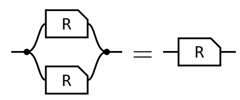
unit: one half of ⊤ per end — the merge's unit law absorbs the ∘−, the copy's counit law the −∘ (4b)



commutative: σ crosses R⊗S by naturality and is absorbed by cocommutativity and commutativity



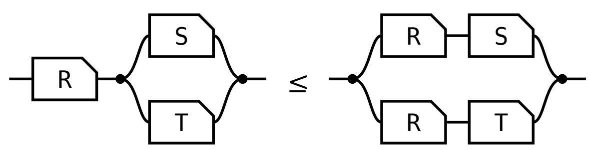
associative: coassociativity and associativity; ⊗ re-brackets for nothing, being strict here



idempotent: the one that is not bookkeeping — the lax copy law is the whole of it, worked in allegory2

So ≤ is the order this monoid induces. R∩S≤R comes from the unit, and idempotency turns anything under both S and T into something under S∩T, since R=R∩R≤S∩T.

And one law relating ∩ to composition, which is **not** an equation:

semi-distributivity, and what supplies it	picture
R (S∩T)⊑RS∩RT — the lax copy law. Equality exactly when R is single valued: the Maps section's F(R∩S)=FR∩FS.	

(4c)

§5. Domain and range

The **domain** `Dom(R)≜𝟙∩RR°` and the **range** `Ran R≜Dom(R°)`. (5a)

(5b)

▷ lands the return leg back on the value ◁ handed out,
so R°▷ cuts to ⊸ — Frobenius

Running `R` and throwing the result away leaves only the fact that `R` could fire, and `Ran R` is the same picture with the box mirrored. In `Rel` both steps are `{(a,a) : ∃b. a R b}`.

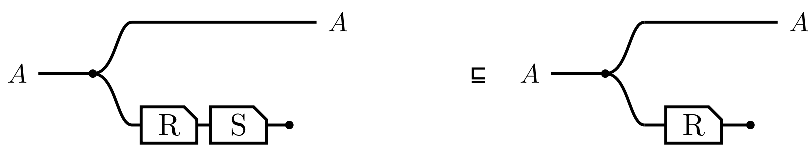
(5c)

`Dom(R)≜𝟙∩RR°`
`Dom(R)⊑A⟺R⊑AR`, for `A` coreflexive
`Dom(RS)⊑Dom(R)`
`Dom(R∩S)=𝟙∩SR°`
`R` entire ⟺`Dom(R)=𝟙⟺𝟙⊑RR°`
`R` simple ⟺`R°R⊑𝟙`
`R` a map ⟺`R` entire and simple
`R,S` entire ⟹`RS` entire — likewise simple, likewise maps ,
`RS` entire ⟹`R` entire

§5.1. Sliding the discard

`Dom(RS)⊑Dom(R)`, and a single glyph for `Dom` would have nothing to slide: with the box and the discard drawn apart, the law is one dot walking back along the lower strand.

(5.1a)



S⊸⊑⊸, the lax axiom for ⊸ in the first section
— the discard slides back past S

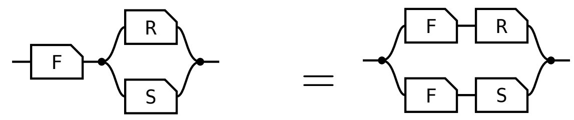
Equality is `S` entire, which is the same picture read as `Dom(R)=𝟙⟺R` entire.

§6. Maps

(6a)

surjective $\mathbb{1} \sqsubseteq R^\circ R$, that is, R° entire.	**single valued** $R^\circ R \sqsubseteq \mathbb{1}$
entire $\mathbb{1} \sqsubseteq R R^\circ$. With **single valued**, a **map**.	**injective** $R R^\circ \sqsubseteq \mathbb{1}$

(6b)

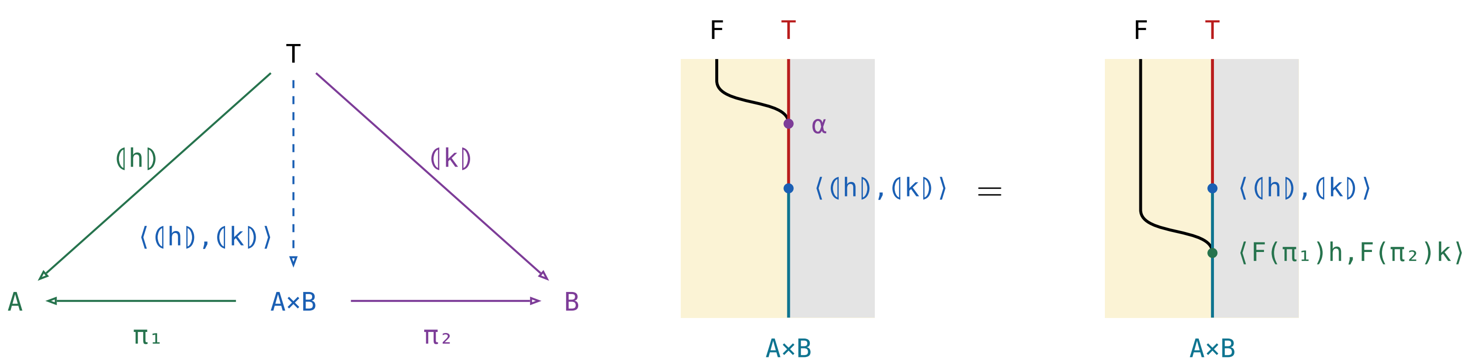
law	picture
$F(R \cap S) = FR \cap FS$ F single valued	 one person who admires both a at A, b at B

§7. Reduce in $\mathcal{S}et$

(7a)

definition	type	note
`listr A::=nil|cons (A,listr A)`	$\mathcal{S}et \to \mathcal{S}et$	The cons-lists over `A`, the datatype every row below folds.
$\mathtt{sum} \triangleq (\![\mathtt{zero}, \mathtt{plus}]\!)$	`listr Nat→Nat`	`plus(a,b)=a+b`.
$\mathtt{length} \triangleq (\![\mathtt{zero}, \pi_2\ \mathtt{succ}]\!)$	`listr A→Nat`	π_2 drops the head and keeps the count of the tail, `succ` adds one for the head.
$\mathtt{average} \triangleq \langle \mathtt{sum}, \mathtt{length} \rangle\ \mathtt{div}$	`listr Nat→Real`	`div(m,n)=m/n`, with `div(0,0)=0` so `average` is total. Traverses the list twice.
banana-split law $\langle (\!\|h\|\!), (\!\|k\|\!) \rangle = (\!\|\langle F(\pi_1)h, F(\pi_2)k \rangle\|\!)$	$T \to A \times B$	Any fork of folds is a single fold, hence one traversal — F the base functor.
what it reduces to $\alpha \langle (\!\|h\|\!), (\!\|k\|\!) \rangle = F(\langle (\!\|h\|\!), (\!\|k\|\!) \rangle) \langle F(\pi_1)h, F(\pi_2)k \rangle$	$FT \to A \times B$	All that $\triangleq$ `reduce` leaves to check: the fork satisfies the defining equation.
the instance $\langle \mathtt{sum}, \mathtt{length} \rangle = (\![\mathtt{zeros}, \mathtt{pluss}]\!)$	`listr Nat→Nat×Nat`	`pluss(a,(b,n))=(a+b,n+1)`, so `average` runs in one pass.
`preds n=[n,n−1,…,1]`	`Nat→[Nat]`	apply the earlier special case to write `preds` as $(\!\|k\|\!)\pi_1$.

(7b)



$\langle (\!\|h\|\!), (\!\|k\|\!) \rangle = (\!\|\langle F(\pi_1)h, F(\pi_2)k \rangle\|\!)$ banana split

§7.1. Fokkinga's mutual recursion theorem

- $\mathsf{F\text{-}Alg}(\mathcal{A})$ — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful $U : \mathsf{F\text{-}Alg}(\mathcal{A}) \to \mathcal{A}$ **creates** them: the product of $h : FA \to A$ and $k : FB \to B$ is carried by $A \times B$, with structure $\langle F(\pi_1)h, F(\pi_2)k \rangle : F(A \times B) \to A \times B$.
- π_1 and π_2 are homomorphisms out of it, and $\langle p,q \rangle : X \to A \times B$ is a homomorphism **iff** p and q both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (7a) IS that product's universal property read at the initial algebra: $\llparenthesis h \rrparenthesis$ and $\llparenthesis k \rrparenthesis$ are the unique homomorphisms to (A,h) and (B,k), so their fork is the unique homomorphism into the product, which is $\llparenthesis \langle F(\pi_1)h, F(\pi_2)k \rangle \rrparenthesis$.
- Fokkinga's theorem is **strictly more general** and is not that product: in mutual recursion the two arrows leave $F(A \times B)$, not FA and FB, so each sees BOTH components. The product is the case where they factor as $F(\pi_1)h$ and $F(\pi_2)k$. What it says: **any algebra on a product carrier is folded by a fork**.

$$\begin{tikzcd} FT \arrow[r, "\alpha"] \arrow[d, "F(\langle f,g \rangle)"'] & T \arrow[d, dashed, "f"] \\ F(A \times B) \arrow[r, "h"'] & A \end{tikzcd} \qquad \begin{tikzcd} FT \arrow[r, "\alpha"] \arrow[d, "F(\langle f,g \rangle)"'] & T \arrow[d, dashed, "g"] \\ F(A \times B) \arrow[r, "k"'] & B \end{tikzcd}$$

$$\alpha f = F(\langle f,g \rangle)h \wedge \alpha g = F(\langle f,g \rangle)k \equiv \langle f,g \rangle = \llparenthesis \langle h,k \rangle \rrparenthesis \tag{7.1a}$$

§7.2. Ruby triangles

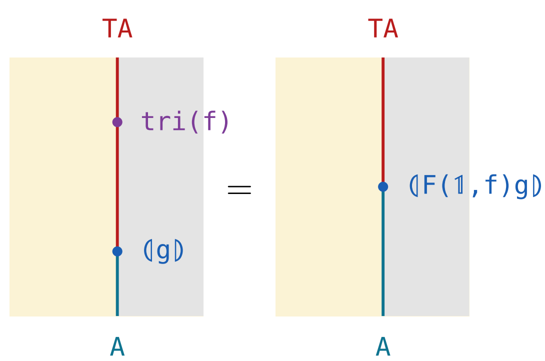
(7.2a)

stage	definition
informally	$\mathrm{tri}(f)\ [a_0, a_1, \ldots, a_i, \ldots, a_n] = [a_0, f\ a_1, \ldots, f^i\ a_i, \ldots, f^n\ a_n]$
for cons-lists	$\mathrm{tri}(f) = \llparenthesis [\mathrm{nil}, (\mathbb{1} \times \mathrm{listr}(f))\ \mathrm{cons}] \rrparenthesis$
with the base functor named	$F(A,B) = 1 + A \times B$, $\alpha = [\mathrm{nil}, \mathrm{cons}]$, so $\mathrm{tri}(f) = \llparenthesis F(\mathbb{1}, \mathrm{listr}(f))\alpha \rrparenthesis$
abstractly	F a bifunctor with initial type (α, T): $\mathrm{tri}(f) = \llparenthesis F(\mathbb{1}, T(f))\alpha \rrparenthesis$

For the definition to make sense $f : A \to A$ is required, and then $\mathrm{tri}(f) : TA \to TA$.

(7.2b)

$$\begin{tikzcd} TA \arrow[rr, "\mathrm{tri}(f)"] \arrow[dr, dashed, "\llparenthesis F(\mathbb{1},f)g \rrparenthesis"'] & & TA \arrow[dl, dashed, "\llparenthesis g \rrparenthesis"] \\ & A & \end{tikzcd}$$



$$\mathrm{tri}(f)\ \llparenthesis g \rrparenthesis = \llparenthesis F(\mathbb{1},f)g \rrparenthesis \quad \Longleftarrow \quad gf = F(f,f)g$$

Horner's rule is Horner's rule, generalised from cons-lists to any initial type. Fusion reduces it to $F(\mathbb{1}, T(f))\alpha \llparenthesis g \rrparenthesis = F(\mathbb{1}, \llparenthesis g \rrparenthesis)F(\mathbb{1},f)g$.

§7.3. Depth of a tree

(7.3a)

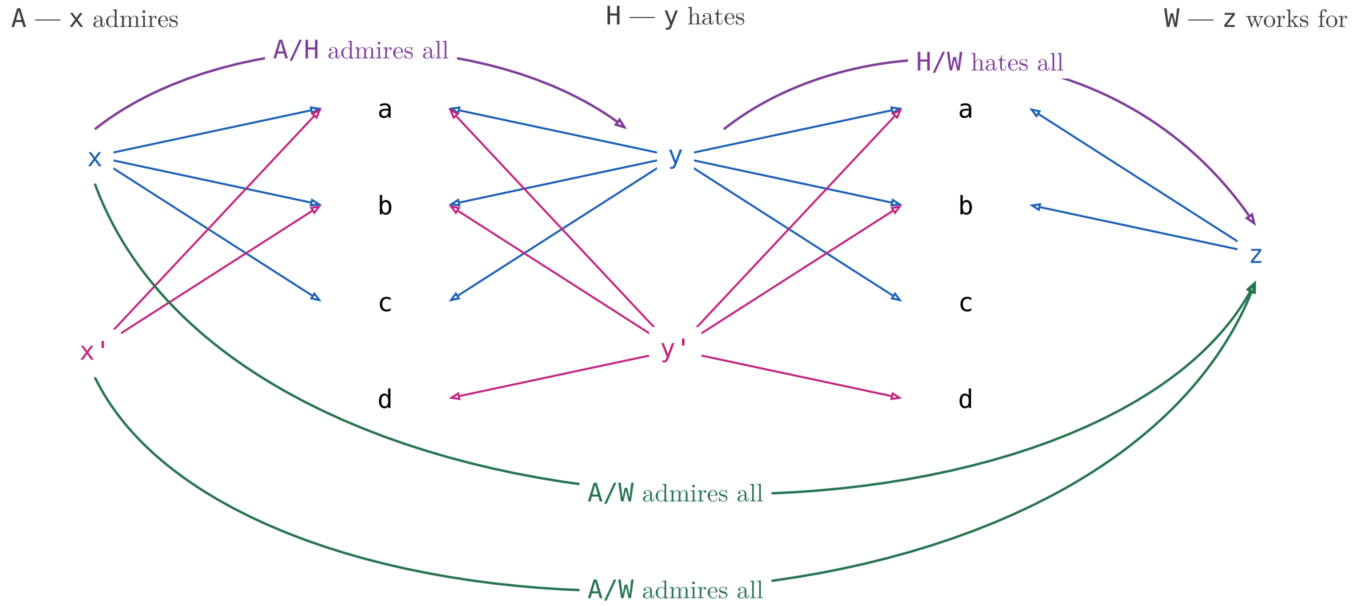
the datatype	`tree A::=tip A|bin (tree A,tree A)`, base functor `F(A,B)=A+B×B`, initial type `([tip,bin],tree)`
the fold	`⦇[g,h]⦈` is the unique `f` with `f (tip a)=g a` and `f (bin (x,y))=h (f x,f y)`
`tree(f)`	`tree(f)=⦇F(f,𝟙) [tip,bin]⦈`; pointwise `tree(f) (tip a)=tip (f a)` and `tree(f) (bin (x,y))=bin (tree(f) x,tree(f) y)`
`max`	`max=⦇[𝟙,bmax]⦈`, where `bmax (a,b)` is the larger of `a` and `b`
`depths`	`depths=tree(zero) tri(succ)` — replaces every tip by its depth in the tree, `zero` the constant function returning 0 and `succ` the successor
`depth`	`depth=depths max`

§8. / is all of

```
x (R/S) y⟺∀p. y S p→x R p
x (S\R) y⟺∀p. p S x→p R y
/compares images: S(y)⊆R(x).   \ compares preimages: S°(x)⊆R°(y).
example: A admires,H hates,W works for
x (A/H) y — x admires everyone y hates.
x (H\A) y — everyone who hates x admires y.
```
(8a)

§8.1. (R/S)(S/W)⊑R/W

(8.1a)

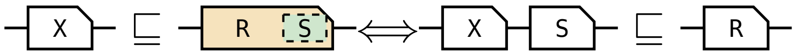


x (A/H) y — x admires everyone y hates
y (H/W) z — y hates everyone z works for
x (A/W) z — x admires everyone z works for (8.1b)

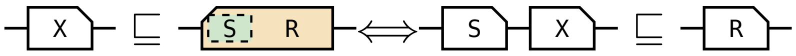
(A/H)(H/W) is a path: x → y → z, and that is all of it. A/W also holds of x', who admires everyone z works for — but x' does not admire everyone anybody hates, so nothing composes to it. The missing path is exactly the strictness of (R/S)(S/W)⊑R/W.

(8.1c)

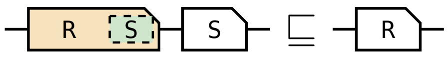
$X \sqsubseteq R/S \iff XS \sqsubseteq R$
X is any x-to-y pairing; one that only pairs x with a y such that x admires everyone y hates lies inside R/S, and R/S is the largest such.



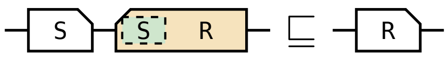
$X \sqsubseteq S\backslash R \iff SX \sqsubseteq R$
The mirror — divide on the left when x comes first.



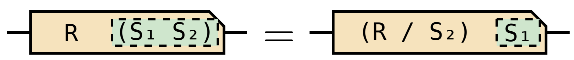
$(R/S)S \sqsubseteq R$
There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S=\varnothing$: R/S is everyone, $(R/S)S=\varnothing$.



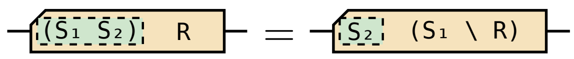
$S\ (S\backslash R) \sqsubseteq R$
The mirror.



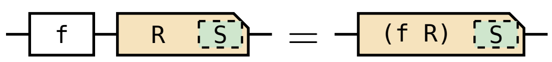
associate: $R/(S_1S_2)=(R/S_2)/S_1$
A friend's enemy is two hops: divide by the far end first.



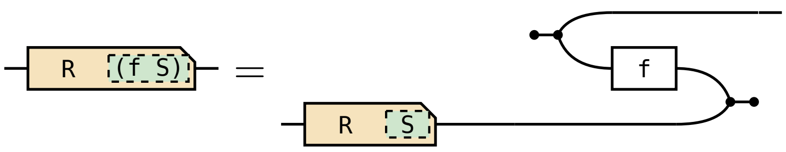
$(S_1S_2)\backslash R=S_2\backslash(S_1\backslash R)$
The mirror.



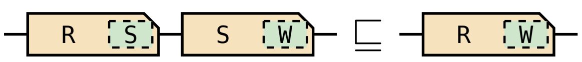
maps: $f\ (R/S)=(fR)/S$
Rename x before or after dividing — the licence to write fR/S.



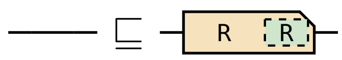
$R/(fS)=(R/S)f°$
Rename y: a map leaves a denominator as $f°$ outside the box.



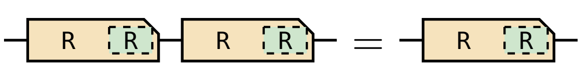
$(R/S)(S/W) \sqsubseteq R/W$
Someone who admires all of a hate-set that already covers everyone z works for admires those people too.



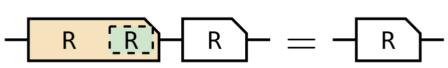
$\mathbb{1} \sqsubseteq R/R$
R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.



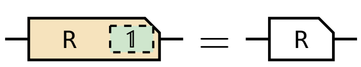
$(R/R)(R/R)=R/R$
R/R is the preorder **admires at least as much as**, and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $\mathbb{1} \sqsubseteq R/R$ above it is an equality.



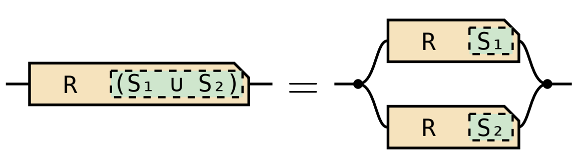
$(R/R)R=R$
Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.



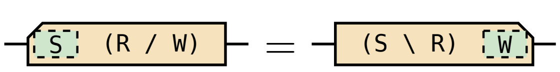
$R/\mathbb{1}=R$
Dividing by $\mathbb{1}$: p's set is just $\{p\}$, so admiring all of it is admiring p.



$R/(S_1 \cup S_2)=R/S_1 \cap R/S_2$
Admiring a combined hate-set is admiring each set in full.



$S\backslash(R/W)=(S\backslash R)/W$
Which is why $S\backslash R/W$ needs no bracket.



Fifteen laws, fifteen pictures, and not one shows a generator: ∩, ∪, ° and composition are what the Frobenius generators build, and / is none of those — it is posited, with nothing to unfold.

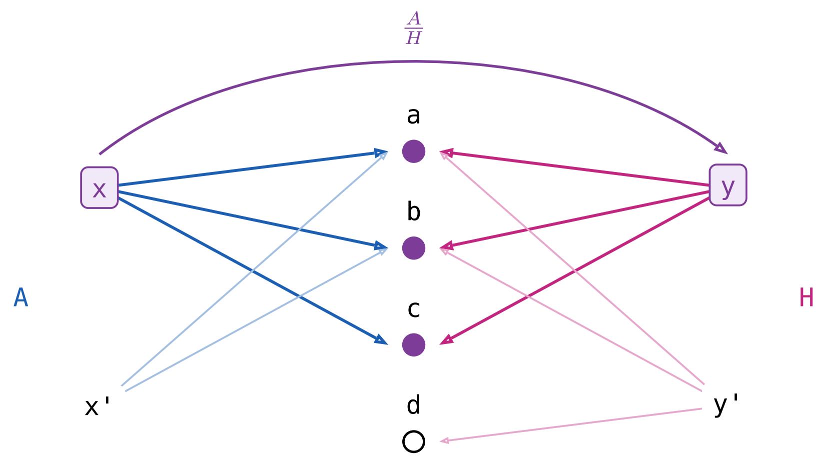
§9. $\frac{R}{S}$

$\frac{R}{S}$ `≜(R/S)∩(S/R)°` . In `Rel` `x` and `y` has the same image: `∀p. (x R p⟺y S p)` (9a)

`x (A/H) y` — `x` admires everyone `y` hates (9b)
`x ((H/A)°) y` — `x` admires only people `y` hates
`x ((A/H)∩(H/A)°) y` — `x` admires only and all whom `y` hates

`x` admires exactly whom `y` hates: `/` is **all of**, $\frac{R}{S}$ is **only all of**.

(9c)



(9d)

law	the reading
$X \sqsubseteq \frac{R}{S} \iff XS \sqsubseteq R$ and `X°R⊑S`	`X` may pair `x` with `y` only when `x` admires exactly whom `y` hates. Both halves must typecheck, so the operation is **partial**.
$\left(\frac{R}{S}\right)^{\circ} = \frac{S}{R}$	Matching is symmetric.
$\frac{R}{S}\frac{S}{W} \sqsubseteq \frac{R}{W}$	And transitive.
$\frac{R}{S}S \sqsubseteq R$	$\left(\exists y.\, x\left(\frac{R}{S}\right)y \wedge ySp\right) \to xRp$ `x only admires whom y hates` $\frac{R}{S}S = \mathrm{Dom}\left(\frac{R}{S}\right)R$

$\frac{R}{R}R = R$	$\left(\exists y.\, x\left(\frac{R}{R}\right)y \wedge yRp\right) \iff xRp$
	`x and y admire the same people`
	`y=x always qualifies (𝟙⊑R%R below)`

$\mathbb{1} \sqsubseteq \frac{R}{R}$	$x\left(\frac{R}{R}\right)y$ if `x` and `y` admires the same peoples.
$\left(\frac{R}{R}\right)^2 = \frac{R}{R}$	So the relation **admires the same people** is an equivalence relation.
$X \sqsubseteq \frac{R}{R} \iff XR \sqsubseteq R$, for symmetric `X`	The largest symmetric arrow that leaves `R` alone.
$\frac{R}{\mathbb{1}}$ is the **simple part** of `R`	The people who admire exactly one person and nobody else. It equals `R` only when `R` is simple, unlike `R/𝟙=R`.
`Dom` $\frac{R}{S}$ `=𝟙∩(R/S)(S/R)`	Its domain is the **domain of simplicity** of `R`.

§10. Freyd's Power allegories

(10a)

A **power allegory** is a division allegory with one unary operation on arrows, ∋ **epsiloff**, subject to

$$\ni_R\Box = R\Box,\quad \ni_R = \ni_{R\,\Box}$$
$$\mathbb{1} \sqsubseteq (R/\ni_R)(\ni_R/R) \qquad \ni_R \text{ is } \textbf{thick}$$
$$\frac{\ni_R}{\ni_R} = \mathbb{1} \qquad \ni_R \text{ is } \textbf{straight}$$

R□ is R's target, an identity arrow. For `R : A→B` write `∋ : EB→B`, dropping the subscript.

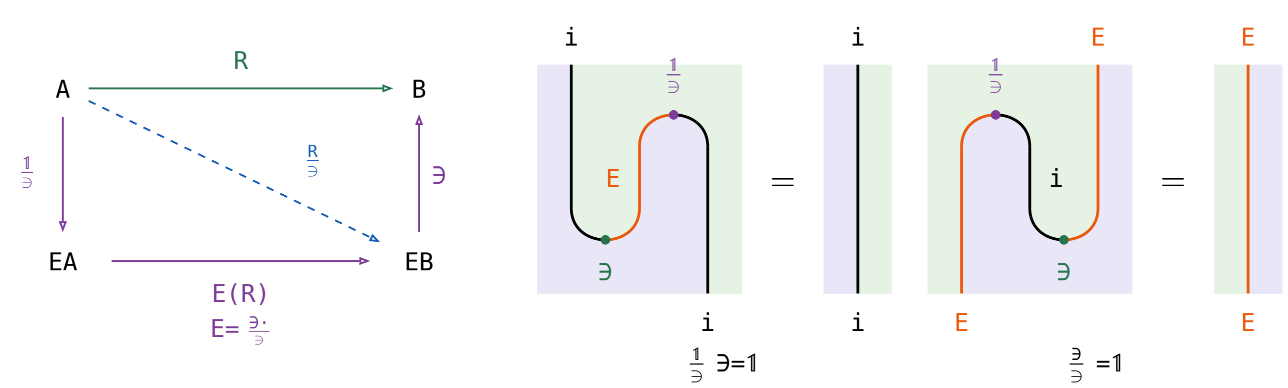
`∈≜∋° : A→EA`

(10b)

law	the reading
1 $\ni_R\Box = R\Box,\ \ni_R = \ni_{R\,\Box}$	One ∋ per object, not per arrow.
2 ∋ is **thick**	**Comprehension**: every `x` has a set of exactly the people `x` admires.
3 $\frac{R}{\ni}$ `: A→EB`, for `R : A→B`	convert a relation to a function. `a` $\frac{R}{\ni}$ `={b|a R b}`
4 $\frac{R}{\ni}$ is a **map**	
5 $\frac{R}{\ni}$ `∋=R`	
6 `F⊑` $\frac{F\ni}{\ni}$, `F` simple	A partial choice of sets is inside the total one.
7 $\frac{1}{\ni}$, the **singleton map**, monic	The one-person set.
8 $\frac{\ni}{\ni}$ `=𝟙`	Make the set of a set, then read it back one level down.
9 **fusion:** $\frac{fR}{\ni}$ `=f` $\frac{R}{\ni}$, `f` a map	Naturality of the unit, `f` $\frac{1}{\ni}$ `=` $\frac{1}{\ni}$ `(E(f))`.
10 $\frac{f}{\ni}$ `=f` $\frac{1}{\ni}$, `f` a map	Rename first or take singletons first — the fusion row above at `R=𝟙`.
11 $\frac{R}{S} = \frac{R}{\ni}\left(\frac{S}{\ni}\right)^{\circ}$	`x` and `y` match when `R` sends `x` and `S` sends `y` to the same set.
12 `E(R)≜` $\frac{\ni R}{\ni}$, `E≜` $\frac{\ni\cdot}{\ni}$	`E(R): EA→EB`, image of a set of A
13 $\frac{R}{\ni}$ `=` $\frac{1}{\ni}$ `E(R)`	$\frac{1}{\ni}$`: x↦{x}`
14 $\frac{S}{\ni}$ `E(R)=` $\frac{S}{\ni}\frac{\ni R}{\ni}$ `=` $\frac{SR}{\ni}$	absorption — the monad's composition law, $\frac{S}{\ni}$ `∘` $\frac{R}{\ni}$ `=` $\frac{SR}{\ni}$, §10.2
15 `subset≜∋/∋ : EA→EA`	`xs subset ys⟺∀a. ys∋a→xs∋a`, that is `ys⊆xs`, not `xs⊆ys`.

§10.1. i⊣E Power Allegory defined as adjunction

(10.1a)



$\frac{R}{\ni}$ `∋=R` EA is the powerset of A — standard mathematics, but here P is already the relator P(R).

`i` is the inclusion Map(𝒜)→𝒜, doing nothing, so `E` is both the functor and the monad `iE`.

§10.2. $\mathcal{A}\cong\mathtt{Kleisli(E)}$

$$\mathtt{E(R)}\triangleq\frac{\ni\mathtt{R}}{\ni},\ \frac{\mathbb{1}}{\ni}\ :\ \mathtt{A}\to\mathtt{EA},\quad \mathtt{union}=\mathtt{E}(\ni)=\frac{\ni\ni}{\ni}\ :\ \mathtt{E(EA)}\to\mathtt{EA} \qquad \text{(10.2a)}$$

$$\mathtt{f}\diamond\mathtt{g}\triangleq\mathtt{f\ E(g)\ union}\ :\ \mathtt{A}\to\mathtt{EC},\quad \text{for } \mathtt{f}\ :\ \mathtt{A}\to\mathtt{EB} \text{ and } \mathtt{g}\ :\ \mathtt{B}\to\mathtt{EC}$$

the monad is on $\mathtt{Map}(\mathcal{A})$, not on the allegory: $\mathtt{E}$ is a relator on all relations, but $\frac{\mathbb{1}}{\ni}$, $\mathtt{union}$ and $\mathtt{f\ \ E(g)\ \ union}$ are maps, and the Kleisli construction happens where they live.

$$\boxed{\frac{\mathtt{S}}{\ni}\diamond\frac{\mathtt{R}}{\ni}}\ \underset{\text{definition of }\diamond,\ \mathtt{union}=\mathtt{E}(\ni)}{=\!=\!=}\ \boxed{\frac{\mathtt{S}}{\ni}\ \mathtt{E}\left(\frac{\mathtt{R}}{\ni}\right)\mathtt{E}(\ni)}\ \underset{\mathtt{E}\text{ a functor, }\mathtt{E(X)E(Y)=E(XY)}}{=\!=\!=}\ \boxed{\frac{\mathtt{S}}{\ni}\ \mathtt{E}\left(\frac{\mathtt{R}}{\ni}\ni\right)} \qquad \text{(10.2b)}$$

$$\underset{\text{(10b), }\frac{\mathtt{R}}{\ni}\ \ni=\mathtt{R}}{=\!=\!=}\ \boxed{\frac{\mathtt{S}}{\ni}\ \mathtt{E(R)}}\ \underset{\text{(10b), absorption}}{=\!=\!=}\ \boxed{\frac{\mathtt{SR}}{\ni}}$$

the first three steps only unfold $\diamond$ — $\mathtt{E}$ is a functor, so $\mathtt{E}(\frac{\mathtt{R}}{\ni})\mathtt{union}=\mathtt{E}(\frac{\mathtt{R}}{\ni}\ni)=\mathtt{E(R)}$ and the $\mathtt{union}$ is gone. Absorption, the row of (10b), is the whole law, and it is the functoriality of $\frac{\cdot}{\ni}$.

$\mathbb{1}$ goes to $\frac{\mathbb{1}}{\ni}$, the Kleisli identity, by definition — so $\frac{\cdot}{\ni}$ is an isomorphism of categories $\mathcal{A}\cong\mathtt{Kleisli(E)}$, and §10.1′s $\mathtt{i}\dashv\mathtt{E}$ is the Kleisli adjunction. $\mathtt{i}$ is the identity on objects, so every object of the allegory is free.

§11. Relator

(11a)

Every hom-set of an allegory is a poset, so an allegory is a **locally posetal 2-category**: the 2-cell from $\mathtt{R}$ to $\mathtt{S}$ IS $\mathtt{R}\sqsubseteq\mathtt{S}$. A **relator** $\mathtt{F}\ :\ \mathcal{C}\to\mathcal{D}$ is a 2-functor between allegories:

$$\mathtt{F}(\mathbb{1})=\mathbb{1}\qquad \mathtt{F(RS)=F(R)F(S)}\qquad \mathtt{R}\sqsubseteq\mathtt{S}\Longrightarrow\mathtt{F(R)}\sqsubseteq\mathtt{F(S)}$$

Preserving $^\circ$ is **not** asked for — $^\circ$ is an identity-on-objects involution $\mathcal{C}^{op}\to\mathcal{C}$, no part of the 2-category.

(11b)

the statement
For $\mathtt{f}$ a map, $\mathtt{F(f)}$ is a map and $\mathtt{F(f^\circ)=F(f)^\circ}$.
Over a **tabular** allegory a functor is a relator $\Longleftrightarrow$ it preserves $^\circ$.
$\mathtt{F(R^\circ)=F(R)^\circ}$ for every $\mathtt{R}$, so $\mathtt{F(R)^\circ}$ needs no bracket.
Two relators agreeing on maps are equal.
$\mathtt{F(X\cap Y)=F(X)\cap F(Y)}$ for $\mathtt{X,Y}$ coreflexive.
$\mathtt{F(R\cap S)\sqsubseteq F(R)\cap F(S)}$, and strictly.
$\mathtt{F(Dom(R))=Dom(F(R))}$ for $\mathtt{F}$ preserving $^\circ$.

The **power relator** $\mathtt{P}$ — $\mathtt{xs\ P(R)\ ys}\Longleftrightarrow(\forall\mathtt{a}\in\mathtt{xs}.\ \exists\mathtt{b}\in\mathtt{ys}.\ \mathtt{a\ R\ b})\wedge(\forall\mathtt{b}\in\mathtt{ys}.\ \exists\mathtt{a}\in\mathtt{xs}.\ \mathtt{a\ R\ b})$ — is where the fourth is strict: for $\mathtt{R}=\{(\mathtt{a_1,b_1}),(\mathtt{a_2,b_2})\}$ and $\mathtt{S}=\{(\mathtt{a_1,b_2}),(\mathtt{a_2,b_1})\}$ the pair $(\{\mathtt{a_1,a_2}\},\{\mathtt{b_1,b_2}\})$ is in $\mathtt{P(R)\cap P(S)}$, while $\mathtt{R\cap S}=\varnothing$.

§11.1. Fork $\langle\mathtt{R,S}\rangle$

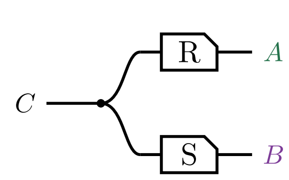
(11.1a)

The **fork** of $\mathtt{R}\ :\ \mathtt{C}\to\mathtt{A}$ and $\mathtt{S}\ :\ \mathtt{C}\to\mathtt{B}$ is $\langle\mathtt{R,S}\rangle\triangleq\mathtt{R}\pi_1{}^\circ\cap\mathtt{S}\pi_2{}^\circ$, where (π_1,π_2) is the tabulation of $\top$.

$$\langle\mathtt{R,S}\rangle\pi_1=\mathtt{Dom(S)R}\qquad\qquad \langle\mathtt{R,S}\rangle\pi_2=\mathtt{Dom(R)S} \qquad \text{(11.1b)}$$

(11.1c)

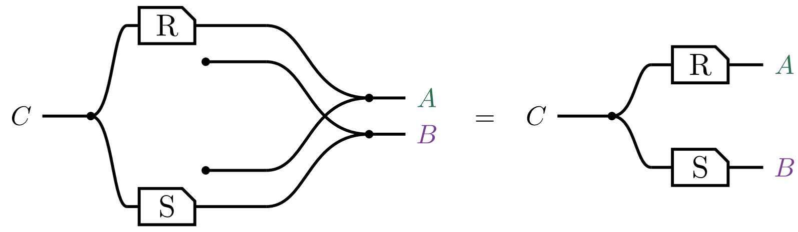
$$\begin{tikzcd}
 & A & \\
C \arrow[ur, "\mathtt{R}"] \arrow[rr, dashed, "\langle R,S\rangle"] \arrow[dr, "\mathtt{S}"'] & & A\times B \arrow[ul, "\pi_1"'] \arrow[dl, "\pi_2"] \\
 & B &
\end{tikzcd}$$



A domain is coreflexive, so $\langle R,S\rangle\pi_1\sqsubseteq R$, with equality exactly when S is entire; for maps both triangles commute and $\langle f,g\rangle$ is unique. In Rel, c $\langle R,S\rangle$ (a,b) iff c R a and c S b — copy c, then R on one strand and S on the other, which is $\lhd(R\otimes S)$ on the right.

No $^\circ$ survives the translation. $\pi_1=\mathbb{1}\otimes\multimap$ discards the second component, so $\pi_1{}^\circ=\mathbb{1}\otimes\multimapinv$ **creates** one out of nothing, and ∩ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:

(11.1d)



$\multimap\!\rhd=\mathbb{1}$ on each half

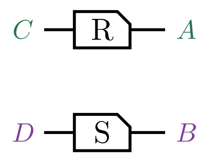
§11.1.1. Relational product R×S

(11.1.1a)

> $R\times S\triangleq\langle\pi_1 R,\pi_2 S\rangle$, a relator in each argument but no longer a categorical product.

(11.1.1b)

$$\begin{tikzcd}
C \arrow[rr, "R"] & & A \\
C\times D \arrow[u, "\pi_1"] \arrow[rr, dashed, "R\times S"] \arrow[d, "\pi_2"'] \arrow[urr, phantom, "\sqsupseteq"] \arrow[drr, phantom, "\sqsupseteq"] & & A\times B \arrow[u, "\pi_1"'] \arrow[d, "\pi_2"] \\
D \arrow[rr, "S"] & & B
\end{tikzcd}$$

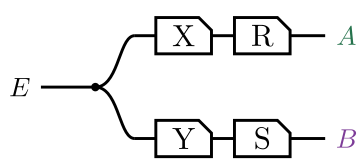


Right-then-up is $(R\times S)\pi_1$, up-then-right is $\pi_1 R$, and $(R\times S)\pi_1\sqsubseteq\pi_1 R$, equality when S is entire. In Rel, (c,d) (R×S) (a,b) iff c R a and d S b — two strands side by side, no copy dot: $R\times S=R\otimes S$.

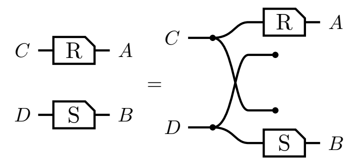
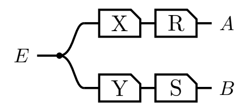
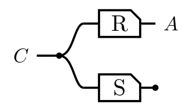
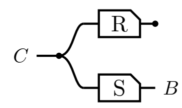
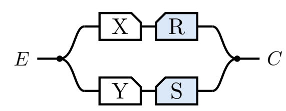
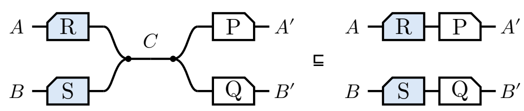
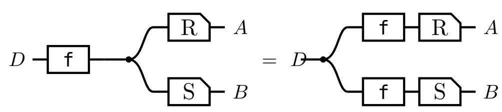
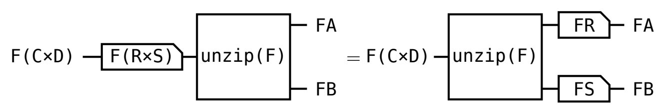
§11.1.2. Absorption

For X : E→C and Y : E→D, $\langle X,Y\rangle(R\times S)=\langle XR,YS\rangle$. Both sides are this picture:

(11.1.2a)

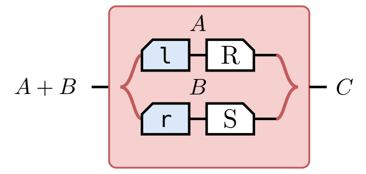
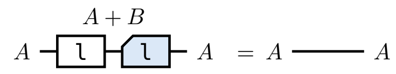
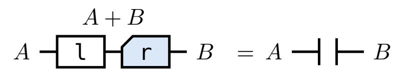
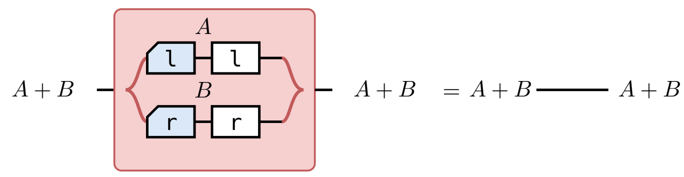


(11.1.2b)

	the law	picture
1	`R×S=⟨π₁R,π₂S⟩`	
2	`⟨X,Y⟩(R×S)=⟨XR,YS⟩` both sides are the same strokes — absorption. Its `⊑` half is that half at `S:=𝟙` and at `R:=𝟙` — stages of their proof of this row; the corollary is the `(R×S)(U×V)=(RU)×(SV)` it yields, at `R:=𝟙` and `V:=𝟙`.	
3	`⟨R,S⟩π₁=Dom(S)R` (11.1b)	
4	`⟨R,S⟩π₂=Dom(R)S` (11.1b)	
5	`⟨X,Y⟩⟨R,S⟩°=(XR°)∩(YS°)`	
6	`⟨R,S⟩°⟨P,Q⟩⊑(R°P)×(S°Q)`	
7	`f⟨R,S⟩=⟨fR,fS⟩` `f` a map; it fails for an arbitrary arrow;	
8	`F(R×S)unzip(F)=unzip(F)(F(R)×F(S))` `unzip(F)≜⟨F(π₁),F(π₂)⟩`, a map	
9	`g=curry(f)⟺(g×𝟙)eval=f` reading `×` as the relational product, does `Rel` have exponentials?	

§11.2. Coproduct `[R,S] : A+B→C`

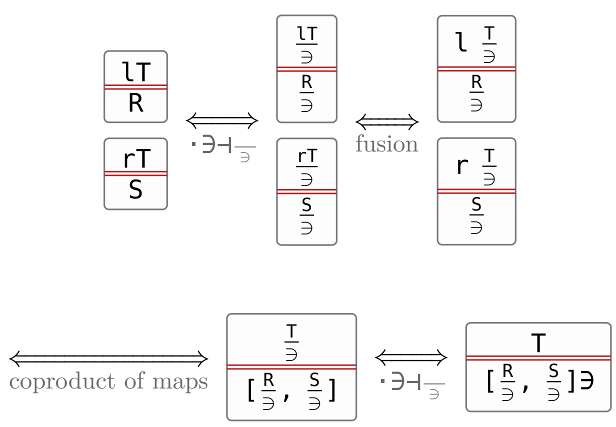
(11.2a)

the statement	**picture**
`[R,S]≜l°R ∪ r°S` The tape is the union — a particle entering at `A+B` takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.	
`[R,S]=[`$\frac{\mathsf{R}}{\ni}$`, `$\frac{\mathsf{S}}{\ni}$`]∋`	
`R+S≜[Rl,Sr]`	
`l[R,S]=R`, `r[R,S]=S`, and `[R,S]` is the only such arrow ,	
`ll°=𝟙=rr°`	
`lr°=⊥=rl°`	
`l°l ∪ r°r=𝟙`	
`[U,V]°[R,S]=U°R ∪ V°S`	

§11.2.1. `[R,S]≜[`$\frac{\mathsf{R}}{\ni}$`, `$\frac{\mathsf{S}}{\ni}$`]∋`

The universal-property row is not free: `l,r` were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because $\frac{}{\ni}$ sends an arrow `A→C` to a map `A→EC` reversibly, so the map coproduct can be applied underneath it. For any `T : A+B→C`,

(11.2.1a)

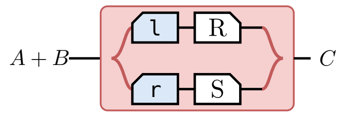
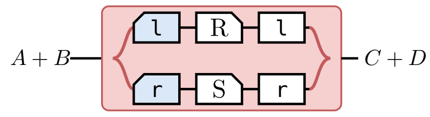
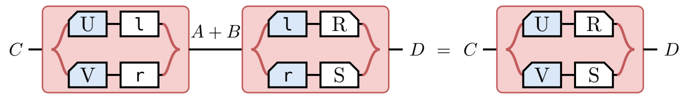
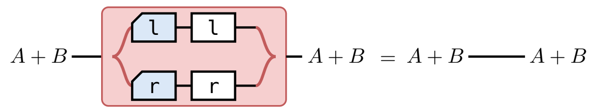


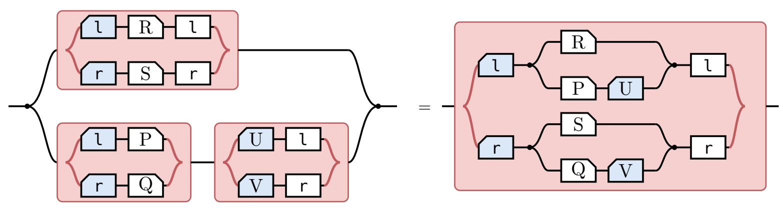
(11.2.1b)

$$
\begin{tikzcd}
& A \arrow[dl, "\mathsf{l}"'] \arrow[dr, "\frac{\mathsf{R}}{\ni}"] \arrow[drr, bend left, "\mathsf{R}"] & & \\
A+B \arrow[rr, dashed, "{[\frac{\mathsf{R}}{\ni},\, \frac{\mathsf{S}}{\ni}]}"] & & EC \arrow[r, "\ni"] & C \\
& B \arrow[ul, "\mathsf{r}"] \arrow[ur, "\frac{\mathsf{S}}{\ni}"'] \arrow[urr, bend right, "\mathsf{S}"'] & &
\end{tikzcd}
$$

Nothing here holds only up to `⊑`: every triangle commutes on the nose, which is the difference from the fork above. The border spells `[R,S]=[`$\frac{\mathsf{R}}{\ni}$`, `$\frac{\mathsf{S}}{\ni}$`]∋`, and pushing `∋` into the union that `[·,·]` on maps already is turns that back into the definition, `(l°`$\frac{\mathsf{R}}{\ni}$` ∪r° `$\frac{\mathsf{S}}{\ni}$`)∋=l°R ∪ r°S`.

(11.2.1c)

the law	**picture**
`[R,S]=(l°R) ∪ (r°S)` (11.2a)'s first row	
`R+S=[Rl,Sr]`	
`[U,V]°[R,S]=(U°R) ∪ (V°S)` (11.2a)'s last row;	
`X≜[𝟙,⊥]=l°` and `Y≜[⊥,𝟙]=r°`, so `(Xl) ∪ (Yr)=[l,r]=𝟙` which is (5.9)	
prove (5.11), and say why duality does not carry it over from the product law	

`(R+S)∩([P,Q][U,V]°)`
`=(R∩(PU°))+(S∩(QV°))`
`[P,Q][U,V]°` is a full 2×2 of composites; `R+S` is diagonal, so the meet cuts the two off-diagonal branches
	

§11.3. The power relator `P(R)`

(11.3a)

For `R : A→B`,

`P(R)≜ ((∋R)/∋)∩((∋R°)/∋)° : EA→EB`

`E(R)≜` $\frac{\ni R}{\ni}$ `= ((∋R)/∋)∩(∋/(∋R))°`

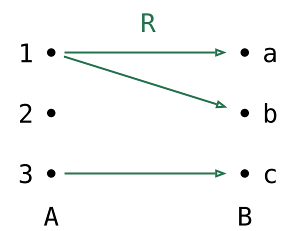
`xs P(R) ys⟺(∀a∈xs. ∃b∈ys. a R b)∧(∀b∈ys. ∃a∈xs. a R b)`

Every element of `xs` is related by `R` to some element of `ys`, and conversely.

(11.3b)

	in Rel	in words	im(xs)	partner
`(∋R)/∋`	`∀y∈ys. ∃x∈xs. x R y`	`∀y. some xs R y`	`ys⊆im(xs)`	every `y` has a partner
`(∋/(∋R))°`	`∀y. (∃x∈xs. x R y)→y∈ys`	every `y` with `some xs R y` is in `ys`	`ys⊇im(xs)`	`ys` leaves out no partner
`E(R)`	`ys={y∣∃x∈xs. x R y}`	`ys` = every `y` with `some xs R y`	`ys=im(xs)`	every `y` has a partner, and none is left out
`((∋R°)/∋)°`	`∀x∈xs. ∃y∈ys. x R y`	`∀x. x R some ys`	—	every `x` has a partner
`P(R)`	`∀y∈ys. ∃x∈xs. x R y` and `∀x∈xs. ∃y∈ys. x R y`	`∀y. some xs R y` and `∀x. x R some ys`	—	every `x` and every `y` has a partner

(11.3c)



xs	**(∋R)/∋**	**(∋/(∋R))°**	**E(R)**	**((∋R°)/∋)°**	**P(R)**
`∅`	`∅`	all 8 subsets of `abc`	`∅`	all 8 subsets of `abc`	`∅`
`{1}`	`∅`, `a`, `b`, `ab`	`ab`, `abc`	`ab`	`a`, `b`, `ab`, `ac`, `bc`, `abc`	`a`, `b`, `ab`
`{2}`	`∅`	all 8 subsets of `abc`	`∅`	none	none
`{3}`	`∅`, `c`	`c`, `ac`, `bc`, `abc`	`c`	`c`, `ac`, `bc`, `abc`	`c`
`{1,2}`	`∅`, `a`, `b`, `ab`	`ab`, `abc`	`ab`	none	none
`{1,3}`	all 8 subsets of `abc`	`abc`	`abc`	`ac`, `bc`, `abc`	`ac`, `bc`, `abc`
`{2,3}`	`∅`, `c`	`c`, `ac`, `bc`, `abc`	`c`	none	none
`{1,2,3}`	all 8 subsets of `abc`	`abc`	`abc`	none	none

(11.3d)

law	the reading
`X⊑P(R)⟺X∋⊑∋R` and `X°∋⊑∋R°`	One containment, and the same one at `R°` — which is the definition read off the two divisions. Hence `P(R°)=P(R)°`, and `R⊑S⟹P(R)⊑P(S)` .
`P(𝟙)=` $\frac{\ni}{\ni}$ `=𝟙`	The straightness axiom verbatim: extensionality **is** `P`'s unit law.
`P(f)=` $\frac{\ni f}{\ni}$, for `f` a map	In `Rel`, `xs P(f) ys⟺ys={f a∣a∈xs}`. The half at `f°` says every `a∈xs` has its `f a` on `ys`; `f` has just the one image per `a`, so that already says `ys` contains everything `xs` reaches, which is the fraction's second half. For a map the two definitions coincide.
`P(RS)=P(R)P(S)`	`⊒` is the division cancellation laws. `⊑` is the one law in this section that is not a calculation: it needs a tabulation of `P(RS)`.

§11.4. Initial algebra

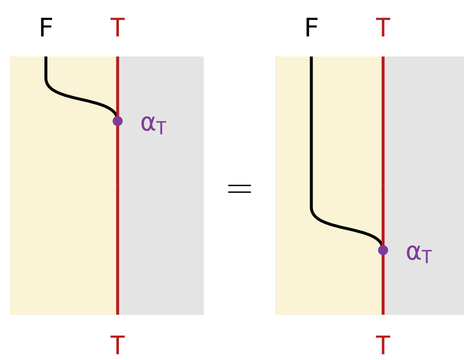
An **F-algebra** on A is a map $\alpha_\mathsf{A} : \mathsf{FA}{\to}\mathsf{A}$. An **F-homomorphism** from α_A to α_B is a map $\mathsf{h} : \mathsf{A}{\to}\mathsf{B}$ with $\alpha_\mathsf{A}\ \mathsf{h}{=}\mathsf{F(h)}\alpha_\mathsf{B}$. The **initial algebra** $\alpha_\mathsf{T} : \mathsf{FT}{\to}\mathsf{T}$ is the F-algebra with exactly one F-homomorphism $\llparenthesis\alpha_\mathsf{A}\rrparenthesis : \mathsf{T}{\to}\mathsf{A}$ to every F-algebra α_A . (11.4a)

$$\begin{tikzcd}
\mathsf{FA} \arrow[r, "\alpha_\mathsf{A}"] \arrow[d, "\mathsf{F(h)}"'] & \mathsf{A} \arrow[d, "\mathsf{h}"] \\
\mathsf{FB} \arrow[r, "\alpha_\mathsf{B}"'] & \mathsf{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
\mathsf{FT} \arrow[r, "\alpha_\mathsf{T}"] \arrow[d, "\mathsf{F}(\llparenthesis\alpha_\mathsf{A}\rrparenthesis)"'] & \mathsf{T} \arrow[d, dashed, "\llparenthesis\alpha_\mathsf{A}\rrparenthesis"] \\
\mathsf{FA} \arrow[r, "\alpha_\mathsf{A}"'] & \mathsf{A}
\end{tikzcd}$$

§11.4.1. Reflection

(11.4.1a)

$$\begin{tikzcd}
\mathsf{FT} \arrow[r, "\alpha_\mathsf{T}"] \arrow[d, "\mathsf{F}(\mathbb{1})"'] & \mathsf{T} \arrow[d, dashed, "\mathbb{1}"] \\
\mathsf{FT} \arrow[r, "\alpha_\mathsf{T}"'] & \mathsf{T}
\end{tikzcd}$$



$\llparenthesis\alpha_\mathsf{T}\rrparenthesis{=}\mathbb{1}$ (2.11)

Taking a value apart with α_T and putting it straight back is doing nothing.

§11.4.2. Fusion

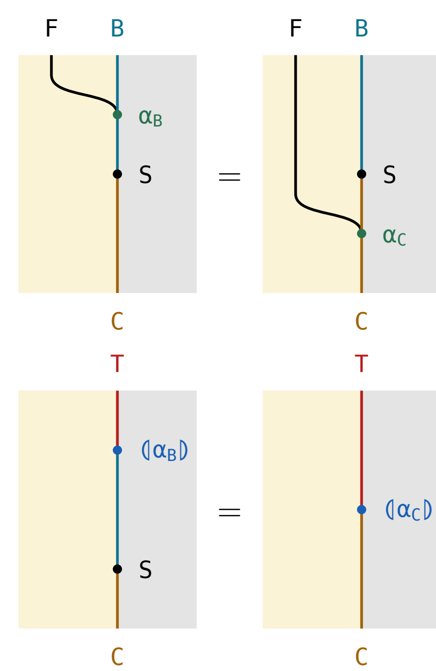
Fusion rewrites $\llparenthesis\alpha_\mathsf{B}\rrparenthesis\mathsf{S}$ through a second algebra $\alpha_\mathsf{C} : \mathsf{FC}{\to}\mathsf{C}$ along an arrow $\mathsf{S} : \mathsf{B}{\to}\mathsf{C}$.

(11.4.2a)

$$\begin{tikzcd}
\mathsf{FT} \arrow[r, "\alpha_\mathsf{T}"] \arrow[d, "\mathsf{F}(\llparenthesis\alpha_\mathsf{B}\rrparenthesis)"'] & \mathsf{T} \arrow[d, dashed, "\llparenthesis\alpha_\mathsf{B}\rrparenthesis"] \\
\mathsf{FB} \arrow[r, "\alpha_\mathsf{B}"] \arrow[d, "\mathsf{F(S)}"'] & \mathsf{B} \arrow[d, "\mathsf{S}"] \\
\mathsf{FC} \arrow[r, "\alpha_\mathsf{C}"] & \mathsf{C}
\end{tikzcd}$$

the side condition

the conclusion



$\llparenthesis\alpha_\mathsf{B}\rrparenthesis\mathsf{S}{=}\llparenthesis\alpha_\mathsf{C}\rrparenthesis \Longleftarrow \alpha_\mathsf{B}\ \mathsf{S}{=}\mathsf{F(S)}\alpha_\mathsf{C}$ (2.12)

§11.5. Type relator

(11.5a)

Let $\mathtt{F}$ be a binary relator with initial type $(\alpha,\mathtt{T})$, so $\mathtt{T}$ is a type functor. $\mathtt{F(R,S)}$ is its action on a pair, and $\mathtt{F(X)}$ abbreviates $\mathtt{F}(\mathbb{1},\mathtt{X})$, the $\mathtt{F}$ of the reduce section. For every object $\mathtt{A}$ the initial algebra is $\alpha : \mathtt{F(A,TA)}\to\mathtt{TA}$, among the maps. $\mathtt{T}$ acts on an arrow $\mathtt{R : A}\to\mathtt{B}$ by

$$\mathtt{T(R)}=(\!|\mathtt{F}(\mathtt{R},\mathbb{1})\alpha|\!) : \mathtt{TA}\to\mathtt{TB}$$

(11.5b)

name	law	what it says
the defining equation	$\mathtt{T(R)}=(\!\lvert\mathtt{F}(\mathtt{R},\mathbb{1})\alpha\rvert\!)$	Rebuild the structure with α, applying $\mathtt{R}$ to the parameter on the way.
functor	$\mathtt{T}(\mathbb{1})=\mathbb{1}$ and $\mathtt{T(R)T(S)=T(RS)}$	Acting by the identity changes nothing, and two actions in a row are one action.
type functor fusion	$\mathtt{T(R)}(\!\lvert\mathtt{Q}\rvert\!)=(\!\lvert\mathtt{F}(\mathtt{R},\mathbb{1})\mathtt{Q}\rvert\!)$	A relator action followed by a fold is a single fold — the intermediate structure is never built. The side condition holds because $\mathtt{F}$ is a bifunctor — $\mathtt{F}(\mathtt{R},\mathbb{1})\mathtt{F}(\mathbb{1},(\!\lvert\mathtt{Q}\rvert\!))=\mathtt{F}(\mathtt{R},(\!\lvert\mathtt{Q}\rvert\!))=\mathtt{F}(\mathbb{1},(\!\lvert\mathtt{Q}\rvert\!))\mathtt{F}(\mathtt{R},\mathbb{1})$.
naturality of α	$\alpha\mathtt{T(R)=F(R,T(R))}\alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $\mathtt{G(R)=F(R,T(R))}$ to $\mathtt{T}$.
type relator	$\mathtt{T(R)}^\circ=\mathtt{T(R^\circ)}$, for $\mathtt{F}$ preserving $^\circ$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

§11.5.1. Type functor

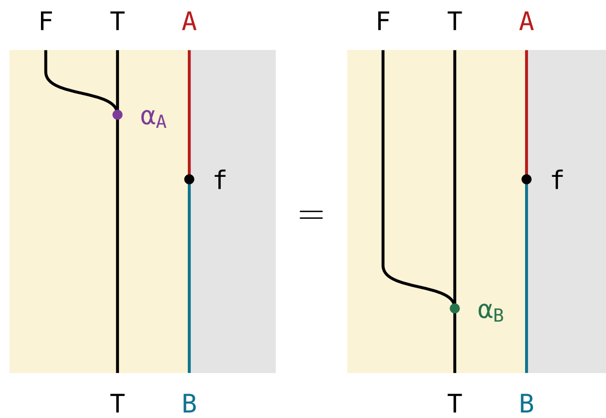
(11.5.1a)

Let $\mathtt{F}$ be a bifunctor taking both the parameter $\mathtt{A}$ and the recursive position $\mathtt{TA}$, with an initial algebra $\alpha_{\mathtt{A}} : \mathtt{F(A,TA)}\to\mathtt{TA}$ for every object $\mathtt{A}$. Then $\mathtt{T}$ is a functor, acting on a map $\mathtt{f} : \mathtt{A}\to\mathtt{B}$ by

$$\mathtt{T(f)} \triangleq \llparenthesis \mathtt{F(f,}\mathbb{1}\mathtt{)}\alpha_{\mathtt{B}} \rrparenthesis : \mathtt{TA}\to\mathtt{TB}$$

(11.5.1b)

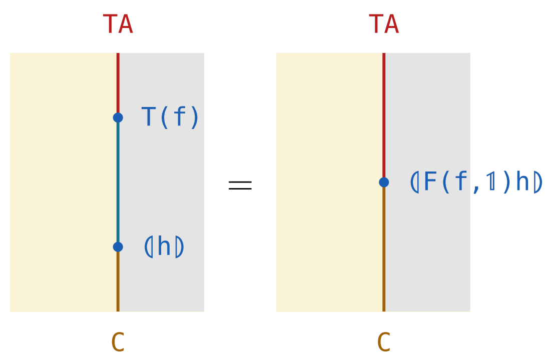
$$\begin{tikzcd}
\mathtt{F(A,TA)} \arrow[r, "\alpha_{\mathtt{A}}"] \arrow[d, "\mathtt{F}(\mathbb{1}\mathtt{,T(f))}"'] & \mathtt{TA} \arrow[d, dashed, "\mathtt{T(f)}"] \\
\mathtt{F(A,TB)} \arrow[r, "\mathtt{F(f,}\mathbb{1}\mathtt{)}\alpha_{\mathtt{B}}"] \arrow[d, "\mathtt{F(f,}\mathbb{1}\mathtt{)}"'] & \mathtt{TB} \\
\mathtt{F(B,TB)} \arrow[ur, "\alpha_{\mathtt{B}}"'] &
\end{tikzcd}$$



$$\alpha_{\mathtt{A}}\ \mathtt{T(f)} = \mathtt{F(f,T(f))}\alpha_{\mathtt{B}}$$

(11.5.1c)

$$\begin{tikzcd}
\mathtt{F(A,TA)} \arrow[r, "\alpha_{\mathtt{A}}"] \arrow[d, "\mathtt{F}(\mathbb{1}\mathtt{,T(f)}\llparenthesis \mathtt{h} \rrparenthesis\mathtt{)}"'] & \mathtt{TA} \arrow[d, dashed, "\llparenthesis \mathtt{F(f,}\mathbb{1}\mathtt{)h} \rrparenthesis"'] \arrow[r, dashed, "\mathtt{T(f)}"] & \mathtt{TB} \arrow[dl, dashed, "\llparenthesis \mathtt{h} \rrparenthesis"] \\
\mathtt{F(A,C)} \arrow[r, "\mathtt{F(f,}\mathbb{1}\mathtt{)h}"'] & \mathtt{C} &
\end{tikzcd}$$



$$\mathtt{T(f)}\llparenthesis \mathtt{h} \rrparenthesis = \llparenthesis \mathtt{F(f,}\mathbb{1}\mathtt{)h} \rrparenthesis$$

§11.6. Reduce

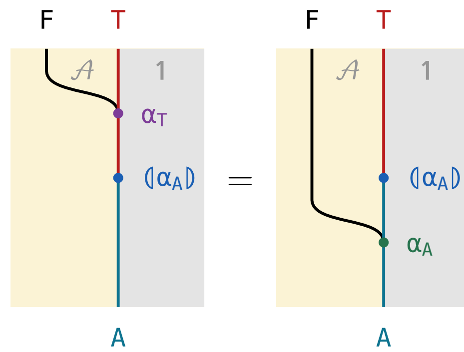
(11.6a)

> let F be a relator and has **initial algebra** $\alpha_T : \mathsf{FT}\to\mathsf{T}$ in the subcategory of functions. α_T is also initial in the allegory:

§11.6.1. The defining equation

(11.6.1a)

$$\begin{tikzcd} \mathsf{FT} \arrow[r, "\alpha_T"] \arrow[d, "\mathsf{F(X)}"'] & \mathsf{T} \arrow[d, dashed, "\mathsf{X}"] \\ \mathsf{FA} \arrow[r, "\alpha_A"'] & \mathsf{A} \end{tikzcd}$$



$$\mathsf{X}=\llparenthesis\alpha_A\rrparenthesis \Longleftrightarrow \alpha_T\ \mathsf{X}=\mathsf{F(X)}\alpha_A$$

An F-algebra is a **weakened** natural transformation. A transformation $\mathsf{F}\Rightarrow\mathbb{1}$ on $\mathcal{A}$ would need a component $\mathsf{FX}\to\mathsf{X}$ at every object and a commuting square at every arrow, but F-Algebra only need it works on T and A.

(11.6.1b)

$\llparenthesis[c,f]\rrparenthesis=\mathsf{reduce}(c,f)$

catamorphism: arrows of the form $\llparenthesis f\rrparenthesis$ are called catamorphisms, and these two examples reveal the notion to be a familiar idea in abstract clothing

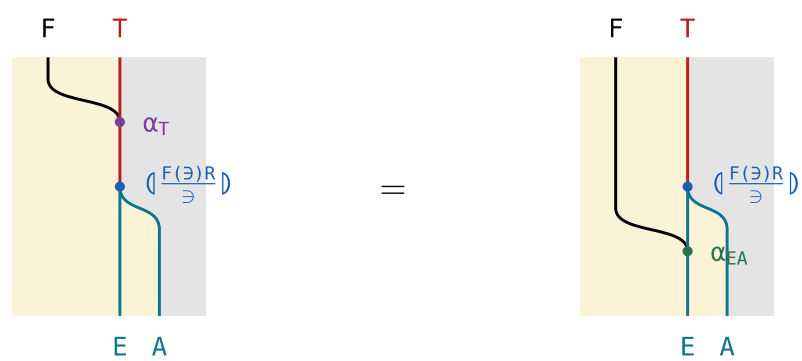
part	`Nat`	`[A]`
datatype	`Nat::=zero|succ Nat`	`[A]::=nil|cons (A,[A])`
base functor F	`F(X)=1+X`	`F(X)=1+A×X`
initial algebra α	`α=[zero,succ]` `: 1+Nat→Nat`	`α=[nil,cons]` `: 1+A×[A]→[A]`
the fold, pointwise	`⦇[c,f]⦈ zero=c` `⦇[c,f]⦈ (succ n)=f (⦇[c,f]⦈n)`	`⦇[c,f]⦈ nil=c` `⦇[c,f]⦈ (cons (a,x))=f (a,⦇[c,f]⦈ x)`

§11.6.2. $\llparenthesis R\rrparenthesis=\llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis\ni$

(11.6.2a)

$$\begin{tikzcd} \mathsf{FT} \arrow[r, "\alpha_T"] \arrow[d, "\mathsf{F}(\llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis)"'] & \mathsf{T} \arrow[d, dashed, "\llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis"] \\ \mathsf{F(EA)} \arrow[r, "\alpha_{EA}", "\frac{F(\ni)R}{\ni}"'] \arrow[d, "\mathsf{F}(\ni)"'] & \mathsf{EA} \arrow[d, "\ni"] \\ \mathsf{FA} \arrow[r, "\alpha_A", "R"'] & \mathsf{A} \end{tikzcd}$$

$\frac{1}{\ni}$ is the inverse of $\ni$



$$\alpha_T\ \llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis=F(\llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis)\ \frac{F(\ni)R}{\ni}$$

(11.6.2b)

$$\boxed{\frac{\alpha_T\ X}{F(X)R}} \overset{}{\underset{\cdot\ni\dashv\frac{1}{\ni}}{\Longleftrightarrow}} \boxed{\frac{\frac{\alpha_T\ X}{\ni}}{\frac{F(X)R}{\ni}}} \underset{\cdot\ni\dashv\frac{1}{\ni}}{\Longleftrightarrow} \boxed{\frac{\frac{\alpha_T\ X}{\ni}}{\frac{F(\frac{X}{\ni}\ni)R}{\ni}}}$$

$$\underset{\text{relator, fusion twice}}{\Longleftrightarrow} \boxed{\frac{\alpha_T\ \frac{X}{\ni}}{F(\frac{X}{\ni})\ \frac{F(\ni)R}{\ni}}} \underset{\text{reduce of maps}}{\Longleftrightarrow} \boxed{\frac{\frac{X}{\ni}}{\llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis}} \underset{\cdot\ni\dashv\frac{1}{\ni}}{\Longleftrightarrow} \boxed{\frac{X}{\llparenthesis\frac{F(\ni)R}{\ni}\rrparenthesis\ni}}$$

the last two rows at $X:=\llparenthesis R\rrparenthesis$:

§11.6.3. $\varphi(Y)\sqsubseteq Y\Longrightarrow(\mu X : \varphi(X))\sqsubseteq Y$

(11.6.3a)

> φ a **monotonic** mapping of the hom-set $A\to B$ into itself: $X\sqsubseteq Y\Longrightarrow\varphi(X)\sqsubseteq\varphi(Y)$.
>
> $(\mu X : \varphi(X))$ the least $X : A\to B$ with $\varphi(X)\sqsubseteq X$.

(11.6.3b)

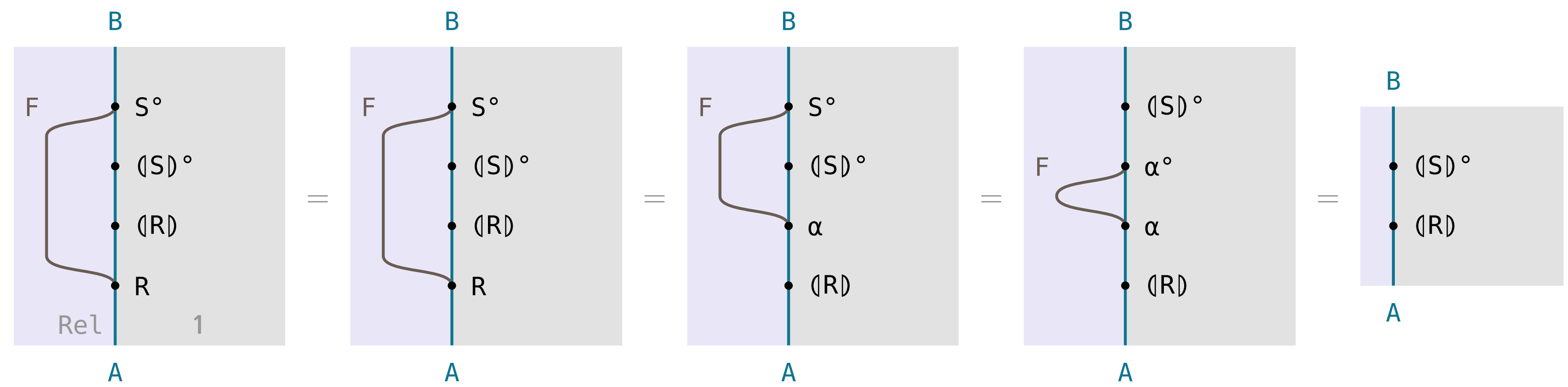
the law	**what it says**
$\varphi(Y)\sqsubseteq Y\Longrightarrow(\mu X : \varphi(X))\sqsubseteq Y$	to bound $(\mu X : \varphi(X))$ above, exhibit one Y the body does not grow past — the half §11.6.4 and every chapter after it uses
$\varphi((\mu X : \varphi(X)))=(\mu X : \varphi(X))$	**Knaster–Tarski**: the least solution of $\varphi(X)\sqsubseteq X$ already solves $\varphi(X)=X$, so the least prefix point and the least fixed point are one relation

§11.6.4. $\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis=(\mu X : S^\circ F(X)R)$

(11.6.4a)

$S^\circ F(\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis)R=\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis$

hylomorphism theorem: a prototypical 'divide and conquer' scheme — the term S° represents the decomposition stage, $F(\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis)$ the stage of solving the subproblems recursively, and R the recombination stage; $R : FA\to A$, $S : FB\to B$, $\alpha : FT\to T$ initial

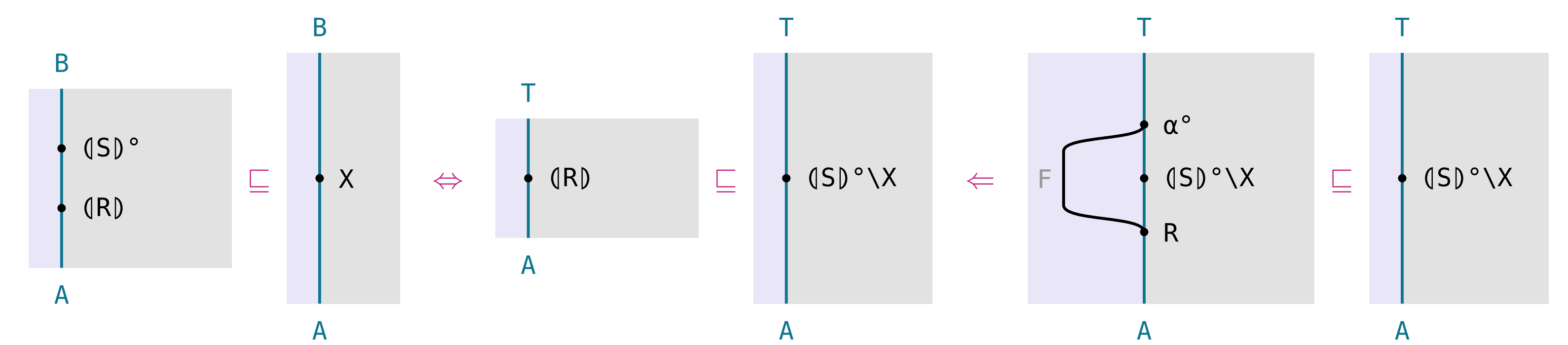


the body at $\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis$ — $F(RS)=F(R)F(S)$ — ≜ relator — ≜ reduce at R: $F(\llparenthesis R\rrparenthesis)R=\alpha\llparenthesis R\rrparenthesis$ — ≜ reduce at S conversed: $\llparenthesis S\rrparenthesis^\circ\alpha^\circ=S^\circ F(\llparenthesis S\rrparenthesis)^\circ$, and $F(\llparenthesis S\rrparenthesis)^\circ=F(\llparenthesis S\rrparenthesis^\circ)$ — (11b) — $\alpha^\circ\alpha=\mathbb{1}$, Lambek

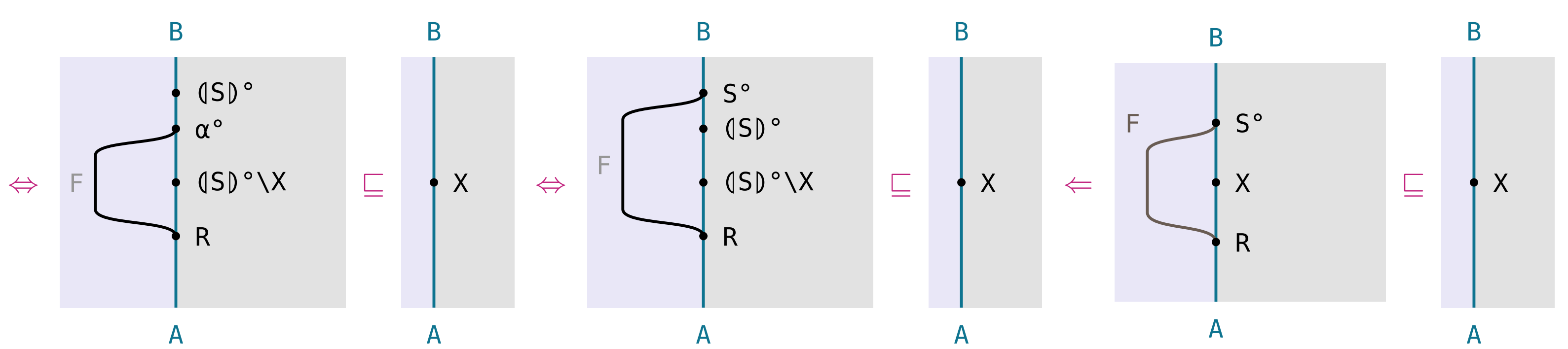
(11.6.4b)

$S^\circ F(X)R\sqsubseteq X\Longrightarrow\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis\sqsubseteq X$

hylomorphism theorem: by Knaster–Tarski, the hylomorphism $\llparenthesis S\rrparenthesis^\circ\llparenthesis R\rrparenthesis$ is included in X if X satisfies the associated recursion inequation



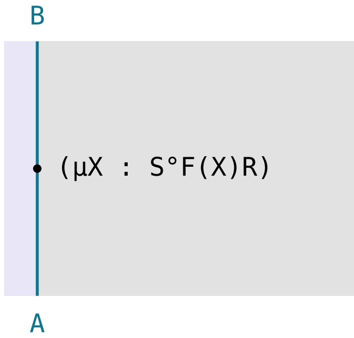
the conclusion — (1.1a)'s $S\cdot\dashv S\backslash$ at $\llparenthesis S\rrparenthesis^\circ$ — (6.2) $\llparenthesis R\rrparenthesis=(\mu X : \alpha^\circ F(X)R)$ — ≜ reduce and (11.6.3b);



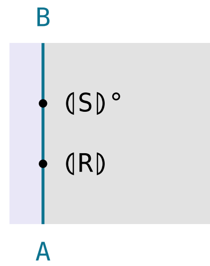
(1.1a)'s $S\cdot\dashv S\backslash$ at $\llparenthesis S\rrparenthesis^\circ$ — $\llparenthesis S\rrparenthesis^\circ\alpha^\circ=S^\circ F(\llparenthesis S\rrparenthesis^\circ)$ — (11.6.4a) — $F(RS)=F(R)F(S)$ — ≜ relator — and $\llparenthesis S\rrparenthesis^\circ(\llparenthesis S\rrparenthesis^\circ\backslash X)\sqsubseteq X$ — (1.1a)

⦇S⦈°⦇R⦈=(μX : S°F(X)R) (11.6.4c)

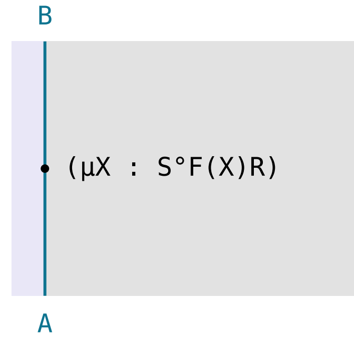
hylomorphism theorem: a hylomorphism is the least fixed point of a certain recursion equation



$\sqsubseteq$



$\sqsubseteq$



≜ μ at φ(X):=S°F(X)R

(11.6.3b)‘s φ(Y)⊑Y⟹(μX : φ(X))⊑Y at Y:=⦇S⦈°⦇R⦈, whose S°F(⦇S⦈°⦇R⦈)R=⦇S⦈°⦇R⦈ is (11.6.4a)

(11.6.4b) at X:=(μX : S°F(X)R), whose S°F((μX : S°F(X)R))R⊑(μX : S°F(X)R) is (11.6.3b)‘s φ((μX : φ(X)))=(μX : φ(X));

§12. Combinatorial functions

definition	type	note
$\mathtt{[A]::=nil} \mid \mathtt{cons\ (A,[A])}$	$\mathcal{A}\to\mathcal{A}$	The list type, under the short name it keeps. (12a)
$\mathtt{list(R)}\triangleq(\!\lvert\mathtt{[nil,(R\otimes\mathbb{1})\ cons]}\rvert\!)$	$\mathtt{[A]\to[B]}$	The relator's action on $\mathtt{R : A\to B}$: one $\mathtt{R}$ per element, the shape untouched.
$\mathtt{subseq}\triangleq(\!\lvert\mathtt{[nil,cons}\cup\pi_2\mathtt{]}\rvert\!)$	$\mathtt{[A]\to[A]}$	`xs subseq ys`: `ys` is `xs` with elements dropped — `cons` keeps the head, π_2 drops it.
$\mathtt{prefix}\triangleq(\!\lvert\mathtt{[nil,nil}\cup\mathtt{cons]}\rvert\!)$ $\mathtt{=cat^\circ}\ \pi_1\mathtt{=init^*}$	$\mathtt{[A]\to[A]}$	`ys` is an initial segment of `xs`; the first `nil` is where it stops early. $\mathtt{init}\triangleq\mathtt{snoc^\circ}\ \pi_1$.
$\mathtt{suffix}\triangleq\mathtt{cat^\circ}\ \pi_2\mathtt{=tail^*}$	$\mathtt{[A]\to[A]}$	The dual, $\mathtt{tail}\triangleq\mathtt{cons^\circ}\ \pi_2$; as a reduce it needs snoc-lists.
$\mathtt{segment}\triangleq\mathtt{suffix\ prefix}$	$\mathtt{[A]\to[A]}$	A contiguous stretch of `xs`: a suffix, then a prefix of that.
$\mathtt{partition}\triangleq\mathtt{concat^\circ}$	$\mathtt{[A]\to[[A]^+]}$	This `cat` is restricted to $\mathtt{[A]^+\times[A]\to[A]}$, so `ys` is a list of non-empty segments of `xs`.
$\mathtt{concat}\triangleq(\!\lvert\mathtt{[nil,cat]}\rvert\!)$	$\mathtt{[[A]]\to[A]}$	Joins the segments back up, which is why its converse splits a list.
`inits`	$\mathtt{[A]\to[[A]]}$	Implements $\frac{\mathtt{prefix}}{\ni}$, listing the prefixes by increasing length.
`tails`	$\mathtt{[A]\to[[A]]}$	Implements $\frac{\mathtt{suffix}}{\ni}$ by decreasing length — the opposite order.
$\mathtt{filter(p)}\triangleq\frac{\mathtt{subseq\ list(p)}}{\ni}\ \mathtt{est(R^\circ)}$	$\mathtt{[A]\to[A]}$	The longest subsequence of `xs` whose every element passes `p`. $\mathtt{est(R^\circ)}$ is $\triangleq$ `est`
$\mathtt{R}\triangleq\mathtt{length}\le\mathtt{length^\circ}$	$\mathtt{[A]\to[A]}$	The preorder `filter` and `takewhile` maximise over: the longer list wins. $\ge\triangleq\le^\circ$
$\mathtt{takewhile(p)}\triangleq\frac{\mathtt{prefix\ list(p)}}{\ni}\ \mathtt{est(R^\circ)}$	$\mathtt{[A]\to[A]}$	The same with `prefix` for `subseq`: the longest prefix whose every element passes `p`.
$\mathtt{mss}\triangleq\frac{\mathtt{segment\ sum}}{\ni}\ \mathtt{est(\ge)}$	$\mathtt{[Int]\to Int}$	Maximum segment sum. `segment=suffix prefix` splits it into $\frac{\mathtt{prefix\ sum}}{\ni}\ \mathtt{est(\ge)}$ on each suffix.

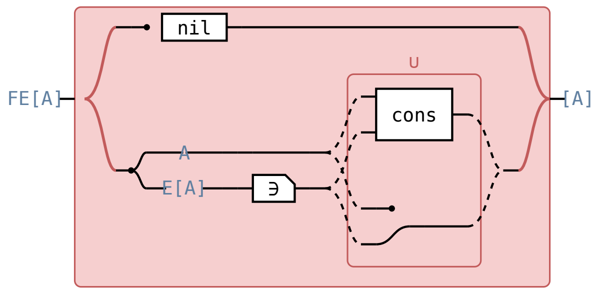
§12.1. $\frac{\mathtt{subseq}}{\ni}=(\!\lvert\mathtt{[nil}\ \frac{\mathbb{1}}{\ni},\langle\frac{\mathbb{1}\times\ni}{\ni}\ \mathtt{E(cons)},\pi_2\rangle\ \mathtt{cup]}\rvert\!)$

$\mathtt{cup}\triangleq\frac{\pi_1\ni\ \cup\ \pi_2\ni}{\ni}\ :\ \mathtt{EA\times EA\to EA}$, so $\frac{\mathtt{R}\ \cup\ \mathtt{S}}{\ni}=\langle\frac{\mathtt{R}}{\ni},\frac{\mathtt{S}}{\ni}\rangle\ \mathtt{cup}$. (12.1a)

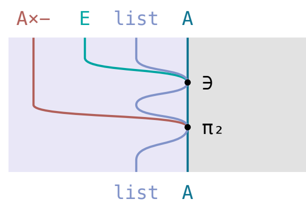
$$\frac{\mathtt{F(\ni)[nil,cons \cup \pi_2]}}{\ni} = \mathtt{[nil}\ \tfrac{1}{\ni}, \frac{\mathtt{(1\times\ni)(cons \cup \pi_2)}}{\ni}\mathtt{]} \qquad (12.1b)$$

the set of lists the algebra builds is, from nothing, just **nil**, and from a head and a set of tails, every tail in the set with the head put on or left off — (11.6.2b) at $\mathtt{subseq} = (\![\mathtt{nil}, \mathtt{cons} \cup \pi_2]\!)$, ≜ **subseq**.

circuit — the fork is `F([A])=1+A×[A]`: `nil` above, the pair below	**Hinze–Marsden**

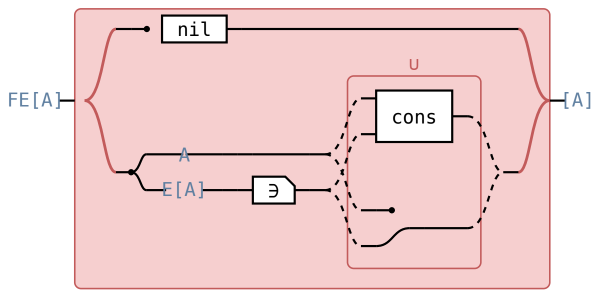


$\frac{\mathtt{F(\ni)[nil,cons \cup \pi_2]}}{\ni}$



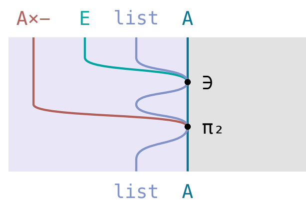
the π_2 operand of `cons ∪ π₂` under the $\mathbb{1}\times\ni$ summand of `F(∋)`, i.e. $(\mathbb{1}\times\ni)\pi_2$

=



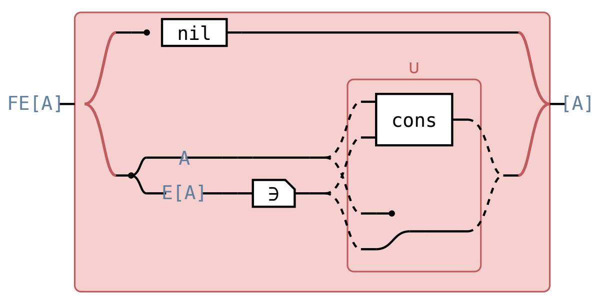
$\frac{\mathtt{(1+1\times\ni)[nil,cons \cup \pi_2]}}{\ni}$

`F(X)=1+A×X` — ≜ `subseq`



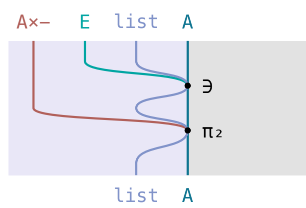
the same operand under $\mathbb{1}+\mathbb{1}\times\ni$, whose $\mathbb{1}\times\ni$ summand it sits in

=



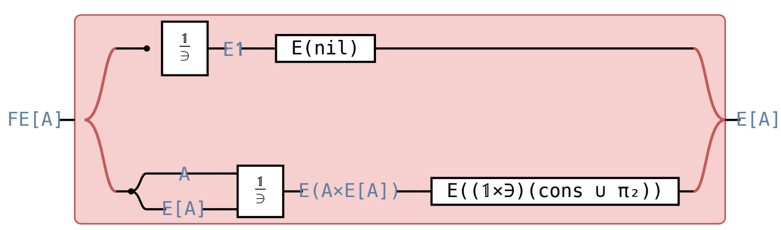
$\frac{\mathtt{[nil,(1\times\ni)(cons \cup \pi_2)]}}{\ni}$

`R+S≜[Rl,Sr]`, `l[R,S]=R`, `r[R,S]=S` — (11.2a)



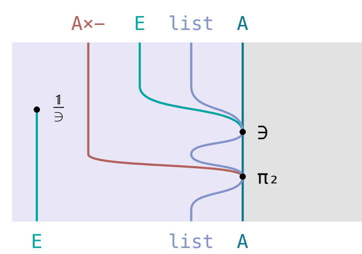
the π_2 operand of the second arm `(𝟙×∋)(cons ∪ π₂)`

=



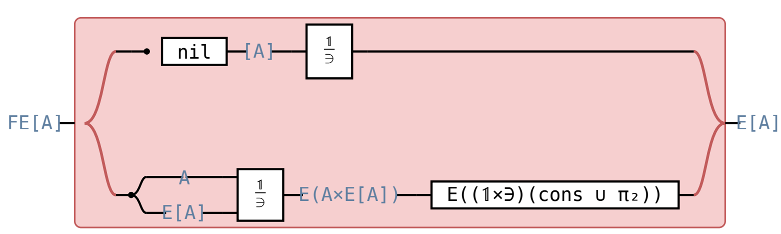
$\mathtt{[}\frac{\mathtt{nil}}{\ni}, \frac{\mathtt{(1\times\ni)(cons \cup \pi_2)}}{\ni}\mathtt{]}$

(11.2.1a) at `T:=[nil,(𝟙×∋)(cons ∪ π₂)]`



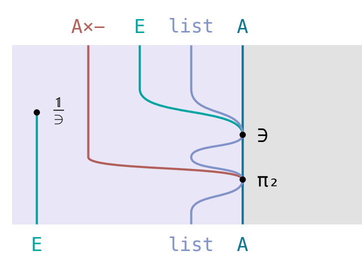
the π_2 operand under its $\mathbb{1}\%\ni$, the arm (12.1c)'s square rewrites, $(\mathbb{1}\times\ni)\pi_2=\pi_2\ni$

=



$\mathtt{[nil}\ \tfrac{1}{\ni}, \frac{\mathtt{(1\times\ni)(cons \cup \pi_2)}}{\ni}\mathtt{]}$

(10b), $\frac{\mathtt{f}}{\ni}$ `=f` $\frac{1}{\ni}$ for `f` a map, at `f:=nil`

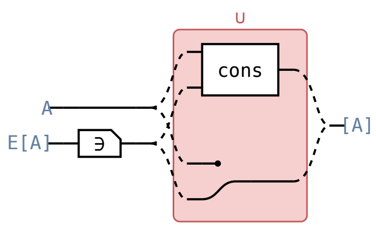
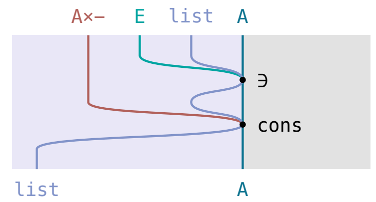
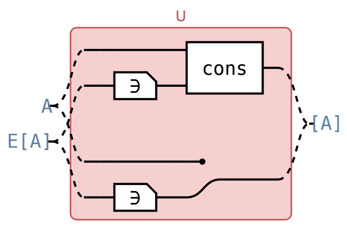
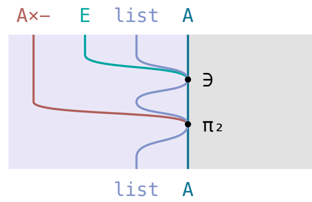
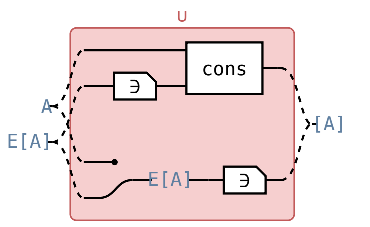
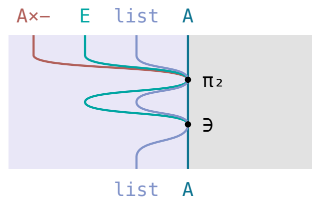
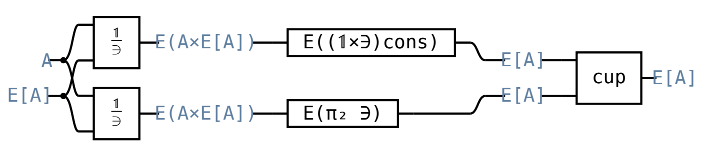
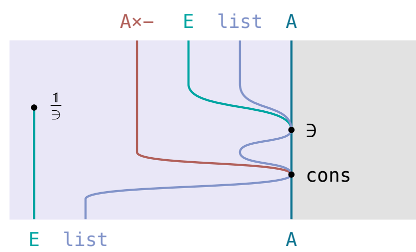
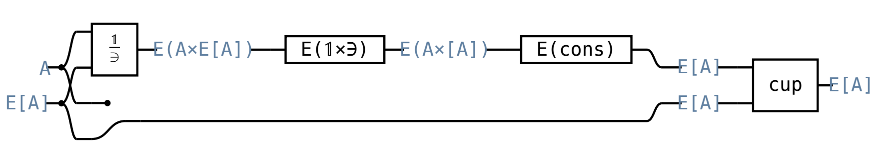
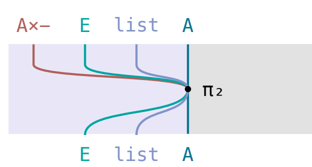


the same operand; the two rows differ only in the `nil` arm

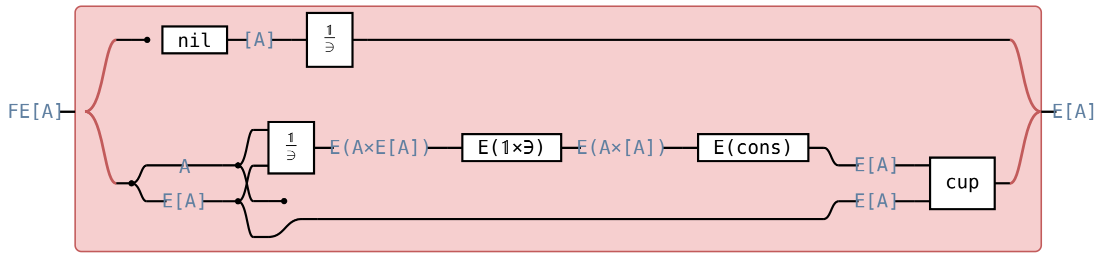
$$\begin{tikzcd} A\times E[A] \arrow[r, "\pi_2"] \arrow[d, "\mathbb{1}\times\ni"'] & E[A] \arrow[d, "\ni"] \\ A\times[A] \arrow[r, "\pi_2"'] & [A] \end{tikzcd} \quad (12.1c)$$

$$\frac{(\mathbb{1}\times\ni)(\mathtt{cons}\ \cup\ \pi_2)}{\ni} = \left\langle \frac{\mathbb{1}\times\ni}{\ni}\ \mathtt{E(cons)}, \pi_2 \right\rangle\ \mathtt{cup} \quad (12.1d)$$

power transpose of join: the power transpose of the join of two relations is $\langle \frac{R}{\ni}, \frac{S}{\ni} \rangle$ `cup`, where `cup` is the function that returns the union of two sets

circuit	Hinze–Marsden
 $\frac{(\mathbb{1}\times\ni)(\mathtt{cons}\ \cup\ \pi_2)}{\ni}$ (12.1b)'s second branch	 the `cons` operand of `cons` $\cup\ \pi_2$
=  $\frac{(\mathbb{1}\times\ni)\mathtt{cons}\ \cup\ (\mathbb{1}\times\ni)\pi_2}{\ni}$ $T(X_1\ \cup\ X_2)=TX_1\ \cup\ TX_2$ — (1.2b)	 the π_2 operand of $(\mathbb{1}\times\ni)$`cons` $\cup\ (\mathbb{1}\times\ni)\pi_2$
=  $\frac{(\mathbb{1}\times\ni)\mathtt{cons}\ \cup\ \pi_2\ni}{\ni}$ $(\mathbb{1}\times\ni)\pi_2=\pi_2\ni$ — (11.1.1b) at π_2, an equality because $\mathbb{1}$ is entire	 the π_2 operand of $(\mathbb{1}\times\ni)$`cons` $\cup\ \pi_2\ni$
=  $\left\langle \frac{(\mathbb{1}\times\ni)\mathtt{cons}}{\ni}, \frac{\pi_2\ni}{\ni} \right\rangle$ `cup` $\frac{R\ \cup\ S}{\ni} = \langle \frac{R}{\ni}, \frac{S}{\ni} \rangle$ `cup` — $\triangleq$ `cup`	 the `cons` operand under its $\mathbb{1}$%$\ni$
=  $\langle \frac{\mathbb{1}\times\ni}{\ni}\ \mathtt{E(cons)}, \pi_2 \rangle$ `cup` (10b), absorption $\frac{S}{\ni}\ \mathtt{E}(R)=\frac{SR}{\ni}$ at $S{:=}\mathbb{1}\times\ni$, $R{:=}$`cons`; fusion and $\frac{\ni}{\ni}=\mathbb{1}$ on the π_2 operand	 the π_2 operand, bare π_2

(12.1e)



$$[\tfrac{\mathtt{nil}}{\ni},\langle \tfrac{1\times\ni}{\ni}\ \mathtt{E(cons)},\pi_2\rangle\ \mathtt{cup}]$$

which writes Pcons; cons is a map, and there P(cons)=E(cons) — (11.3d).

§13. Optimisation Problems

§13.1. est(R)≜∋∩(∈\R°)

(13.1a)

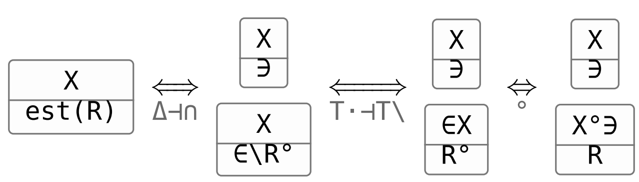
> For R : A→A, est(R)≜∋∩(∈\R°) : EA→A .
>
> xs (est(R)) x⟺x∈xs∧(∀y∈xs. x R y) the same predicate under the same letter, min R, so est(R)=min(R°) once R is an arrow;
>
> xs (est(R)) x⟺(x in xs) and all x R\: xs in q
>
> est(R)=∋nall R° all R≜∈\R, q's all; the chains below keep it written ∈\
>
> E(R)≜ $\frac{\ni R}{\ni}$: EA→EB, xs E(R) ys⟺ys={y|∃x∈xs. x R y} the image of xs, (10b)
>
> P(R) : EA→EB, xs P(R) ys⟺(∀x∈xs. ∃y∈ys. x R y)∧(∀y∈ys. ∃x∈xs. x R y) every x and every y has a partner, (11.3b)

(13.1b)

the law	what it says
X⊑est(R)⟺X⊑∋ and X°∋⊑R	in the set, and below every element of it
$\frac{1}{\ni}$ (∈\R)=R	bounding a singleton is bounding its element
$\frac{S}{\ni}$ (∈\R)=S°\R	bound S's image without building the set
union≜ $\frac{\ni\ni}{\ni}$: E(EA)→EA	flattens a set of sets
union (∈\R)=∈\(∈\R)	bound a union by bounding each member set
$\frac{1}{\ni}$ est(R)=𝟙∩R°	a singleton's minimum is its element, where R is reflexive $\frac{S}{\ni}$ est(R) at S:=𝟙
$\frac{S}{\ni}$ est(R)=S∩(S°\R°)	an S-value that points to every S-value
$\frac{S}{\ni}$ est(R)= $\frac{S}{\ni}$ est(R∩S°S)	only R between values S gives one argument counts — context
E(S) est(R)=(∋S)∩((∋S)°\R°)	the same for the image of a set $\frac{S}{\ni}$ est(R) at S:=∋S
P(f) est(R)=est(fRf°) f	shunt a function through a minimum
P(S) est(R)=(∋S)∩(∈\(SR°)) R reflexive	fusion with the power relator ⊒ is the only proof here that tabulates
P(S) est(R)⊑(∋S)∩(∈\(SR°))	the half of the row above that costs nothing
P(est(R)) est(R)⊑union est(R) R transitive	a minimum in each set, then a minimum of those
P(est(R)) est(R)=P(Dom(est(R))) union est(R) R a preorder	the same as an equality, once empty sets are dropped

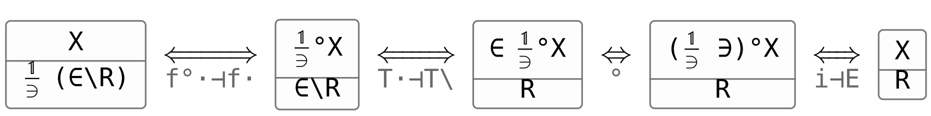
§13.1.1. X⊑est(R)⟺X⊑∋ and X°∋⊑R

(13.1.1a)



§13.1.2. $\frac{1}{\ni}$ (∈\R)=R

(13.1.2a)



§13.1.3. $\frac{S}{\ni}$ (∈\R)=S°\R

$\frac{S}{\ni}$ gathers the S-image of a point into one set and ∈\R asks that every member of that set be R-related to the target, so the set cancels and S°\R asks it of the S-image directly.

$$\boxed{\begin{array}{c} X \\ \hline \tfrac{S}{\ni}\ (\in\backslash R) \end{array}} \xLeftrightarrow[f^\circ\cdot\dashv f\cdot]{} \boxed{\begin{array}{c} \tfrac{S}{\ni}{}^\circ X \\ \hline \in\backslash R \end{array}} \xLeftrightarrow[T\cdot\dashv T\backslash]{} \boxed{\begin{array}{c} \in\ \tfrac{S}{\ni}{}^\circ X \\ \hline R \end{array}} \xLeftrightarrow[\circ]{} \boxed{\begin{array}{c} (\tfrac{S}{\ni}\ \ni)^\circ X \\ \hline R \end{array}} \xLeftrightarrow[\cdot\ni\dashv_{\frac{S}{\ni}}]{} \boxed{\begin{array}{c} S^\circ X \\ \hline R \end{array}} \xLeftrightarrow[T\cdot\dashv T\backslash]{} \boxed{\begin{array}{c} X \\ \hline S^\circ\backslash R \end{array}} \tag{13.1.3a}$$

§13.1.4. union (∈\R)=∈\(∈\R)

$$\boxed{\begin{array}{c} X \\ \hline \mathrm{union}\ (\in\backslash R) \end{array}} \xLeftrightarrow[f^\circ\cdot\dashv f\cdot]{} \boxed{\begin{array}{c} \mathrm{union}^\circ\ X \\ \hline \in\backslash R \end{array}} \xLeftrightarrow[T\cdot\dashv T\backslash]{} \boxed{\begin{array}{c} \in\mathrm{union}^\circ\ X \\ \hline R \end{array}} \xLeftrightarrow[\circ]{} \boxed{\begin{array}{c} (\mathrm{union}\ni)^\circ X \\ \hline R \end{array}} \xLeftrightarrow[\cdot\ni\dashv_{\ni}]{} \boxed{\begin{array}{c} (\ni\ni)^\circ X \\ \hline R \end{array}} \xLeftrightarrow[\circ,\ T\cdot\dashv T\backslash]{} \boxed{\begin{array}{c} X \\ \hline \in\backslash(\in\backslash R) \end{array}} \tag{13.1.4a}$$

§13.1.5. $\tfrac{S}{\ni}$ est(R)=S∩(S°\R°)

$$\boxed{\begin{array}{c} X \\ \hline \tfrac{S}{\ni}\ \mathrm{est}(R) \end{array}} \xLeftrightarrow[f^\circ\cdot\dashv f\cdot]{} \boxed{\begin{array}{c} \tfrac{S}{\ni}{}^\circ X \\ \hline \mathrm{est}(R) \end{array}} \xLeftrightarrow[\Delta\dashv\cap]{} \begin{matrix} \boxed{\begin{array}{c} \tfrac{S}{\ni}{}^\circ X \\ \hline \ni \end{array}} \\ \boxed{\begin{array}{c} \tfrac{S}{\ni}{}^\circ X \\ \hline \in\backslash R^\circ \end{array}} \end{matrix} \xLeftrightarrow[f^\circ\cdot\dashv f\cdot]{} \begin{matrix} \boxed{\begin{array}{c} X \\ \hline \tfrac{S}{\ni}\ \ni \end{array}} \\ \boxed{\begin{array}{c} X \\ \hline \tfrac{S}{\ni}\ (\in\backslash R^\circ) \end{array}} \end{matrix} \xLeftrightarrow[\cdot\ni\dashv_{\frac{S}{\ni}},\ (13.1.3a)]{} \begin{matrix} \boxed{\begin{array}{c} X \\ \hline S \end{array}} \\ \boxed{\begin{array}{c} X \\ \hline S^\circ\backslash R^\circ \end{array}} \end{matrix} \xLeftrightarrow[\Delta\dashv\cap]{} \boxed{\begin{array}{c} X \\ \hline S\cap(S^\circ\backslash R^\circ) \end{array}} \tag{13.1.5a}$$

§13.1.6. $\tfrac{S}{\ni}$ est(R)= $\tfrac{S}{\ni}$ est(R∩S°S)

$$\boxed{\begin{array}{c} X \\ \hline \tfrac{S}{\ni}\ \mathrm{est}(R\cap S^\circ S) \end{array}} \xLeftrightarrow[(13.1.5a),\ \circ]{} \boxed{\begin{array}{c} X \\ \hline S\cap(S^\circ\backslash(R^\circ\cap S^\circ S)) \end{array}} \xLeftrightarrow[\Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \begin{matrix} \boxed{\begin{array}{c} X \\ \hline S \end{array}} \\ \boxed{\begin{array}{c} S^\circ X \\ \hline R^\circ\cap S^\circ S \end{array}} \end{matrix} \tag{13.1.6a}$$

$$\xLeftrightarrow[\Delta\dashv\cap,\ S^\circ\cdot\ \text{monotone}]{} \begin{matrix} \boxed{\begin{array}{c} X \\ \hline S \end{array}} \\ \boxed{\begin{array}{c} S^\circ X \\ \hline R^\circ \end{array}} \end{matrix} \xLeftrightarrow[T\cdot\dashv T\backslash,\ \Delta\dashv\cap]{} \boxed{\begin{array}{c} X \\ \hline S\cap(S^\circ\backslash R^\circ) \end{array}} \xLeftrightarrow[(13.1.5a)]{} \boxed{\begin{array}{c} X \\ \hline \tfrac{S}{\ni}\ \mathrm{est}(R) \end{array}}$$

§13.1.7. P(f) est(R)=est(fRf°) f

$$\underset{f\ \text{a map}}{\boxed{P(f)\ \mathrm{est}(R)}} \xlongequal[P=E\ \text{on maps},\ (13.1.5a)]{} \boxed{(\ni f)\cap((\ni f)^\circ\backslash R^\circ)} \xlongequal[\circ,\ T\cdot\dashv T\backslash,\ f^\circ\cdot\dashv f\cdot]{} \boxed{(\ni f)\cap(\in\backslash(fR^\circ))} \tag{13.1.7a}$$

$$\xlongequal[\text{modular law},\ f\ \text{simple}]{} \boxed{(\ni\cap((\in\backslash(fR^\circ))f^\circ))f} \xlongequal[\cdot f\dashv\cdot f^\circ,\ \circ,\ \mathrm{est}]{} \boxed{\mathrm{est}(fRf^\circ)\ f}$$

§13.1.8. P(S) est(R)⊑(∋S)∩(∈\(SR°))

$$\boxed{\begin{array}{c} P(S)\ \mathrm{est}(R) \\ \hline (\ni S)\cap(\in\backslash(SR^\circ)) \end{array}} \xLeftarrow[\Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \begin{matrix} \boxed{\begin{array}{c} P(S)\ \mathrm{est}(R) \\ \hline \ni S \end{array}} \\ \boxed{\begin{array}{c} \in P(S)\ \mathrm{est}(R) \\ \hline SR^\circ \end{array}} \end{matrix} \xLeftarrow[\text{UP of est}]{} \begin{matrix} \boxed{\begin{array}{c} P(S)\ni \\ \hline \ni S \end{array}} \\ \boxed{\begin{array}{c} \in P(S) \\ \hline S\in \end{array}} \end{matrix} \tag{13.1.8a}$$

§13.1.9. P(est(R)) est(R)⊑union est(R)

$$\underset{R\ \text{transitive}}{\boxed{\begin{array}{c} P(\mathrm{est}(R))\ \mathrm{est}(R) \\ \hline \mathrm{union}\ \mathrm{est}(R) \end{array}}} \xLeftrightarrow[(13.1.5a)]{} \boxed{\begin{array}{c} P(\mathrm{est}(R))\ \mathrm{est}(R) \\ \hline (\ni\ni)\cap((\ni\ni)^\circ\backslash R^\circ) \end{array}} \xLeftarrow[\circ,\ \Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \begin{matrix} \boxed{\begin{array}{c} P(\mathrm{est}(R))\ \mathrm{est}(R) \\ \hline \ni\ni \end{array}} \\ \boxed{\begin{array}{c} \in P(\mathrm{est}(R)) \\ \mathrm{est}(R) \\ \hline R^\circ \end{array}} \end{matrix} \xLeftarrow[\text{UP of est},\ R\ \text{transitive}]{} \begin{matrix} \boxed{\begin{array}{c} P(\mathrm{est}(R))\ni \\ \hline \ni\mathrm{est}(R) \end{array}} \\ \boxed{\begin{array}{c} \in P(\mathrm{est}(R)) \\ \hline \mathrm{est}(R)\in \end{array}} \end{matrix} \tag{13.1.9a}$$

, R transitive

§13.2. Lax natural transformations (LaT)

(13.2a)

For relators $\mathtt{G,F} : \mathcal{C}\to\mathcal{D}$ and components $\varphi_A : \mathtt{GA}\to\mathtt{FA}$, φ is **lax at** $\mathtt{R} : \mathtt{A}\to\mathtt{B}$ when $\mathtt{G(R)}\varphi_B\sqsubseteq\varphi_A\mathtt{F(R)}$ — both sides $\mathtt{GA}\to\mathtt{FB}$, the left through the component at $\mathtt{B}$, the right through the one at $\mathtt{A}$.

$\varphi : \mathtt{G}\Rightarrow\mathtt{F}$ is a **lax natural transformation** (LaT) when it is lax at **every** $\mathtt{R}$. (5.13)

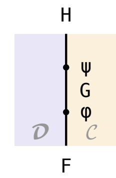
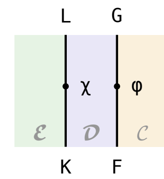
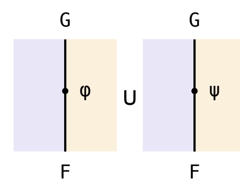
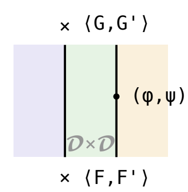
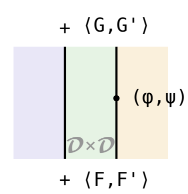
Lax at every **map** already gives LaT, and at a map the inequation is an equality $\mathtt{G(f)}\varphi=\varphi\mathtt{F(f)}$: laxness is about relations only.

(13.2b)

$$
\begin{tikzcd}
\mathtt{GA} \arrow[r, "\varphi_A"] \arrow[d, "\mathtt{G(R)}"'] \arrow[dr, phantom, "\sqsubseteq"] & \mathtt{FA} \arrow[d, "\mathtt{F(R)}"] \\
\mathtt{GB} \arrow[r, "\varphi_B"'] & \mathtt{FB}
\end{tikzcd}
$$

$\mathtt{G(R)}\varphi_B\sqsubseteq\varphi_A\mathtt{F(R)}$

(13.2c)

closed under	commutative diagram	string diagram
composition $\psi\varphi$	$\begin{tikzcd} HA \arrow[r, "\psi_A"] \arrow[d, dotted, "H(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & GA \arrow[r, "\varphi_A"] \arrow[d, dotted, "G(R)"] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, dotted, "F(R)"] \\ HB \arrow[r, "\psi_B"'] & GB \arrow[r, "\varphi_B"'] & FB \end{tikzcd}$ $H(R)\psi_B \sqsubseteq \psi_A G(R)$ and $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$ give $H(R)(\psi_B\varphi_B) \sqsubseteq (\psi_A\varphi_A)F(R)$	
horizontal composition $\chi\circ\varphi$	$\begin{tikzcd} L(GA) \arrow[r, "\chi_{GA}K(\varphi_A)"] \arrow[d, dotted, "L(G(R))"'] \arrow[dr, phantom, "\sqsubseteq"] & K(FA) \arrow[d, dotted, "K(F(R))"] \\ L(GB) \arrow[r, "\chi_{GB}K(\varphi_B)"'] & K(FB) \end{tikzcd}$ $\varphi : G \Rightarrow F$ with $G,F : \mathcal{C}\to\mathcal{D}$ and $\chi : L \Rightarrow K$ with $L,K : \mathcal{D}\to\mathcal{E}$ give $\chi\circ\varphi : L\circ G \Rightarrow K\circ F$, the family $A \mapsto \chi_{GA}K(\varphi_A)$ $L(G(R))\chi_{GB} \sqsubseteq \chi_{GA}K(G(R))$ is χ lax at $G(R)$; then $K(G(R)\varphi_B) \sqsubseteq K(\varphi_A F(R))$ is K applied to φ's own inequation; $\chi := \mathbb{1}_K$ gives $K(\varphi) : K\circ G \Rightarrow K\circ F$, K composed on the outside; $\varphi := \mathbb{1}_G$ gives $\chi G : L\circ G \Rightarrow K\circ G$, G composed on the inside two candidates, $\chi_{GA}K(\varphi_A)$ and $L(\varphi_A)\chi_{FA}$, ordered by $\sqsubseteq$; this row is the first	
union $\varphi \cup \psi$	$\begin{tikzcd} GA \arrow[r, "\varphi_A"] \arrow[d, dotted, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, dotted, "F(R)"] \\ GB \arrow[r, "\varphi_B"'] & FB \end{tikzcd} \cup \begin{tikzcd} GA \arrow[r, "\psi_A"] \arrow[d, dotted, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, dotted, "F(R)"] \\ GB \arrow[r, "\psi_B"'] & FB \end{tikzcd}$ $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$ and $G(R)\psi_B \sqsubseteq \psi_A F(R)$ give $G(R)(\varphi_B \cup \psi_B) \sqsubseteq (\varphi_A \cup \psi_A)F(R)$	
a relator K $K(\varphi)$	$\begin{tikzcd} K(GA) \arrow[r, "K(\varphi_A)"] \arrow[d, dotted, "K(G(R))"'] \arrow[dr, phantom, "\sqsubseteq"] & K(FA) \arrow[d, dotted, "K(F(R))"] \\ K(GB) \arrow[r, "K(\varphi_B)"'] & K(FB) \end{tikzcd}$ $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$ gives $K(G(R))K(\varphi_B) \sqsubseteq K(\varphi_A)K(F(R))$	
product $\varphi\times\psi$	$\begin{tikzcd} GA\times G'A \arrow[r, "\varphi_A\times\psi_A"] \arrow[d, dotted, "G(R)\times G'(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA\times F'A \arrow[d, dotted, "F(R)\times F'(R)"] \\ GB\times G'B \arrow[r, "\varphi_B\times\psi_B"'] & FB\times F'B \end{tikzcd}$ $\varphi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\varphi\times\psi : G\times G' \Rightarrow F\times F'$ $(R\times S)(U\times V)=(RU)\times(SV)$ and monotonicity in both slots — the row above's K applied to the inequation, run on a bifunctor; the fork is the derived case $\langle\varphi,\psi\rangle = \lhd(\varphi\times\psi)$, and its ONE cost is the lax copy law $R\lhd \sqsubseteq \lhd(R\times R)$ — (2b)	
coproduct $\varphi+\psi$	$\begin{tikzcd} GA+G'A \arrow[r, "\varphi_A+\psi_A"] \arrow[d, dotted, "G(R)+G'(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA+F'A \arrow[d, dotted, "F(R)+F'(R)"] \\ GB+G'B \arrow[r, "\varphi_B+\psi_B"'] & FB+F'B \end{tikzcd}$ $\varphi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\varphi+\psi : G+G' \Rightarrow F+F'$ $(R+S)(U+V)=(RU)+(SV)$ and monotonicity in both slots; the cofork is the derived case $[\varphi,\psi]=(\varphi+\psi)\triangledown$, and $\triangledown$ costs nothing	
meet — **fails** $\varphi\cap\psi$	the step it would need is $(\varphi_A F(R))\cap(\psi_A F(R)) \sqsubseteq (\varphi\cap\psi)_A F(R)$, the wrong direction of (4c) a counterexample: (13.2.1a)	
Each row is the general slide $R_A X \sqsubseteq X' R_B$ and $R_B Y \sqsubseteq Y' R_C$ give $R_A(XY) \sqsubseteq (X'Y')R_C$ at $R_A,R_B := G(R),F(R)$ and $X,X' := \varphi_B,\varphi_A$, quantified over R; at $X'=X$ with R_A,R_B endorelations it is the monotonicity reading instead — the $\forall R$ sits outside the law, and is the only difference		

§13.2.1. meet is not closed in LaT

counterexample — $\varphi,\psi\ :\ G\Rightarrow F$ lax natural does NOT give $\varphi\cap\psi$ lax natural (13.2.1a)

$$
\begin{tikzcd}
(0,1) & & \varnothing \\[-2em]
A\times A \arrow[rr, "\pi_1\cap\pi_2"] \arrow[d, "R\times R"'] \arrow[drr, phantom, "\not\Leftarrow"] & & A \arrow[d, "R"] \\
B\times B \arrow[rr, "\pi_1\cap\pi_2"'] & & B \\[-2em]
(0,0) & & \{0\}\not\subseteq\varnothing
\end{tikzcd}
$$

$A=B\triangleq\{0,1\}$, $R\triangleq\{(0,0),(1,0)\}$, $\varphi\triangleq\pi_1\cap\pi_2\ :\ \Delta\Rightarrow Id$

$\pi_1,\pi_2\ :\ \Delta\Rightarrow Id$ are both LaTs (13.4.3c)'s g row and $\pi_1\cap\pi_2=\{((x,x),x)\}$

§13.2.2. Two stacked towers

(13.2.2a)

	lower — inside ONE allegory	**upper — between allegories**
0-cell	an object A	an allegory $\mathcal{C}$ — **a whole lower tower**
1-cell	a relation $R\ :\ A\rightarrow B$	a relator $F\ :\ \mathcal{C}\rightarrow\mathcal{D}$
2-cell	$R\sqsubseteq S$	a LaT $\varphi\ :\ G\Rightarrow F$ $\triangleq$ lax
order	$R\sqsubseteq S$	$\varphi\sqsubseteq\psi$ componentwise
union	$R\cup S$	$\varphi\cup\psi$, **survives** (13.2c)
product	$R\times S$ $\triangleq\times$	$\varphi\times\psi$, **survives**; a TENSOR, not a categorical product; SYMMETRIC (11.1b) (13.2c)
coproduct	$R+S$	$\varphi+\psi$, **survives**; a BIPRODUCT (13.2c)
meet	$R\cap S$	$\varphi\cap\psi$ componentwise, **fails** (13.2.1a)
converse	R°	φ°, **fails**, oplax: $\varphi_A{}^\circ G(R)\sqsubseteq F(R)\varphi_B{}^\circ$
zero object	z with $\mathbb{1}\ z=\emptyset$	the constant relator at such a z, initial and terminal

§13.2.3. $R\sqsubseteq S$ is a special case of LaT

(13.2.3a)

$$
\begin{tikzcd}
GX \arrow[r, "\varphi_X"] \arrow[d, "G(R)"'] \arrow[dr, phantom, "\Leftarrow"] & FX \arrow[d, "F(R)"] & \Longrightarrow & A \arrow[r, "\varphi_X"] \arrow[d, "\mathbb{1}_A"'] \arrow[dr, phantom, "\Leftarrow"] & B \arrow[d, "\mathbb{1}_B"] & \Longrightarrow & A \arrow[r, bend left, "\varphi_X"] \arrow[r, bend right, "\varphi_Y"'] \arrow[r, phantom, "\Leftarrow"] & B \\
GY \arrow[r, "\varphi_Y"'] & FY & & A \arrow[r, "\varphi_Y"'] & B & & &
\end{tikzcd}
$$

$G\triangleq const\ A$, $F\triangleq const\ B$ — the relators $X\mapsto A$ and $X\mapsto B$, each sending every arrow to $\mathbb{1}$ $\triangleq$ lax

with $\mathcal{C}$ two objects and one non-identity arrow $X\rightarrow Y$, a LaT $const\ A\Rightarrow const\ B$ is the pair $\varphi_X,\varphi_Y\ :\ A\rightarrow B$ with $\varphi_Y\sqsubseteq\varphi_X$ — the datum $R\sqsubseteq S$

not every LaT is constant: $\ni\ :\ P\Rightarrow Id$ (11.3d) $\pi_2\ :\ (A\times -)\Rightarrow Id$ (12.1c) $\lhd\ :\ Id\Rightarrow\Delta$ (2b)

§13.3. Monotonic algebras

An F-algebra $\varphi : FA \to A$ is **monotonic on** $R : A \to A$ when it is lax at R at $G := F$, $F := Id$: $F(R)\varphi \sqsubseteq \varphi R$. R is an **endorelation**, so $B = A$ and the two components φ_A, φ_B are the one arrow φ — an algebra, not a family. (13.3a)

For a map $f : FA \to A$ that is $f^\circ F(R) f \sqsubseteq R$ (1.1a)'s $f^\circ \cdot \dashv f \cdot$ at $X := F(R)f$, $Y := R$, equivalently $F(R) \sqsubseteq f R f^\circ$ $\cdot f \dashv \cdot f^\circ$ then $f^\circ \cdot \dashv f \cdot$, .

$(\leq \times \leq) + \sqsubseteq + \leq$ — addition on Nat is monotonic on $\leq$, which at the point level reads $c = a + b \wedge a \leq a' \wedge b \leq b' \Longrightarrow c \leq a' + b'$.

$$\begin{tikzcd}
FA \arrow[r, "\varphi"] \arrow[d, "F(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & A \arrow[d, "R"] \\
FA \arrow[r, "\varphi"'] & A
\end{tikzcd}$$

$F(R)\varphi \sqsubseteq \varphi R$ (13.3b)

§13.3.1. Function f is monotonic on R iff it distributes over R

$f : FA \to A$ **distributes over** R if $F(est(R)) f \sqsubseteq \frac{F(\ni)f}{\ni} est(R)$. (13.3.1a)

$+$ distributes over $\leq$, at the point level $\min xs + \min ys = \min\{x + y \mid x \in xs \wedge y \in ys\}$ for xs, ys non-empty and $\min \triangleq est(\leq)$.

$$\begin{tikzcd}
F(EA) \arrow[r, "\frac{F(\ni)f}{\ni}"] \arrow[d, "F(est(R))"'] \arrow[dr, phantom, "\sqsubseteq"] & EA \arrow[d, "est(R)"] \\
FA \arrow[r, "f"'] & A
\end{tikzcd}$$

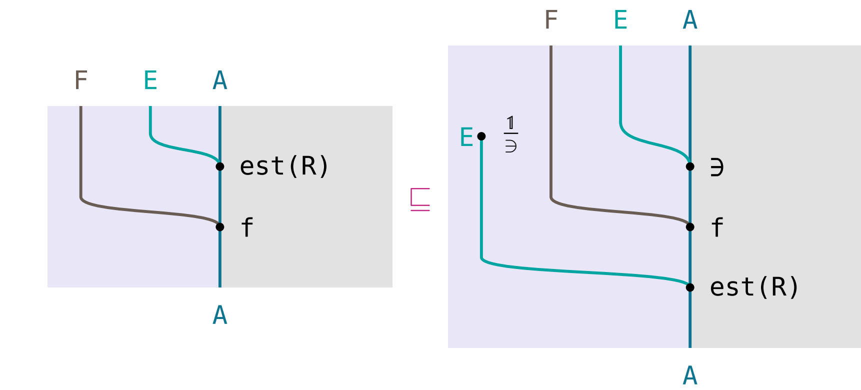
$F(est(R)) f \sqsubseteq \frac{F(\ni)f}{\ni} est(R)$

$$\begin{tikzcd}
\underset{(xs,ys)}{E\,Nat \times E\,Nat} \arrow[r, "\frac{(\ni\times\ni)+}{\ni}"] \arrow[d, "est(\leq)\times est(\leq)"'] \arrow[dr, phantom, "\sqsubseteq"] & \underset{\{x+y \mid x \in xs \wedge y \in ys\}}{E\,Nat} \arrow[d, "est(\leq)"] \\
\underset{(\min xs, \min ys)}{Nat \times Nat} \arrow[r, "+"'] & \underset{\min xs + \min ys}{Nat}
\end{tikzcd}$$

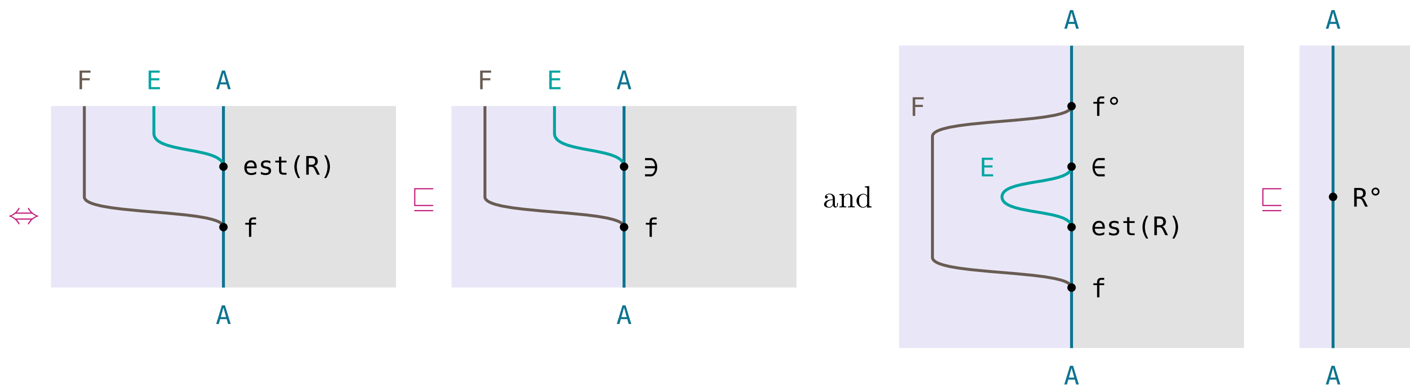
$(est(\leq) \times est(\leq)) + \sqsubseteq \frac{(\ni\times\ni)+}{\ni} est(\leq)$ (13.3.1b)

$$f^\circ F(R) f \sqsubseteq R \iff F(est(R)) f \sqsubseteq \frac{F(\ni) f}{\ni}\ est(R)$$ (13.3.1c)

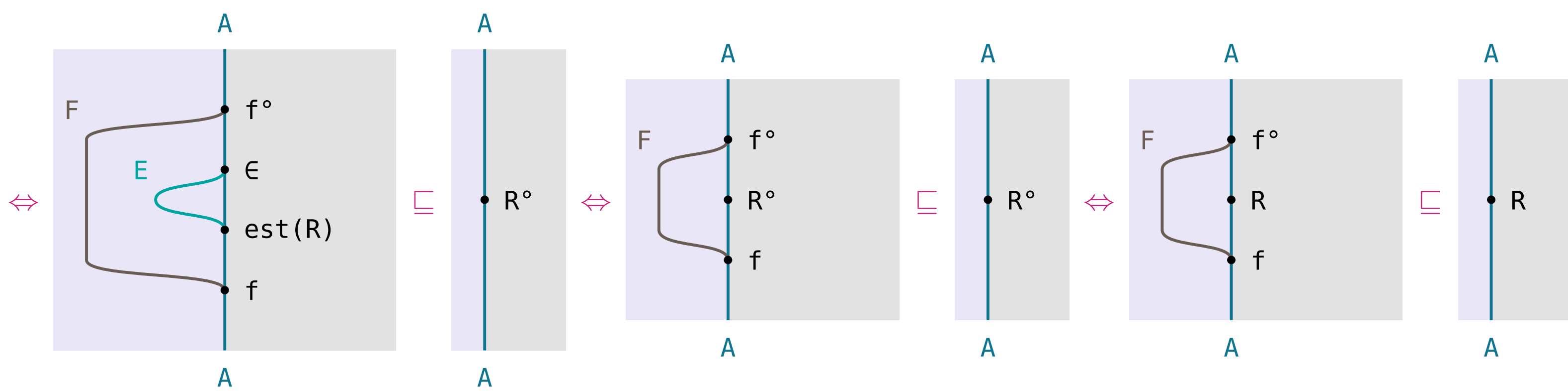
function f is monotonic over R if and only if it distributes over R; f a map, R reflexive



f distributes over R — ≙ distributes — the fraction bent as (10.1a)



(13.1.5a) splits the bound in two, (8.1c) moving $(F(\ni)f)^\circ$ across



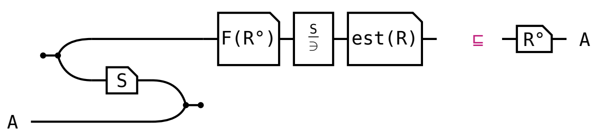
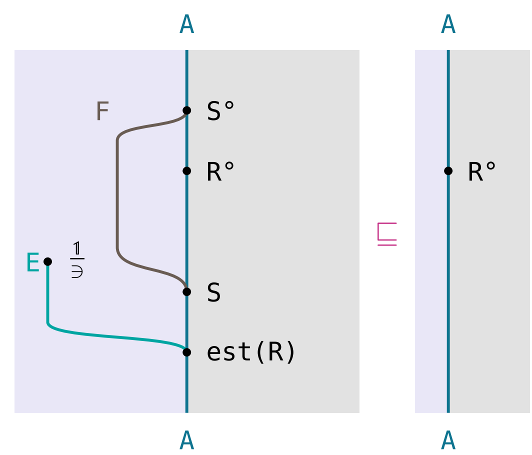
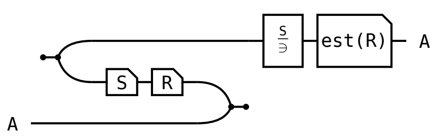
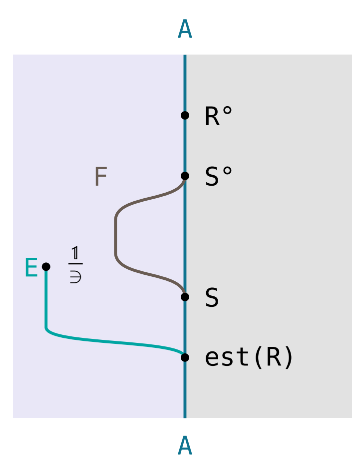
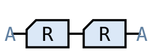
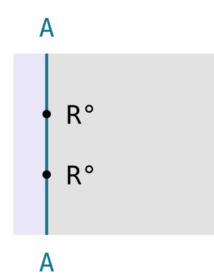
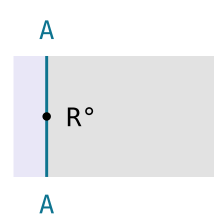
est(R)⊑∋ — ≙ est — so the first conjunct drops

$(F(\ni)f)^\circ = f^\circ F(\in)$ — ≙ ° — and ∈ est(R)=R° — ≙ est, R reflexive

both sides conversed — $F(R^\circ)^\circ = F(R)$, (11b)

§13.3.2. Greedy Theorem: $(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!) \sqsubseteq \frac{(\!|S|\!)}{\ni}\ \mathtt{est(R)}$, given S monotoic on R, F preserving °, and R transitive

(13.3.2a)

circuit	Hinze–Marsden
$(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!) \sqsubseteq \frac{(\!|S|\!)}{\ni}\ \mathtt{est(R)}$ the conclusion: one minimum kept at each step is below every result collected and one minimum taken at the end	
$(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!) \sqsubseteq (\!|S|\!)$ and $\Leftrightarrow$ $(\!|S|\!)^\circ(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!) \sqsubseteq R^\circ$ (13.1.5a) at $(\!|S|\!)$; $(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!) \sqsubseteq (\!|S|\!)^\circ \backslash R^\circ \Longleftrightarrow (\!|S|\!)^\circ(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!) \sqsubseteq R^\circ$ — (8.1c)	
$\Leftarrow$ $\frac{S}{\ni}\ \mathtt{est(R)} \sqsubseteq S$ the left conjunct; (13.1.5a), and $(\!|-|\!)$ is monotone	
 $\Leftarrow$ the right conjunct; $(\!|S|\!)^\circ(\!|\frac{S}{\ni}\ \mathtt{est(R)}|\!)$ is the least X with $X = S^\circ F(X)(\frac{S}{\ni}\ \mathtt{est(R)})$ hylomorphism theorem — so Knaster–Tarski leaves this one inequation $\frac{S}{\ni} = \frac{1}{\ni}\ E(S)$ — (10.1a)	
 $\sqsubseteq$ $S^\circ F(R^\circ) \sqsubseteq R^\circ S^\circ$ — ≙ monotonic at S, conversed; $F(R)^\circ = F(R^\circ)$ — (11b)	
$\sqsubseteq$  $S^\circ(\frac{S}{\ni}\ \mathtt{est(R)}) \sqsubseteq S^\circ(S^\circ \backslash R^\circ) \sqsubseteq R^\circ$ — (13.1.5a), (8.1c)	
$\sqsubseteq$  R transitive	

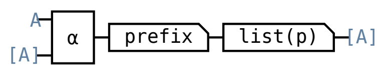
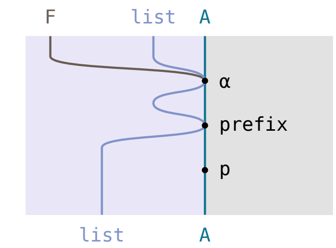
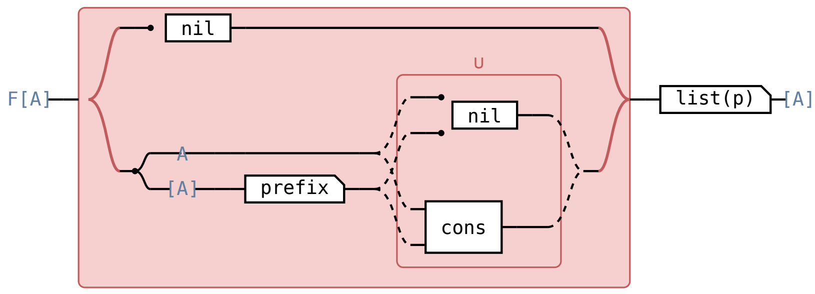
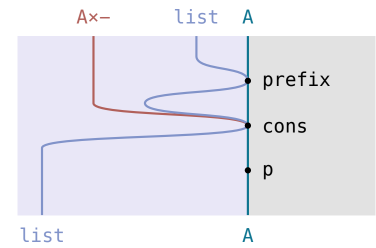
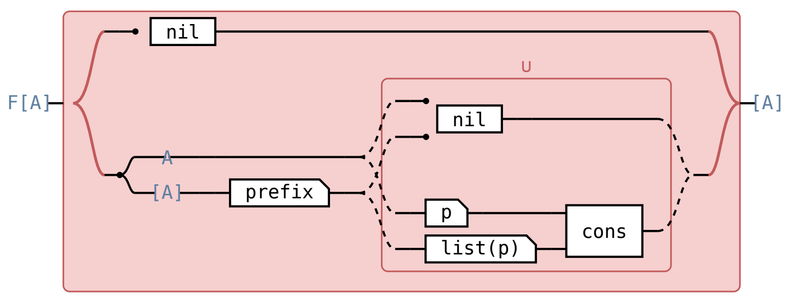
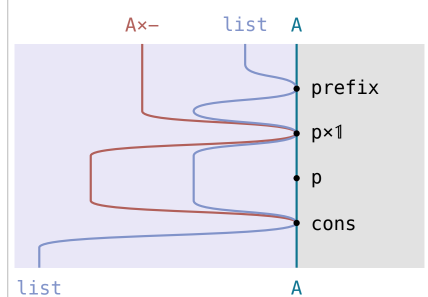
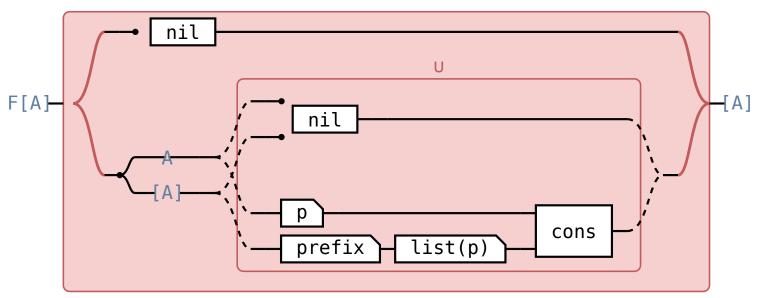
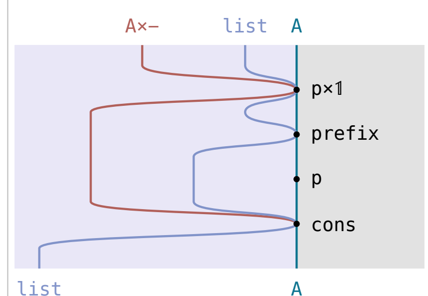
§13.3.3. `takewhile(p)=⦇[nil,(π₁p→cons,⊸ nil)]⦈`

(13.3.3a)

name	definition	type	example	in words
F	FX=1+A×X	𝒜→𝒜		nothing, or a head and a tail
nil, cons	[A]::=nil|cons (A,[A]) ≙ subseq	1→[A], A×[A]→[A]	cons(3,[1,2])=[3,1,2]	the empty list; a head onto a tail
α	[nil,cons]	F([A])→[A]		both constructors as one map
p	a coreflexive	A⇸A	p≙even — 2 p 2, and 3∉Dom(p)	{(a,a)|a passes the test}
R	length≤length°, a preorder	[A]⇸[A]	[1] R [1,2]	xs R ys⟺length xs≤length ys
⊸ nil	the constant nil — the second nil of prefix	A×[A]→[A]	(⊸ nil)(3,[1,2])=nil	drop the pair, return nil
prefix	⦇[nil,⊸ nil ∪ cons]⦈ ≙ subseq	[A]⇸[A]	[3,1,2] prefix [3,1]	xs prefix ys⟺∃zs. xs=ys++zs — at each cons, stop or keep the head
S	[nil,⊸ nil ∪ (p×𝟙) cons]	F([A])⇸[A]	(4,[2]) S [4,2], and (3,[2]) S nil only	prefix's algebra with one extra p — stop, or keep a head that passes p
(g→X,Y)	X where g is defined and Y where it is not	A⇸B at X,Y : A⇸B		the test picks the branch

nil R=⊤, nil R°=nil nil is the shortest list — below every list, and above only itself, so it loses every est(R°)

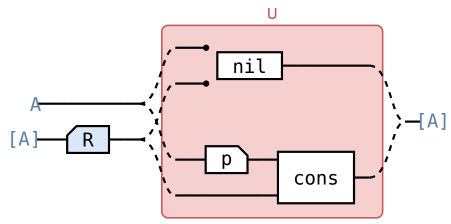
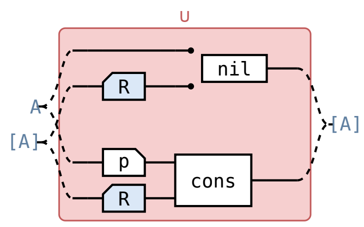
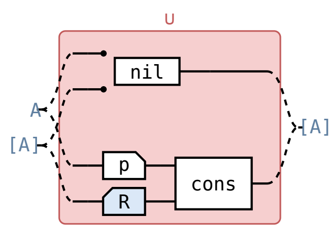
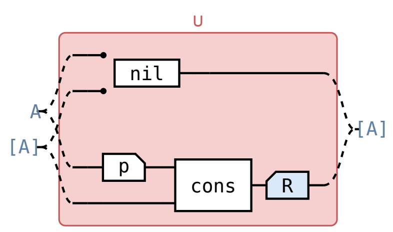
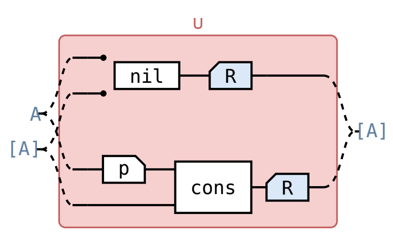
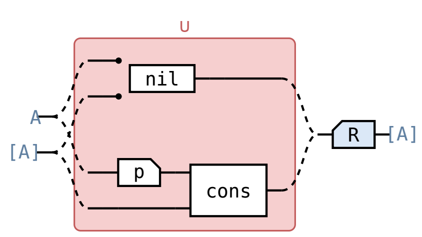
3.3.3b)

α prefix list(p)=F(prefix list(p))S	
building the list and then keeping a **p**-passing prefix of it is keeping one of the tail first, and then building with **S**	
this same diagram is **subseq**'s: algebra **[nil,π₂ ∪ cons]**, type **[A]→[A]**	
circuit — the fork is F([A])=1+A×[A]: nil above, the pair below	**Hinze–Marsden**
 α prefix list(p)	 the cons branch alone, without 1+ or ⊸ nil
=  F(prefix) [nil,⊸ nil ∪ cons] list(p) defining equation	 the cons operand of ⊸ nil ∪ cons
=  F(prefix) [nil,⊸ nil ∪ (p×list(p)) cons] list(p) through cons	 the (p×list(p)) cons operand of ⊸ nil ∪ (p×list(p)) cons
=  [nil,⊸ nil ∪ (p×(prefix list(p))) cons] relator, prefix entire	
=  F(prefix list(p))S prefix list(p) entire	

≙ reduce reads that off as prefix list(p)=⦇S⦈. fusion cannot: list(p) is not entire, (𝟙×list(p))⊸ nil⊏⊸ nil, and no algebra meets the side condition. ,

(𝟙×R°)(⊸ nil ∪ (p×𝟙) cons)⊑(⊸ nil ∪ (p×𝟙) cons)R° (13.3.3c)

shortening the tail and then taking the step lands inside taking the step and then shortening the result

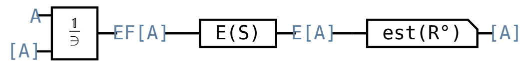
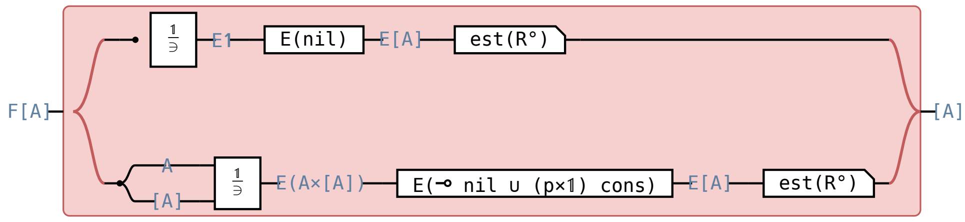
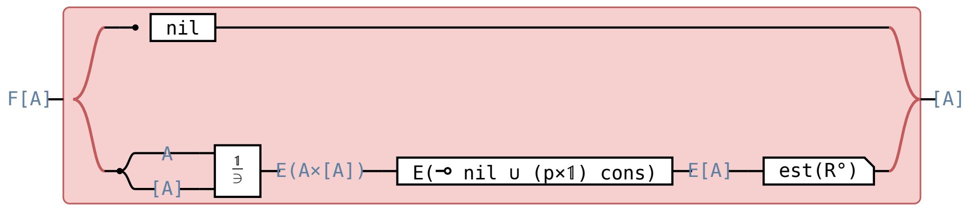
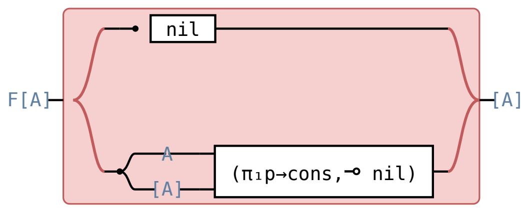
circuit — the cons branch of F(R°)S⊑SR°	**reason**
 (𝟙×R°)(⊸ nil ∪ (p×𝟙) cons)	
=  (𝟙×R°)⊸ nil ∪ (p×R°) cons	each operand is reached on its own (1.1a) — and (𝟙×R°)(p×𝟙) is p and R° on the pair's two strands at once
⊑  ⊸ nil ∪ (p×R°) cons	⊸ is the greatest arrow into 1, so (𝟙×R°)⊸⊑⊸
⊑  ⊸ nil ∪ (p×𝟙) cons R°	cons length=(𝟙×length)π_2 succ with succ monotone — a shorter tail makes a shorter list
=  ⊸ nil R° ∪ (p×𝟙) cons R°	nil R°=nil ≙ takewhile — so the constant branch may carry the R° the other one already has
=  (⊸ nil ∪ (p×𝟙) cons)R°	one R° past the join is the two inside it (1.1a)

the nil branch, which no row above draws, is nil⊑nil R°.

(13.3.3d)

$\frac{\texttt{S}}{\ni}$ est(R°)=[nil,(π₁p→cons,⊸ nil)]

the longest of the lists the algebra allows is the **cons** where the head passes **p**, and **nil** where it does not

formula	reason
	
=  [$\frac{\texttt{nil}}{\ni}$ est(R°), $\frac{\texttt{⊸ nil ∪ (p×1) cons}}{\ni}$ est(R°)]	coproduct of maps
=  [nil, $\frac{\texttt{⊸ nil ∪ (p×1) cons}}{\ni}$ est(R°)]	singleton, R° reflexive
= 	nil R=⊤

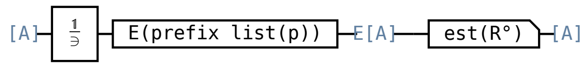
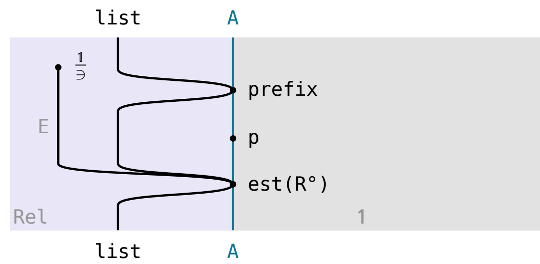
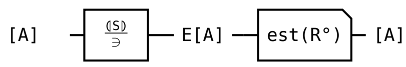
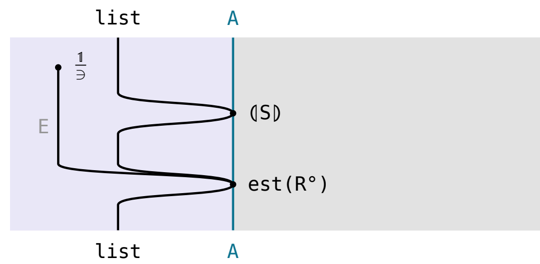
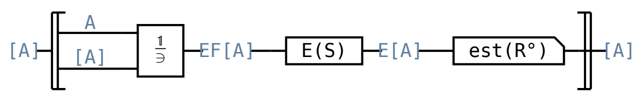
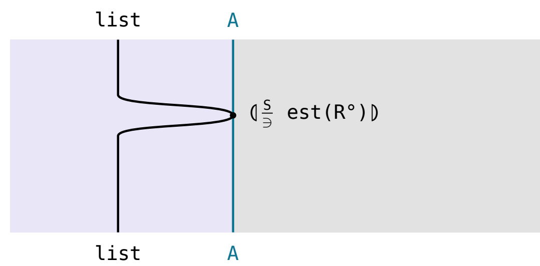
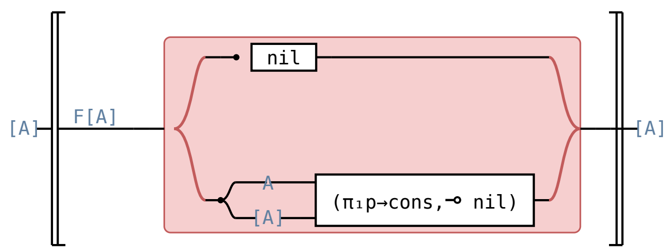
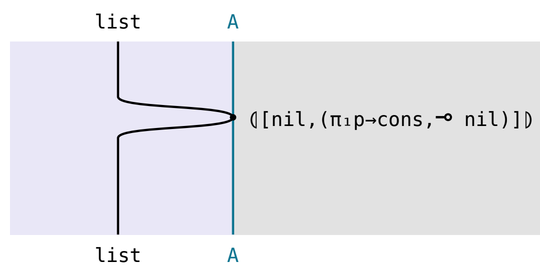
the set is {nil} where p fails on the head and {nil,cons(a,xs)} where it holds, and nil loses the second — ≜ est at a two-element set.

(13.3.3e)

name	definition	type	example	in words
S	[nil,⊸ nil ∪ (p×1) cons] ≜ takewhile	F([A])→[A]	(4,[2]) S [4,2], and (3,[2]) S nil only	prefix's algebra with one extra p — stop, or keep a head that passes p

$$\mathtt{takewhile(p)} \triangleq \frac{\mathtt{prefix\ list(p)}}{\ni}\ \mathtt{est(R^\circ)} = (\![\mathtt{nil},(\pi_1\mathtt{p}\rightarrow\mathtt{cons},\multimap\ \mathtt{nil})]\!) \qquad (13.3.3f)$$

takewhile: `takewhile(p) x` returns the longest prefix of `x` with the property that all its elements satisfy `p`; the catamorphism is the standard implementation.

circuit	Hinze–Marsden
 the specification — ≙ `est`'s `est(R°)`.	
=  (13.3.3b)	
⊒  greedy theorem at `R°`, with $\mathtt{F(R^\circ)S \sqsubseteq SR^\circ}$ — (13.3.3c) — for its hypothesis: one longest `p`-prefix kept at each `cons`, instead of every `p`-prefix collected and one chosen at the end.	
=  (13.3.3d)	
$\mathtt{takewhile(p)^\circ\ takewhile(p) \sqsubseteq prefix^\circ\ prefix \cap R \cap R^\circ \sqsubseteq 1\!\!1}$ $\mathtt{takewhile(p) \sqsubseteq prefix\ list(p)}$ and $\mathtt{(prefix\ list(p))^\circ}$ $\mathtt{takewhile(p) \sqsubseteq R}$ — (13.1.5a) at `est(R°)` — and two prefixes of one list of equal length are equal, so `takewhile(p)` is simple: **the** longest, not **a** longest.	
$\frac{\mathtt{prefix\ list(p)}}{\ni}\ \mathtt{est(R^\circ)}$ entire `nil` is always a `p`-prefix and `R` is connected on the prefixes of one list, so the longest exists.	
$\mathtt{X \sqsubseteq Y}$, `X` entire, `Y` simple $\Longrightarrow \mathtt{X=Y}$ $(\![\mathtt{nil},(\pi_1\mathtt{p}\rightarrow\mathtt{cons},\multimap\ \mathtt{nil})]\!)$ is a reduce of maps, hence entire — what turns the ⊒ above into the heading's =.	

§13.3.4. $\mathtt{mss} = (\![\mathtt{zero}\langle 1\!\!1,\frac{1}{\ni}\rangle,\langle(1\!\!1\times\pi_1)\oplus,\langle(1\!\!1\times\pi_1)\oplus\ \frac{1}{\ni},\pi_2\pi_2\rangle\ \mathtt{cup}\rangle]\!)\ \pi_2\ \mathtt{est(\geq)}$

$\mathtt{FX=1+A\times X}$, $\mathtt{A:=Int}$, $\alpha \triangleq [\mathtt{nil},\mathtt{cons}]$, $\mathtt{sum} = (\![\mathtt{zero},\mathtt{plus}]\!)$ and `segment=suffix prefix` from (7a) and ≙ `subseq`. (13.3.4a)

$\mathtt{head} \triangleq \mathtt{cons}^\circ\ \pi_1$, $\mathtt{wrap} \triangleq \langle 1\!\!1,\multimap\ \mathtt{nil}\rangle\ \mathtt{cons}$ the head of a list and the one-element list, beside ≙ `subseq`'s $\mathtt{tail} \triangleq \mathtt{cons}^\circ\ \pi_2$

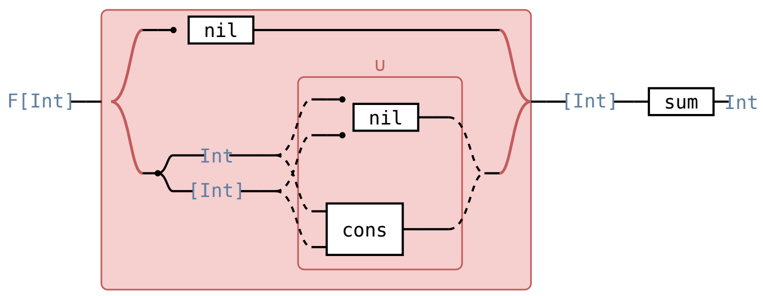
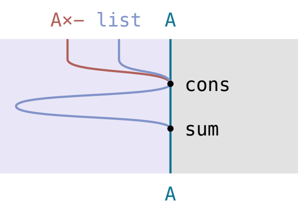
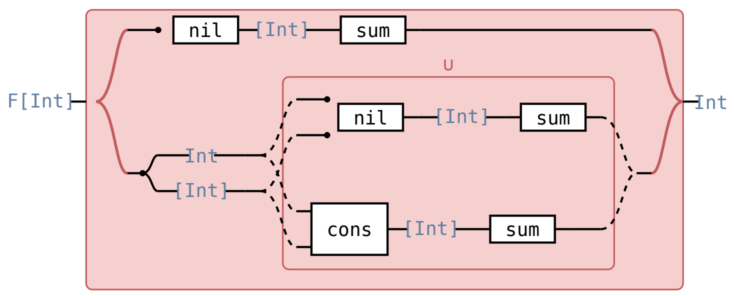
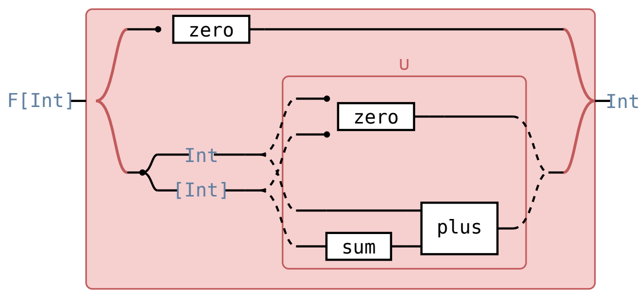
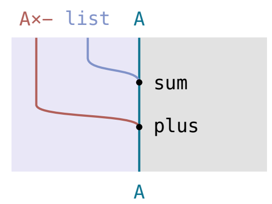
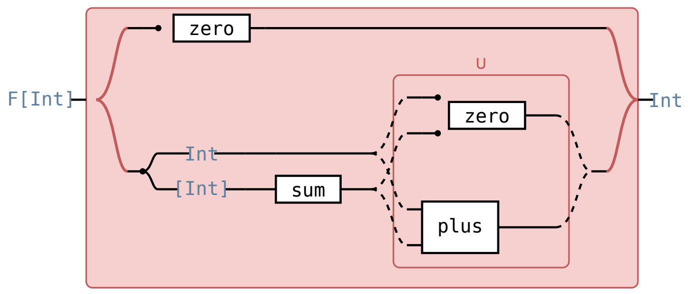
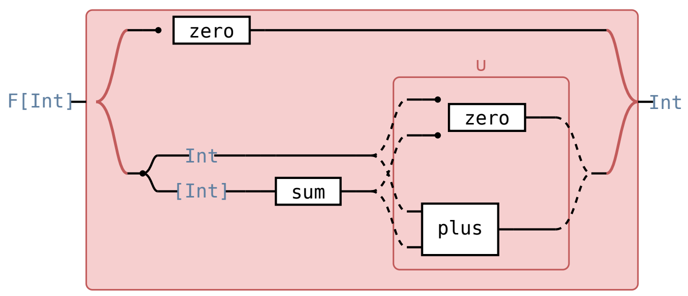
$\oplus \triangleq \frac{\multimap\ \mathtt{zero} \cup \mathtt{plus}}{\ni}\ \mathtt{est(\geq)}$ the set at `(a,b)` is `{0,a+b}`, so ⊕ is the larger of the two,

$\frac{\texttt{segment sum}}{\ni}$ **est(≥)=** $\frac{\texttt{suffix}}{\ni}$ **E(** $\frac{\texttt{prefix sum}}{\ni}$ **est(≥)) est(≥)** maximum segment sum problem: using **segment=suffix prefix**, the specification is expressed in this form	3.3.4b)
formula — one wire from [A] to A, its type written along it	**reason**
· [A] $\boxed{\frac{\texttt{segment sum}}{\ni}}$ EA $\boxed{\texttt{est(≥)}}$ A ·	
= · [A] $\boxed{\frac{\texttt{suffix (prefix sum)}}{\ni}}$ EA $\boxed{\texttt{est(≥)}}$ A ·	segment=suffix prefix ≙ subseq, ≙ mss
= · [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E[A] $\boxed{\texttt{E(prefix sum)}}$ EA $\boxed{\texttt{est(≥)}}$ A ·	absorption (10b) — frac(S,∋) E(R)=frac(SR,∋) at S:=suffix, R:=prefix sum
= · [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E[A] $\boxed{\texttt{E(}\frac{\texttt{prefix sum}}{\ni}\texttt{)}}$ E(EA) $\boxed{\texttt{union}}$ EA $\boxed{\texttt{est(≥)}}$ A ·	$\frac{\texttt{R}}{\ni}$ ∋=R, union=E(∋) (10b)'s frac(R,∋)∋=R at R:=prefix sum and E(R)≙frac(∋R,∋); (13.1b)'s union≙frac(∋∋,∋); the middle equality is ≙ relator's F(RS)=F(R)F(S) at F:=E
= · [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E[A] $\boxed{\texttt{E(}\frac{\texttt{prefix sum}}{\ni}\texttt{)}}$ E(EA) $\boxed{\texttt{E(est(≥))}}$ EA $\boxed{\texttt{est(≥)}}$ A ·	(13.1b), the sets non-empty
= · [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E[A] $\boxed{\texttt{E(}\frac{\texttt{prefix sum}}{\ni}\texttt{ est(≥))}}$ EA $\boxed{\texttt{est(≥)}}$ A ·	relator ≙ relator — F(RS)=F(R)F(S) at F:=E

The union step is (13.1b)'s P(est(R)) est(R)=P(Dom(est(R))) union est(R) — every suffix has the empty prefix, so Dom is 𝟙 here — and P(f)=E(f) at the map it is applied to ((11.3d)).

`[nil,⊸ nil ∪ cons] sum=F(sum)[zero,⊸ zero ∪ plus]` (13.3.4c)

fusion: **`prefix`** expressed as a catamorphism on cons-lists, ⦇**`[nil,⊸ nil ∪ cons]`**⦈, and this is the fusion condition that expresses **`prefix sum`** as a catamorphism

circuit — the fork is the bracket's case split `F([A])=𝟙+A×[A]`: `nil` above, the pair and its `∪` below	**Hinze–Marsden**
 `[nil,⊸ nil ∪ cons] sum`	 the `cons` operand of `⊸ nil ∪ cons`
=  `[nil sum,⊸ nil sum ∪ cons sum]` coproduct of maps, composition over `∪`	
=  `[zero,⊸ zero ∪ (𝟙×sum) plus]` `sum`'s defining equation	 the `(𝟙×sum) plus` operand of `⊸ zero ∪ (𝟙×sum) plus`
=  `[zero,(𝟙×sum)(⊸ zero ∪ plus)]` `(𝟙×sum)⊸=⊸`, `sum` entire	
=  `F(sum) [zero,⊸ zero ∪ plus]` relator	

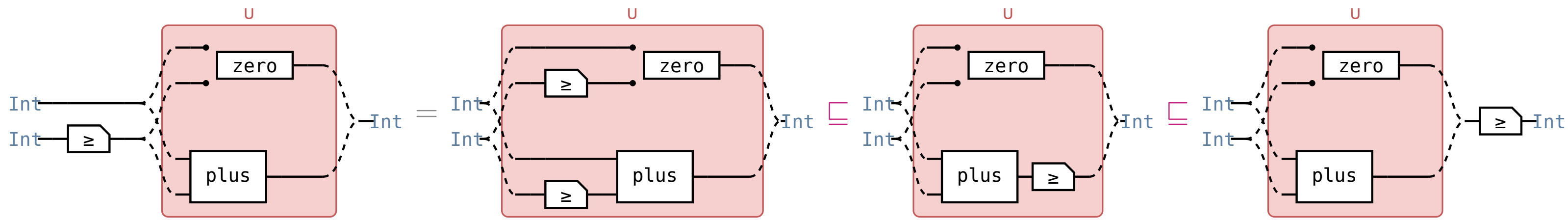
fusion at α_B `:=[nil,⊸ nil ∪ cons]`, `S:=sum`: the side condition, so `prefix sum=⦇[zero,⊸ zero ∪ plus]⦈`. `prefix` is the reduce, `sum` the map fused into it — the intermediate list is gone.

(𝟙×≥)(⊸ zero ∪ plus)⊑(⊸ zero ∪ plus)≥ (13.3.4d)

monotonic algebra: an F-algebra S is monotonic on a relation R if F(R)S⊑SR — the plus branch of F(≥)S⊑S≥; the zero branch is zero⊑zero≥

circuit — the head above, the running sum below; the tape is the ∪

(𝟙×≥)(⊸ zero ∪ plus) (𝟙×≥)⊸ zero ∪ (𝟙×≥) plus ⊸ zero ∪ plus≥ (⊸ zero ∪ plus)≥



relator, composition over ∪

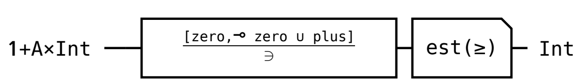
(5.1a), (≥×≥) plus⊑plus≥; ≥ reflexive (≤×≤) plus⊑plus≤ is ≙ monotonic, written + there, and plus is a map, so it is monotonic on an order and on its opposite together, which carries it to ≥.

$\frac{[\texttt{zero},⊸\ \texttt{zero} ∪ \texttt{plus}]}{\ni}$ est(≥)=[zero,⊕] (13.3.4e)

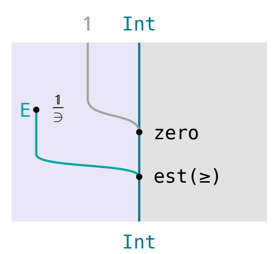
the largest sum the algebra offers is zero from nothing and, from a head and a running sum, the larger of zero and the head added to it

circuit — the tape is the coproduct: zero's branch above, plus's below

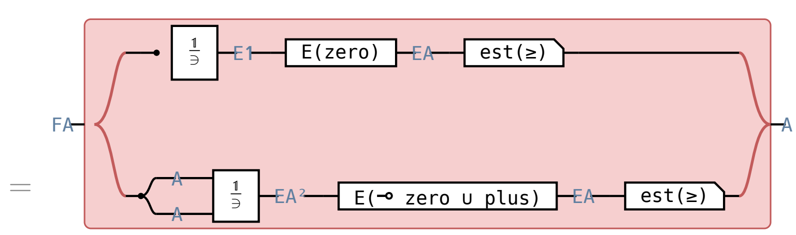
Hinze–Marsden



$\frac{[\texttt{zero},⊸\ \texttt{zero} ∪ \texttt{plus}]}{\ni}$ est(≥)

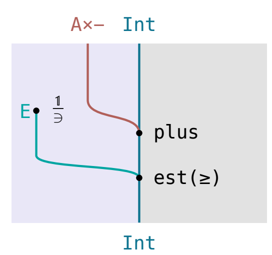


the zero arm of the bracket

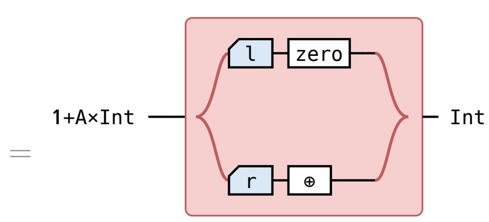


[$\frac{\texttt{zero}}{\ni}$ est(≥), $\frac{⊸\ \texttt{zero} ∪ \texttt{plus}}{\ni}$ est(≥)]

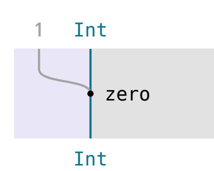
coproduct of maps — (11.2.1a) at T:=[zero,⊸ zero ∪ plus], then [U,V]Z=[UZ,VZ] — (11.2a), composition over ∪



the plus operand of the lower arm's ⊸ zero ∪ plus, under its 𝟙%∋ E(…) and est(≥)



singleton, ≥ reflexive — (13.1b)'s $\frac{1}{\ni}$ est(R)=𝟙∩R at R:=≥, zero a map; the lower branch is ⊕'s definition, ≙ mss, and no law



with (13.3.4d) the greedy theorem gives ⦇[zero,⊕]⦈⊑ $\frac{\texttt{prefix sum}}{\ni}$ est(≥) — and (13.3.4g) makes it an equality.

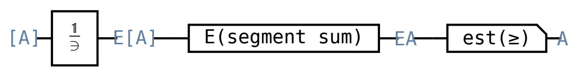
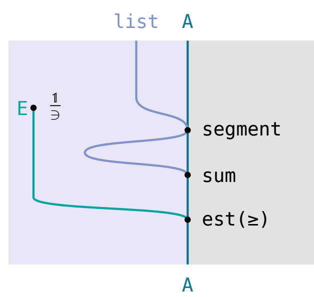
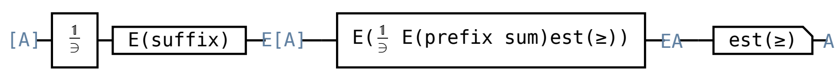
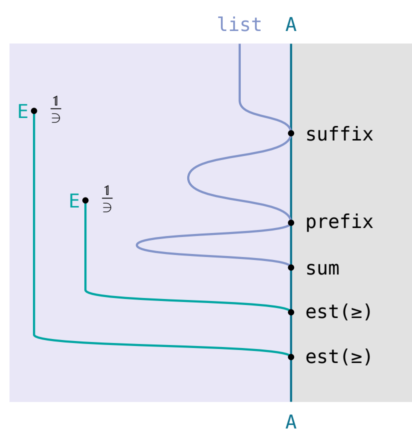
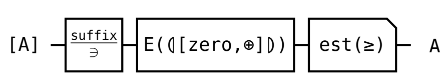
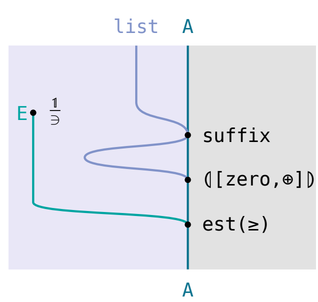
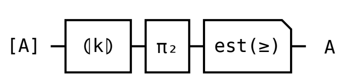
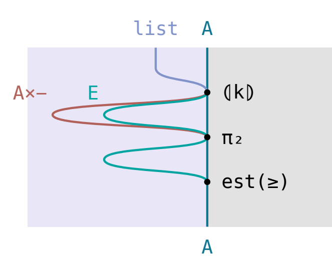
$$[\mathtt{nil},\mathtt{cons}]\langle g,\tfrac{\mathtt{suffix}}{\ni}\ E(g)\rangle = F(\langle g,\tfrac{\mathtt{suffix}}{\ni}\ E(g)\rangle)k \qquad (13.3.4f)$$

$k \triangleq [\mathtt{zero}\langle \mathbb{1},\tfrac{1}{\ni}\rangle,\langle w,\langle w\ \tfrac{1}{\ni},\pi_2\pi_2\rangle\ \mathtt{cup}\rangle]$, $w \triangleq (\mathbb{1}\times\pi_1)\oplus$: the value at the whole list, paired with the set of the values at its suffixes, runs k's recursion.

the equation at that branch	why
$\mathtt{nil}\langle g,\tfrac{\mathtt{suffix}}{\ni}\ E(g)\rangle = \mathtt{zero}\langle \mathbb{1},\tfrac{1}{\ni}\rangle$	`nil` has one suffix, itself, and `g nil=zero`, so the set is the singleton `{zero}`
$\mathtt{cons}\langle g,\tfrac{\mathtt{suffix}}{\ni}\ E(g)\rangle = (\mathbb{1}\times\langle g,\tfrac{\mathtt{suffix}}{\ni}\ E(g)\rangle)\langle w,\langle w\ \tfrac{1}{\ni},\pi_2\pi_2\rangle\ \mathtt{cup}\rangle$	`g(cons(a,x))=a⊕(g x)`, which is `w` reading `g x` off π_1; and the suffixes of `cons(a,x)` are `cons(a,x)` itself, whose value is that same `w`, together with those of `x`, which $\pi_2\pi_2$ carries — so the two sets meet at `cup`

$$\mathtt{mss} = (\!|k|\!)\ \pi_2\ \mathtt{est}(\geq) \qquad (13.3.4g)$$

maximum segment sum problem: $\tfrac{\mathtt{suffix}}{\ni}\ E((\!|[\mathtt{zero},\oplus]|\!))$ expressed as the catamorphism $(\!|k|\!)$, hence `mss` implemented by a linear-time algorithm, $\oplus \triangleq \tfrac{\multimap\ \mathtt{zero}\ \cup\ \mathtt{plus}}{\ni}\ \mathtt{est}(\geq)$ — $\triangleq$ `mss`; k and w — (13.3.4f).

circuit	Hinze–Marsden
 `mss` is the greatest of the segment sums — $\triangleq$ `mss`	
=  (13.3.4b)	
=  $\tfrac{\mathtt{suffix}}{\ni}\ E((\!|[\mathtt{zero},\oplus]|\!))\ \mathtt{est}(\geq)$ the greedy theorem greedy theorem at $R{:=}\geq$, $S{:=}[\mathtt{zero},\multimap\ \mathtt{zero}\ \cup\ \mathtt{plus}]$ — (13.3.4d) is its condition and (13.3.4e) its $\tfrac{S}{\ni}\ \mathtt{est}(\geq)$; its $\sqsubseteq$ is an $=$ because $(\!|[\mathtt{zero},\oplus]|\!)$ is entire and $\tfrac{\mathtt{prefix\ sum}}{\ni}\ \mathtt{est}(\geq)$ simple (13.3.3f)'s last row	
=  $(\!|k|\!)\ \pi_2\ \mathtt{est}(\geq)$ $\triangleq$ `reduce` at (13.3.4f)'s equation, so $(\!|k|\!) = \langle (\!|[\mathtt{zero},\oplus]|\!),\tfrac{\mathtt{suffix}}{\ni}\ E((\!|[\mathtt{zero},\oplus]|\!))\rangle$, of which π_2 is the row above	

one fold builds the $n+1$ running maxima and the final `est(≥)` reads them in one more pass, so `mss` is linear.

§13.3.5. $\mathtt{filter(p)=(\!|[nil,(\pi_1 p \to cons,\pi_2)]|\!)}$

(13.3.5a)

name	definition	type	example	in words
F, α, p, R	as in ≜ `takewhile`			
π_2	π_2 where ≜ `takewhile` has ⊸ `nil`	$A\times[A]\to[A]$	$\pi_2(3,[1,2])=[1,2]$	drop the head, keep the tail
`subseq`	$(\!|[nil,\pi_2 \cup cons]|\!)$ ≜ `subseq`	$[A]\to[A]$	`[3,1,2] subseq [3,2]`	`xs subseq ys`⟺`ys` is `xs` with elements dropped — at each `cons`, drop the head or keep it
S	$[nil,\pi_2 \cup (p\times\mathbb{1})\ cons]$	$F([A])\to[A]$	`(4,[2]) S [2]` and `(4,[2]) S [4,2]`, but `(3,[2]) S [2]` only	`subseq`'s algebra with one extra `p` — drop the head, or keep a head that passes `p`
$\mathbb{1}\sqsubseteq\pi_2 R\ cons^\circ$				the tail is one shorter than the cons, so π_2 loses the `est(R°)` at every step — where ≜ `takewhile`'s loser is `nil`

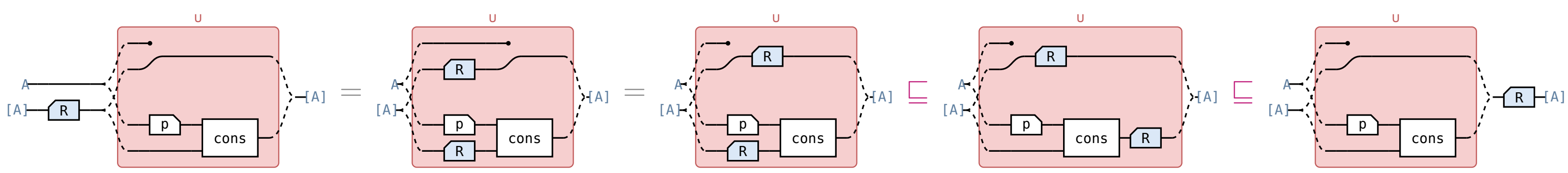
(13.3.5b)

$(\mathbb{1}\times R^\circ)(\pi_2 \cup (p\times\mathbb{1})\ cons)\sqsubseteq(\pi_2 \cup (p\times\mathbb{1})\ cons)R^\circ$

shortening the tail and then taking the step lands inside taking the step and then shortening the result

formula — the `cons` branch of $F(R^\circ)S\sqsubseteq SR^\circ$; **reason** under each circuit

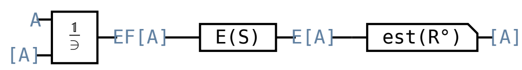
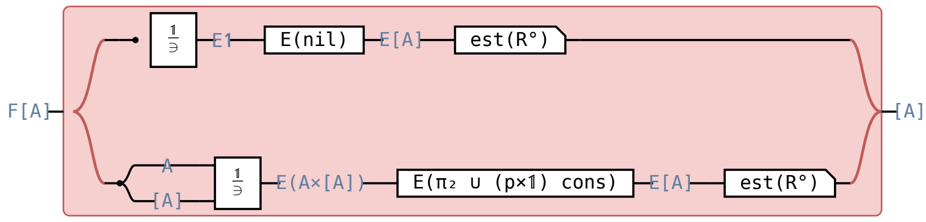
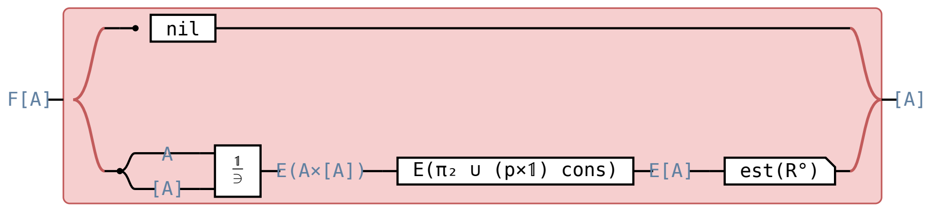
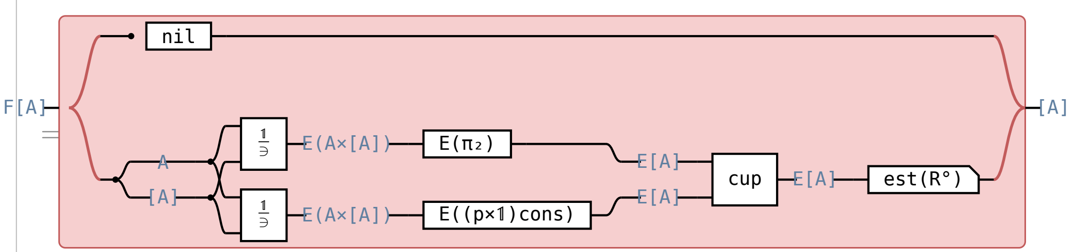
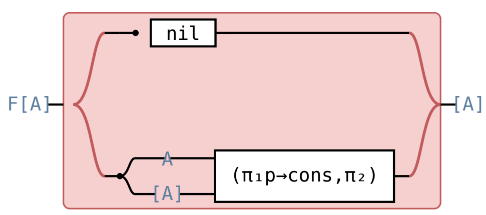
$(\mathbb{1}\times R^\circ)(\pi_2 \cup (p\times\mathbb{1})\ cons)$	$(\mathbb{1}\times R^\circ)\pi_2 \cup (p\times R^\circ)\ cons$	$\pi_2 R^\circ \cup (p\times R^\circ)\ cons$	$\pi_2 R^\circ \cup (p\times\mathbb{1})\ cons\ R^\circ$	$(\pi_2 \cup (p\times\mathbb{1})\ cons)R^\circ$



	$(\mathbb{1}\times R^\circ)(p\times\mathbb{1})=p\times R^\circ$ (1.1a)	$(\mathbb{1}\times R^\circ)\pi_2=\pi_2 R^\circ$ (12.1c)	$(p\times R^\circ)\ cons\sqsubseteq(p\times\mathbb{1})\ cons\ R^\circ$ (13.3.3c)	(1.1a)

the `nil` branch: $nil\sqsubseteq nil\ R^\circ$.

$\frac{\mathtt{S}}{\ni}$ `est(R°)=[nil,(π₁p→cons,π₂)]` the longest of the lists the algebra allows is the **`cons`** where the head passes **`p`**, and the tail where it does not	(13.3.5c)

formula	reason
	
=  $[\frac{\mathtt{nil}}{\ni}$ `est(R°)`, $\frac{\mathtt{π_2\ ∪\ (p×𝟙)\ cons}}{\ni}$ `est(R°)`$]$	`S=[nil,π₂ ∪ (p×𝟙) cons]` ≜ `filter` — and the `%∋` of a coproduct of maps is the coproduct of their `%∋` (11.2.1a)
=  `[nil,` $\frac{\mathtt{π_2\ ∪\ (p×𝟙)\ cons}}{\ni}$ `est(R°)]`	`nil%∋` is the singleton `{nil}`, and `est(R°)` of a singleton is its element because `R°` is reflexive ≜ `est`
= 	$\frac{\mathtt{π_2\ ∪\ (p×𝟙)\ cons}}{\ni}$ `=⟨`$\frac{\mathtt{π_2}}{\ni}$`,` $\frac{\mathtt{(p×𝟙)\ cons}}{\ni}$`⟩ cup` ≜ `cup`
= 	`𝟙⊑π₂R cons°` ≜ `filter` — `xs R cons(a,xs)`, so `est(R°)` returns the `cons` where `p a` puts it in the set and `xs` where the set is `{xs}` ≜ `est`

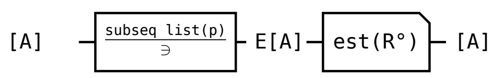
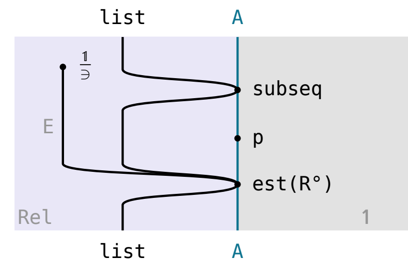
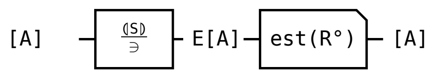
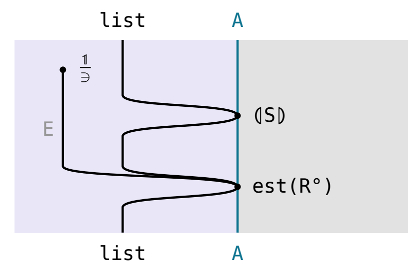
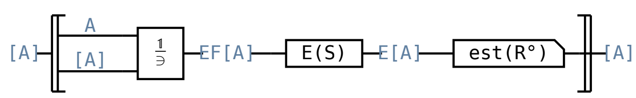
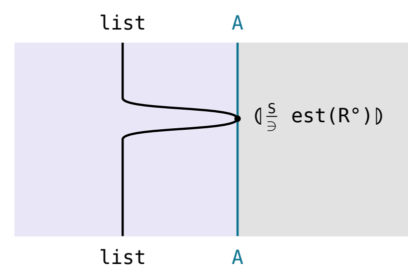
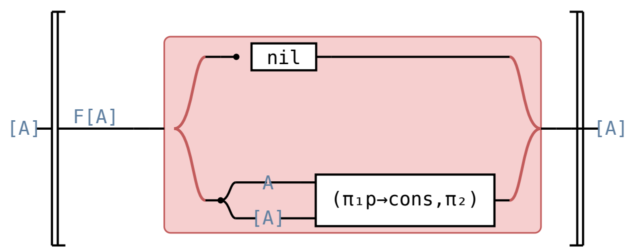
the head is dropped, not the whole tail: that is the one place `π₂` shows against (13.3.3d)'s ⊸ `nil`.

name	definition	type	example	in words
`S`	`[nil,π₂ ∪ (p×𝟙) cons]` ≜ `filter`	`F([A])→[A]`	`(4,[2]) S [2]` and `(4,[2]) S [4,2]`, but `(3,[2]) S [2]` only	`subseq`'s algebra with one extra `p` — drop the head, or keep a head that passes `p`

(13.3.5d)

$\mathtt{filter(p)=⦇[nil,(\pi_1 p{\to}cons,\pi_2)]⦈}$ (13.3.5e)

filter: **`filter(p)` `x`** returns the longest subsequence of **`x`** with the property that all its elements satisfy **`p`**; the catamorphism is the standard program; **`R`** a preorder.

circuit — one wire, its type written along it	**Hinze–Marsden**
 ≜ `subseq`	
=  `subseq list(p)=⦇S⦈` — (13.3.3b)'s header, `subseq` for `prefix`	
⊒  greedy theorem at `R°`, whose hypothesis `F(R°)S⊑SR°` is (13.3.5b)	
=  (13.3.5c)	
= the catamorphism is entire and `filter(p)` simple, so ⊒ is =	

`(subseq list(p))°(subseq list(p))∩R∩R°⊑1` fails — two `p`-subsequences of one list can be of equal length and different — so §13.3.3's uniqueness argument does not transfer. What survives `est(R°)` is the one keeping **exactly** the passing elements: a subsequence that drops a passing element is beaten by the one that keeps it.

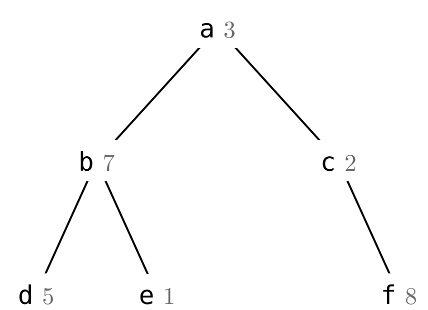
§13.4. Planning a company party

(13.4a)

definition	type	note
`tree A::=node (A,[tree A])`	$\mathcal{A}\to\mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
`F(A,B)=A×[B]`	$\mathcal{A}\times\mathcal{A}\to\mathcal{A}$	The base functor `tree` folds: an employee beside the recursive position, one layer deep.
`rating`	`A→Real`	What one employee is worth as a guest.
`cost≜list(rating) sum`	`[A]→Real`	What a guest list is worth.
`R≜cost≤cost°`	`[A]→[A]`	The preorder the guest list is maximised over.
`choose≜`$\pi_1 \cup \pi_2$	`[A]×[A]→[A]`	Takes one of the two parties a subtree returns.
`include≜(𝟙×(list(`π_2`) concat)) cons`	`F(A,[A]×[A])→[A]`	The party that invites the root, which puts every immediate subtree's root out. A map.
`exclude≜(𝟙×(list(choose) concat))`π_2	`F(A,[A]×[A])→[A]`	The party that leaves the root out, so each subtree is free to choose. Not a map.
`S≜⟨include,exclude⟩`	`F(A,[A]×[A])→[A]×[A]`	The algebra: one step returns both parties of a subtree at once.
`party≜⦇S⦈ choose`	`tree A→[A]`	Every guest list the president's ruling allows.
the specification $\underset{\ni}{\texttt{party}}$ `est(R°)`	`tree A→[A]`	A guest list of greatest total conviviality.

§13.4.1. `include` and `exclude`

(13.4.1a)



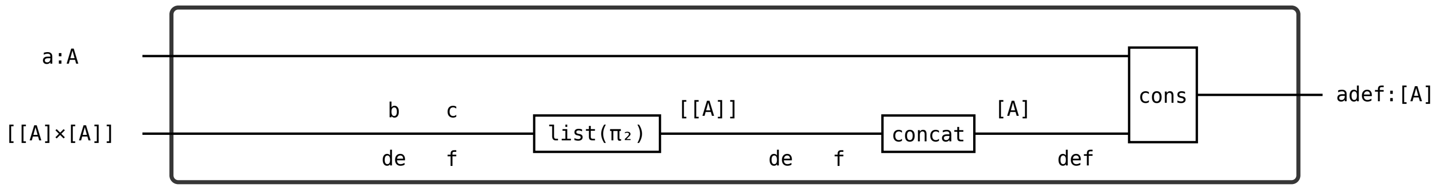
the small grey number is the employee's rating

(13.4.1b)

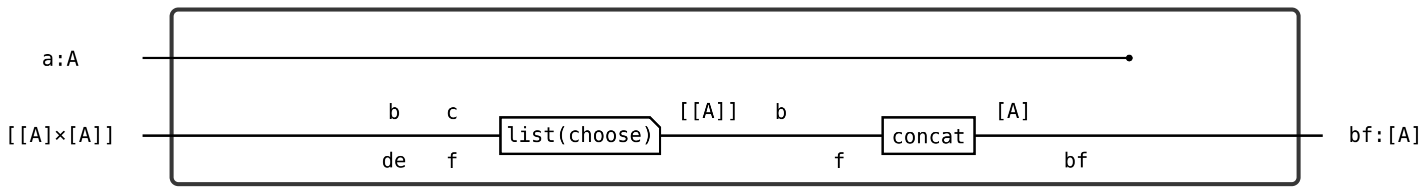
node	the algebra's input `A×[[A]×[A]]`	`include`	`exclude`
`d`, `e`, `f`	`(d,[])`	`[d]`	`[]`
`b`	`(b,[([d],[]),([e],[])])`	`[b]`	`[d,e]`, `[d]`, `[e]`, `[]`
`c`	`(c,[([f],[])])`	`[c]`	`[f]`, `[]`
`a`	`(a,[([b],…),([c],…)])`	`[a]⧺exclude(b)⧺exclude(c)`	`choose(b)⧺choose(c)`

with those ratings, `est(R°)` keeps `([b]=7,[d,e]=6)` at `b`, where `include` wins, and `([c]=2,[f]=8)` at `c`, where `exclude` wins, so at the root `include=[a,d,e,f]=17` beats `exclude=[b,f]=15`, and `choose` takes 17.

include= (13.4.1c)



exclude= (13.4.1d)



(13.4.1e)

```
list(choose) [([d],[]),([e],[])]
  1st item              ([d],[])  choose↦[d]  or  []
  2nd item              ([e],[])  choose↦[e]  or  []
  2×2=4 combinations:   [[d],[e]]   [[d],[]]   [[],[e]]   [[],[]]
  concat flattens:        [d,e]       [d]        [e]        []
```

§13.4.2. list((R×R)°)

(13.4.2a)

```
[  ([b],[d,e])  ,  ([c],[f])  ]
        |              |
        ▼              ▼            list((R×R)°), elementwise
[   ([b],[d])   ,  ([c],[])   ]
```

```
([b],[d,e])
 |    └── exclude: the best party in b's subtree when b does not come
 └─────── include: the best party in b's subtree when b comes
```

(13.4.2b)

element	1st component include	2nd component exclude
1st, ([b],[d,e])	[b] R° [b] 7≥7, reflexivity	[d,e] R° [d] 6≥5, and ([b],[d,e]) is the pair est(R°) keeps at b
2nd, ([c],[f])	[c] R° [c] 2≥2, reflexivity	[f] R° [] 8≥0, and ([c],[f]) is the pair est(R°) keeps at c

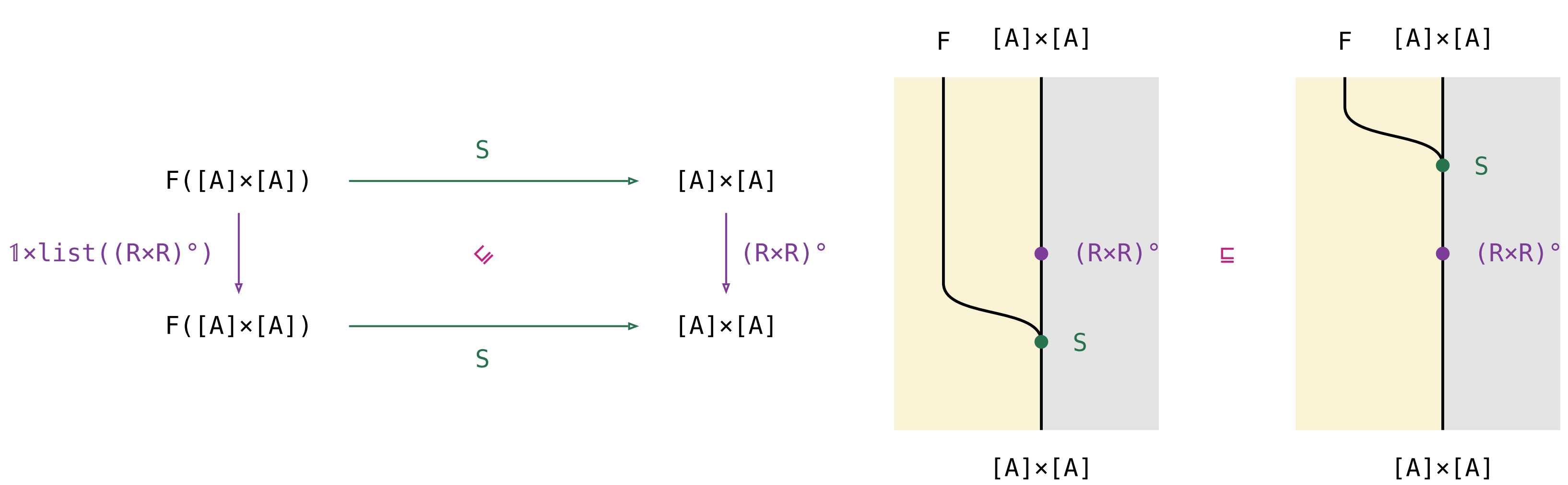
the first component of 𝟙×list((R×R)°) is the root employee.

R≙cost≤cost° is a preorder, so the components that do not move are related by reflexivity — (R×R)° demands a relation in **both** components, it cannot skip one.

R compares cost only, not contents: [f] R° [] is legal though [f] has an element [] does not. That is why the three leaves of §13.4.3's proof all reduce to "cost is a sum".

§13.4.3. (𝟙×list((R×R)°))S⊑S(R×R)° — S : F([A]×[A])→[A]×[A] monotonic on (R×R)°

(13.4.3a)



(𝟙×list((R×R)°))S⊑S(R×R)°

$$\text{tree } A \xrightarrow{(\!|S|\!)_{\ni}\ \text{choose}} E[A] \quad = \quad \text{tree } A \xrightarrow{(\!|S|\!)_{\ni}} E([A]\times[A]) \xrightarrow{E(\text{choose})} E[A] \qquad (13.4.3b)$$

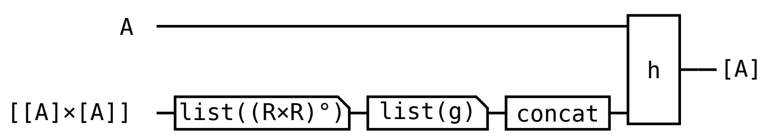
$(\!|S|\!)_{\ni}\ \text{choose} = (\!|S|\!)_{\ni}\ E(\text{choose})$ (10b), absorption,

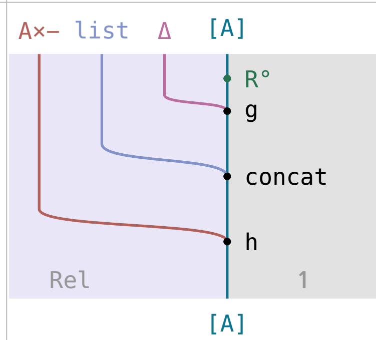
S≜(include,exclude) (13.4.3c)

(𝟙×list((R×R)°))S ⊑ S(R×R)°
(𝟙×list((R×R)°))include ⊑ include R°
(𝟙×list((R×R)°))exclude ⊑ exclude R°

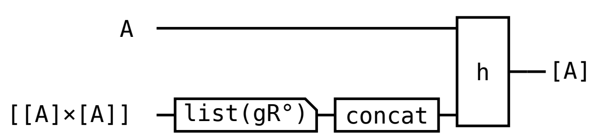
bettering both parties of every subtree before the node's algebra runs gets no further than running it first and bettering the two parties it returns.

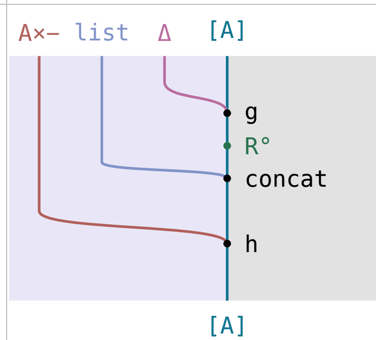
circuit | **Hinze–Marsden**



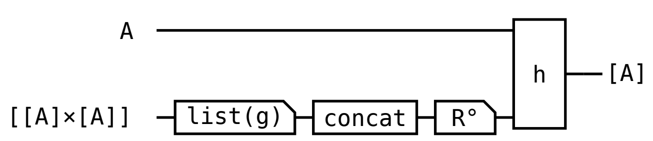


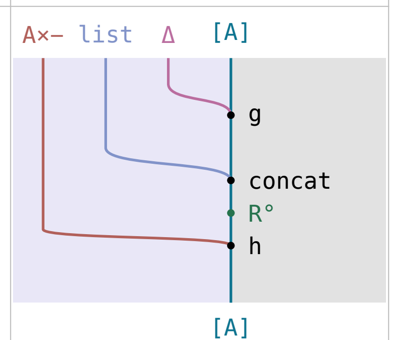
⊑



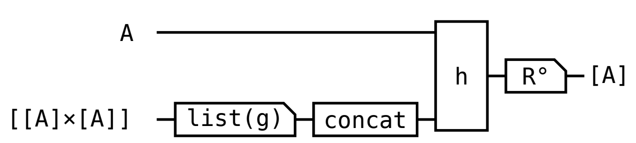


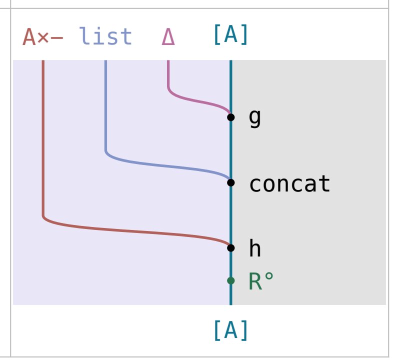
⊑





⊑





g (R×R)°g⊑gR°
g:=π₂ is (R×R)°π₂=(Dom(π₁R°))π₂R°⊑π₂R° , g:=π₁ its mirror (Dom(π₂R°))π₁R°⊑π₁R° — 1 and 4 of (11.1.2b), then Dom⊑𝟙; g:=choose≜π₁ ∪ π₂ is the union of the two (13.2c). list monotonic, ≜ relator

concat list(R°)concat⊑concat R°
a LEAF: no law above it. cost is a sum, so cost(concat xss)=sum(list(cost)xss) and a cheaper part makes a cheaper whole.

h (𝟙×R°)h⊑hR°
a LEAF: cost(cons(a,xs)) =rating(a)+cost(xs), so a cheaper tail makes a cheaper list. For h:=π₂ it is an EQUALITY (𝟙×R°)π₂=(Dom(π₁))π₂R° =π₂R° : π₁ is a map, hence entire, so Dom(π₁)=𝟙 — (5c)

(𝟙×(list(g) concat))h	g	h
include	π₂	cons
exclude	choose	π₂

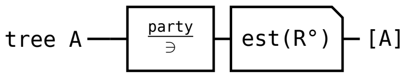
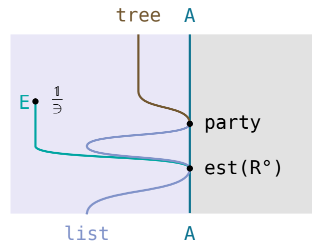
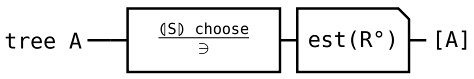
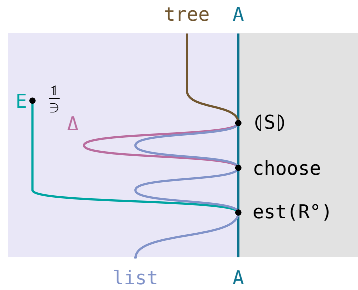
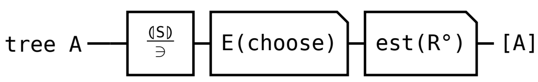
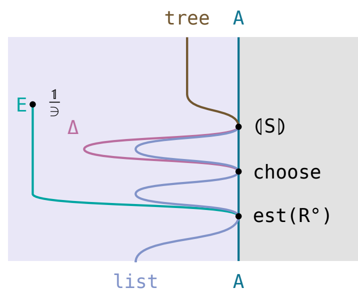
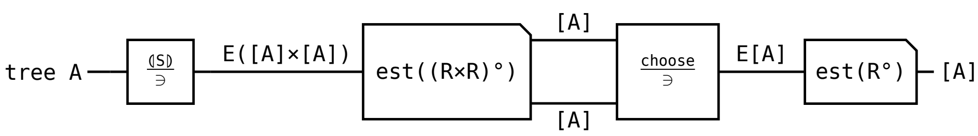
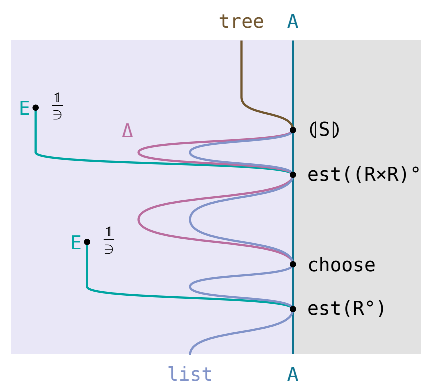
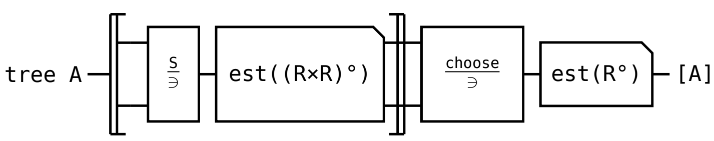
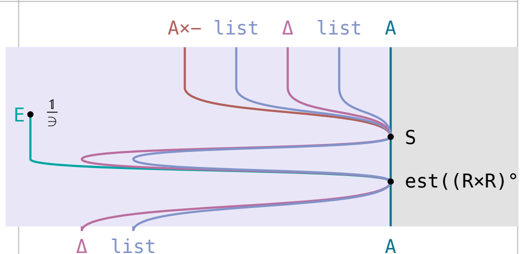
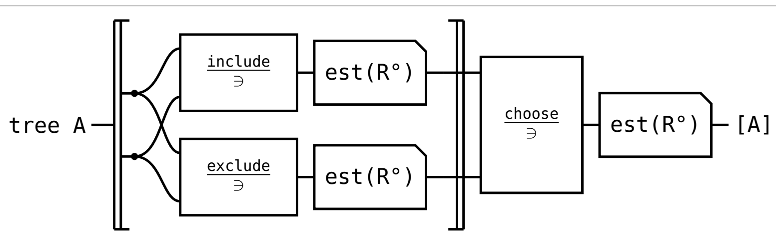
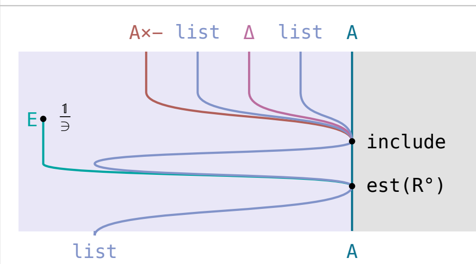
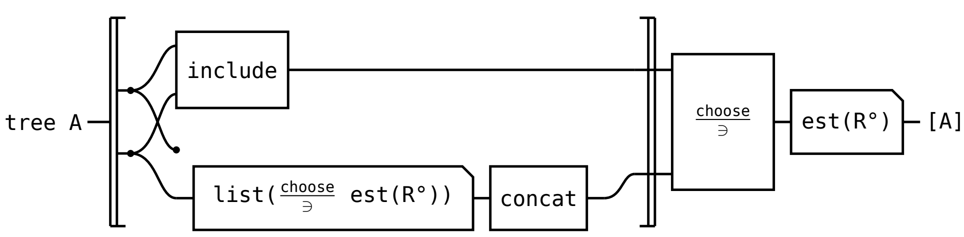
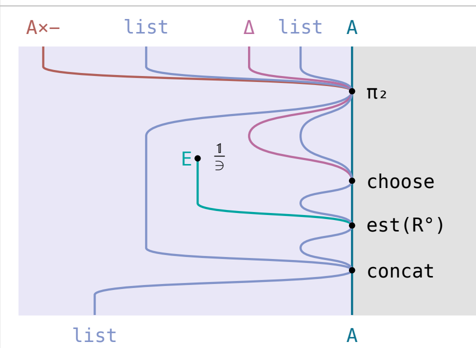
(13.4.3d)

`R≙{(0,0)} : {0,1}→{0,1}`	not entire — undefined at `1`
`(R×R)choose` at `(0,1)`	nothing — `R×R` needs BOTH components related, and `R` is undefined at `1`
`choose R` at `(0,1)`	`0` — `choose` picks the first component `0`, and `R 0 0` holds
`((0,1),0)`	in `choose R`, not in `(R×R)choose`
`(R×R)choose=choose R`	iff `R` is entire — `Dom⊑𝟙` is the `g` row's only inequality step, and it is an equality iff `Dom(R)=𝟙`

§13.4.4. The derivation

$$\frac{\texttt{party}}{\ni}\ \texttt{est}(R^\circ) \sqsupseteq (\!|\langle \texttt{include}, \pi_2\ \texttt{list}(\frac{\texttt{choose}}{\ni}\ \texttt{est}(R^\circ))\ \texttt{concat}\rangle|\!)\ \frac{\texttt{choose}}{\ni}\ \texttt{est}(R^\circ) \qquad (13.4.4a)$$

the best of every guest list the president allows is one pass up the tree, each subtree handing up its best party with its boss in and its best with the boss out, and **choose** taking the better of the two at the root

circuit	**Hinze–Marsden** — outside the ⦇ ⦈ above, inside it below; a fork drawn at one branch
 the specification — ≙ party	
=  party≙⦇S⦈ choose — ≙ party	
=  ⦇S⦈ choose/∋ = ⦇S⦈/∋ E(choose) — (13.4.3b)	
⊒  (R×R)°choose⊑choose R° — (13.4.3c)'s g row,	
⊒  from here ⦇ ⦈ is drawn open — the two bars, with the algebra's own circuit between them; (𝟙×list((R×R)°))S⊑S(R×R)° at (R×R)°, (13.4.3a),	
⊒  ⟨include/∋ est(R°), exclude/∋ est(R°)⟩⊑$^{S}_{\ni}$ est((R×R)°),	
⊒  include a map, est(R°) into each branch,	

§13.5. Shortest paths on a cylinder

definition	type	note
F(A,X)=A+A×X	𝓐×𝓐→𝓐	The base functor: one square of the new column, alone or in front of the path so far.
L=list⁺	𝓐→𝓐	A path is a non-empty list of squares, one per column crossed.
α the initial algebra	F(A,LA)→LA	α(5)=[5] and α(1,[5])=[1,5]: a path is started by one square, or extended by one.
N the n-tuple relator	𝓐→𝓐	One component per row of a column, so the fold carries n answers at once.
R≜sum≤sum°	L Nat→L Nat	The cost of a path, which the cheapest minimises.
setify	NA→EA	setify(1,2,3,4)={1,2,3,4} — which row a component came from is forgotten.
moves	NA→E(NA)	moves(5,6,7,8)={(6,7,8,5),(5,6,7,8),(8,5,6,7)} — rotated up, unrotated, rotated down.
trans	E(NA)→N(EA)	trans{(6,7,8,5),(5,6,7,8),(8,5,6,7)} =({6,5,8},{7,6,5},{8,7,6},{5,8,7}) — row k collects rows k-1, k, k+1.
zip	F(NA,NB)→NF(A,B)	zip((1,2,3,4),({[5]},{[6]},{[7]},{[8]}))=((1,{[5]}),(2,{[6]}),(3,{[7]}),(4,{[8]})).
cp≜ $\frac{F(𝟙,∋)}{∋}$	F(A,EB)→E(F(A,B))	cp(1,{[5],[6],[8]})={(1,[5]),(1,[6]),(1,[8])}, and P(α) of that is {[1,5],[1,6],[1,8]}.
generate≜F(𝟙,moves trans N(union)) zip N(cp P(α))	F(NA,N(E(LA)))→N(E(LA))	generate((1,2,3,4),({[5]},{[6]},{[7]},{[8]})) is worked out in (13.5.1b).
paths≜⦇generate⦈ setify union	L N Nat→E(L Nat)	paths[(1,2,3,4),(5,6,7,8)] is the union of (13.5.1b)'s four sets: 12 paths, 3 from each entry row.
the specification paths est(R)	L N Nat→L Nat	A cheapest path from the entry side to the exit side.

(13.5a)

§13.5.1. N(E(L A))

```
1  5
2  6      [(1,2,3,4),(5,6,7,8)] : L N Nat
3  7
4  8
```
(13.5.1a)

```
⦇generate⦈[(5,6,7,8)] = ({[5]},{[6]},{[7]},{[8]})
```
(13.5.1b)

```
generate((1,2,3,4),({[5]},{[6]},{[7]},{[8]})) =
    ( {[1,5],[1,6],[1,8]},
      {[2,5],[2,6],[2,7]},
      {[3,6],[3,7],[3,8]},
      {[4,5],[4,7],[4,8]} )
```

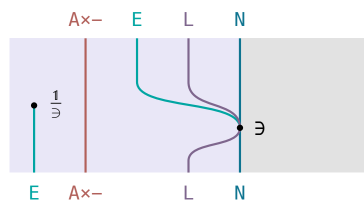
(13.5.1c)

N	the 4-tuple, one component per row of the entry column
E	the set of paths that can start in that row
L	a path is the non-empty list of the squares it crosses

row 1′s set has no [1,7] — from row 1 only rows 4, 1 and 2 are reachable — and it does have [1,8], because the cylinder glues the bottom row to the top. That is moves trans N(union) of ≜ paths: component k collects the paths of rows k-1, k, k+1.

§13.5.2. The cross product cp≜$\frac{\mathtt{F(𝟙,∋)}}{∋}$

(13.5.2a)



(13.5.2b)

type	note
F(A,–)=A+A×–	the picture draws the A×– summand: a sum is a case split, which one wire cannot carry
B=L N	one path: the non-empty list of squares it crosses, N≜Nat
∋:E(L N)→L N	the ∋ inside F(𝟙,∋): a set of paths, one of them
F(𝟙,∋)=𝟙+𝟙×∋	:A+A×E B→A+A×B, the relator acting on each summand
∋:E(A+A×B)→A+A×B	the ∋ under the bar: a set of cells, one of them — a different ∋
cp:A+A×E B→E(A+A×B)	$\frac{R}{\ni}$ turns R:X→Y into X→E Y
cp=$\frac{1}{\ni}$ E(F(𝟙,∋))	what the picture draws: the unit makes the outer E, leaving one ∋ — (10b)

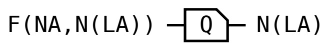
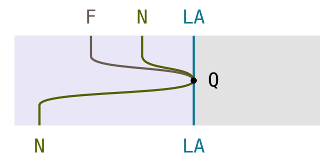
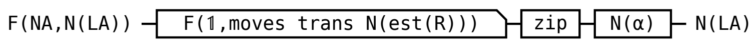
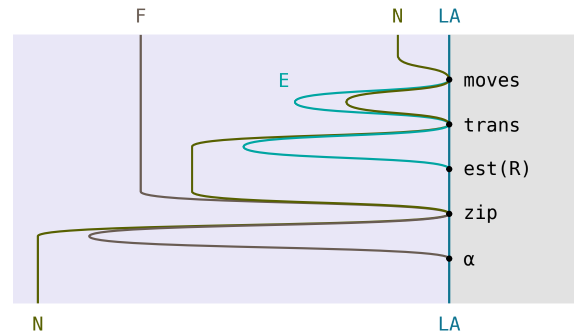
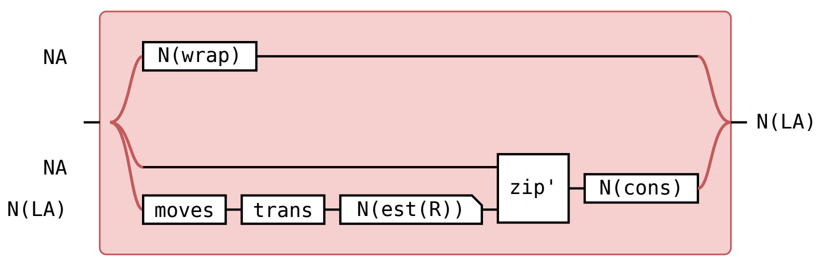
(13.5.2c)

```
w      = (1,{[5],[6],[8]})          : A×E(L N)    the right summand, A×E B
w (𝟙×∋) (1,x)  ⟺  x ∈ {[5],[6],[8]}   : A×L N      𝟙 keeps 1, ∋ picks one path
cp w   = {(1,[5]),(1,[6]),(1,[8])}   : E(A+A×B)    the three of them, collected
cp P(α) w = {[1,5],[1,6],[1,8]}      : E B         q: 1 ,/: 5 6 8 – `/:` is cp, `,` is α
```

§13.5.3. Q=F(𝟙,moves trans N(est(R))) zip N(α)

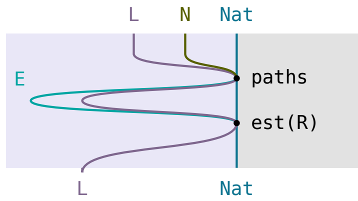
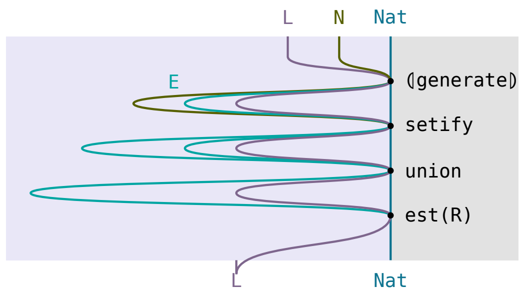
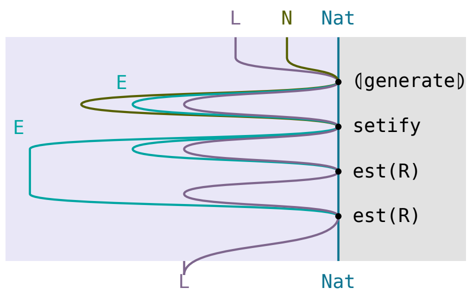
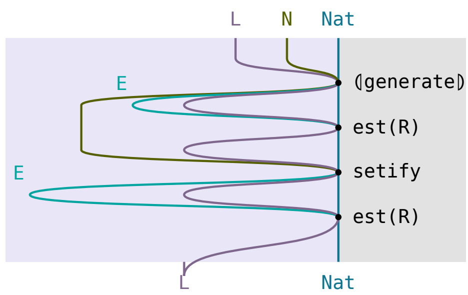
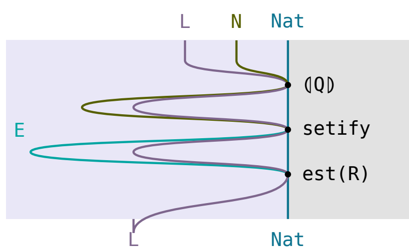
Q=[N(wrap),(𝟙×moves trans N(est(R))) zip' N(cons)] (13.5.3a)

at the last column **Q** starts one path per row, and at each earlier one it puts each square in front of the cheapest of the three kept paths it can step to — the algebra (13.5.3b)‘s last step folds

circuit — the fork is F(NA,N(LA))=NA+NA×N(LA)	**Hinze–Marsden**
	
=  the fusion condition (13.5.3c) read as a definition,	
=  zip=𝟙+zip', α=[wrap,cons]	

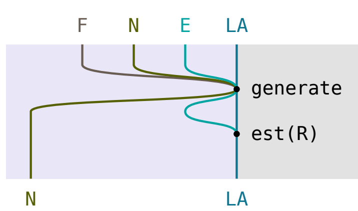
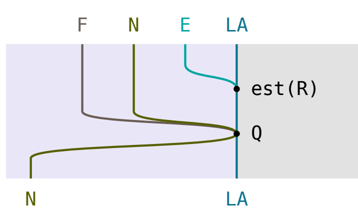
`paths est(R)⊒⦇Q⦈ setify est(R)` (13.5.3b)

shortest paths on a cylinder: the dynamic programming approach to exhaustive search allows a path of least cost to be found in `O(n×m)` time; `Q` is (13.5.3a)'s algebra.

circuit	Hinze–Marsden
`L N Nat ─[paths]─[est(R)]─ L Nat`	
= `L N Nat ─[⦇generate⦈]─[setify]─[union]─[est(R)]─ L Nat` ≜ `paths` at `paths`	
⊒ `L N Nat ─[⦇generate⦈]─[setify]─[P(est(R))]─[est(R)]─ L Nat` (13.1b) at `union`, `R` a preorder	
⊒ `L N Nat ─[⦇generate⦈]─[N(est(R))]─[setify]─[est(R)]─ L Nat` `setify` lax natural	
⊒ `L N Nat ─[⦇Q⦈]─[setify]─[est(R)]─ L Nat` fusion at (13.5.3c)	

`generate N(est(R))⊒F(𝟙,N(est(R)))Q` (13.5.3c)

fusion: the condition for fusion in (13.5.3b)'s last step, used to derive a definition of `Q`.

circuit	Hinze–Marsden
`F(NA,N(E(LA))) ─[generate]─[N(est(R))]─ N(LA)`	
⊒ `F(NA,N(E(LA))) ─[F(𝟙,N(est(R)))]─[Q]─ N(LA)` (7.13), then `zip`, `trans`, `moves` lax natural	

(7.13) is `F(𝟙,est(R))α⊑cp P(α) est(R)`, (13.3.1c) at the map `α` with $\frac{\texttt{F(𝟙,∋)α}}{\ni}$ `=cp P(α)`: extending every path in a set and then taking a minimum is beaten by extending one minimum. It is the crux here, not the greedy theorem.

§13.6. The security van problem

definition	type	note	(13.6a)
`R≙length≤length°`	`[[Int]]→[[Int]]`	The order the schedule is minimised over: fewer van visits is better.	
`ceiling≙` $\frac{\texttt{prefix sum}}{\ni}$ `est(≥)`	`[Int]→Int`	The highest the bank's balance reaches over a stretch of transactions.	
`floor≙` $\frac{\texttt{prefix sum}}{\ni}$ `est(≤)`	`[Int]→Int`	The lowest it reaches, so `ceiling-floor` is the cash the stretch has to carry.	

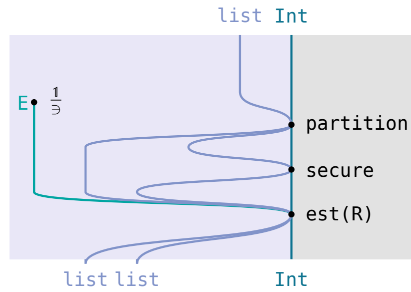
`secure`
the coreflexive on `x` with
`bmax(ceiling x,ceiling x-floor x)≤N`
	`[Int]→[Int]`	The stretches one van visit can serve: some starting reserve keeps the cash between `0` and `N`.	

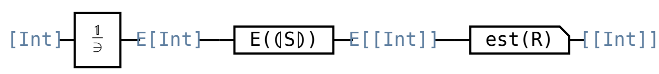
`ok` the coreflexive on `(a,xs)` with `xs` non-empty and `[a]++head xs` secure	`Int×[[Int]]` `→Int×[[Int]]`	The test the final program runs, in place of `old`'s `secure`.	
`new≙(wrap×𝟙) cons`	`Int×[[Int]]→[[Int]]`	Calls the van: the transaction opens a segment of its own.	
`glue≙(𝟙×cons°) assocl (cons×𝟙) cons`	`Int×[[Int]]→[[Int]]`	Puts the transaction on the front of the first segment.	
`old≙(𝟙×cons°) assocl` `((cons secure)×𝟙) cons`	`Int×[[Int]]→[[Int]]`	`glue` restricted to a first segment that stays secure.	
`partition=⦇[nil,new ∪ glue]⦈`	`[Int]→[[Int]]`	Every splitting of the transactions into consecutive segments.	
`S≙[nil,new ∪ old]`	`1+Int×[[Int]]→[[Int]]`	The algebra whose fold is every splitting of the transactions into secure segments.	
`partition list(secure)=⦇S⦈`	`[Int]→[[Int]]`	Fusion: keeping only secure segments is what turns `glue` into `old`.	
`H≙(head prefix° head°) ∪ (nil° nil)`	`[[Int]]→[[Int]]`	One schedule's first segment is a prefix of the other's, or both are empty.	
`R;H≙R∩(R°⇒H)`	`[[Int]]→[[Int]]`	The refinement of `R` that makes both halves of `S` monotonic.	
$\lvert\texttt{R}\rvert$`≙R∩¬R°`	`[[Int]]→[[Int]]`	The strict part `R` splits into: `R;H=`$\lvert\texttt{R}\rvert$` ∪ (R∩H)`.	
the specification $\frac{\texttt{partition list(secure)}}{\ni}$ `est(R)`	`[Int]→[[Int]]`	A schedule with the fewest secure segments.	

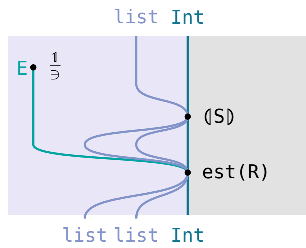
$$\frac{\texttt{partition list(secure)}}{\ni}\ \texttt{est(R)}\sqsupseteq(\![\texttt{[nil,(ok→glue,new)]}]\!) \qquad (13.6b)$$

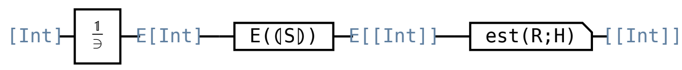
the fewest secure segments the transactions can be cut into are one pass along them, the next transaction glued onto the open segment wherever that segment stays secure and the van called where it does not

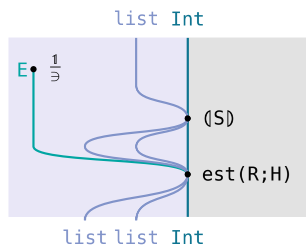
<table>
<tr><th>circuit</th><th>Hinze–Marsden</th></tr>
<tr><td>

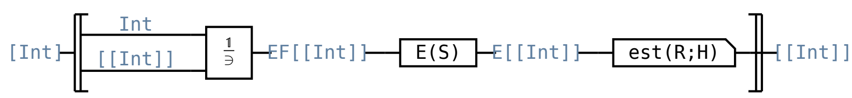

$\frac{\texttt{partition list(secure)}}{\ni}$ est(R)
the specification — ≜ secure
</td><td>


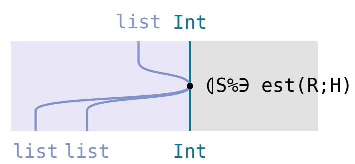
</td></tr>
<tr><td>


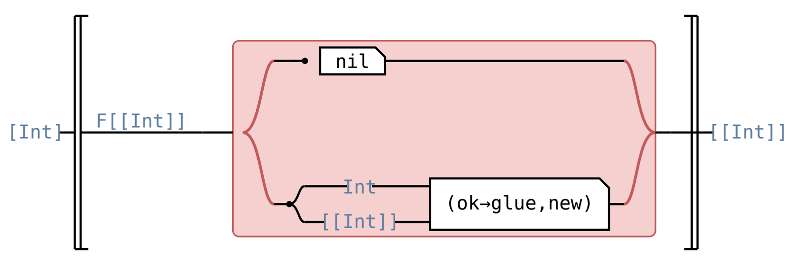
= $\frac{⦇S⦈}{\ni}$ est(R)
partition list(secure)=⦇S⦈ — ≜ secure, fusion at secure
prefix⊑prefix secure
</td><td>


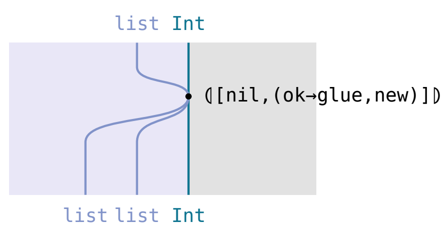
</td></tr>
<tr><td>


⊒ $\frac{⦇S⦈}{\ni}$ est(R;H)
R;H⊑R — ≜ secure; (7.15) (𝟙×R)old⊑(new ∪ old)R is FALSE, the shorter partition need not stay secure, where (7.14) (𝟙×R)new⊑(new ∪ old)R holds,
</td><td>


</td></tr>
<tr><td>


⊒ ⦇$\frac{S}{\ni}$ est(R;H)⦈
greedy theorem at R;H, its hypothesis F(R;H)S⊑S(R;H) the old half (7.17) — (13.6c) — and the new half (7.16), which rests on (7.18) (𝟙×⊤)new⊑new H
</td><td>


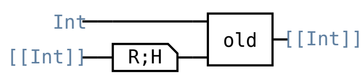
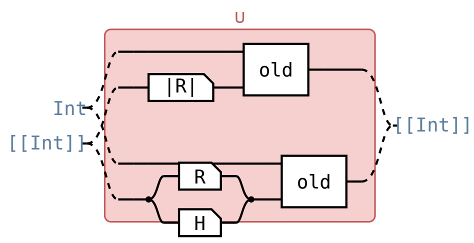
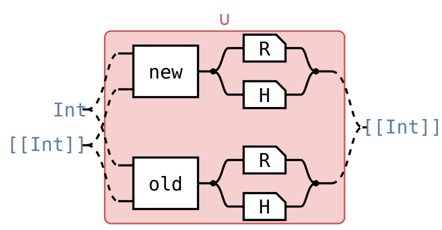
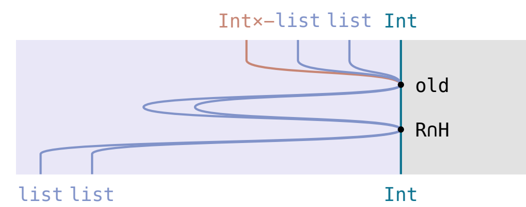
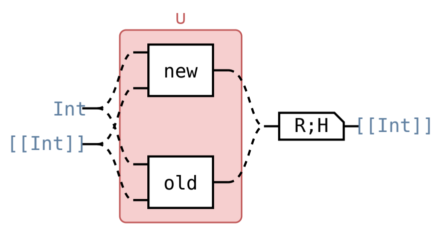
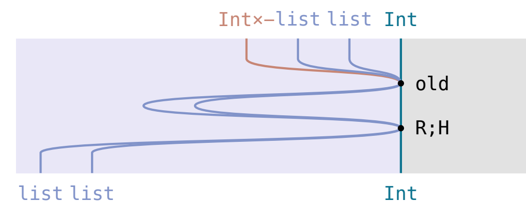
</td></tr>
<tr><td>
⊒ 

⦇[nil,(ok→glue,new)]⦈
old⊑new (R;H)°: old returns the shorter result wherever it returns one, and ok is where it does
</td><td>


</td></tr>
</table>

$$(\mathbb{1}\times(R;H))\mathtt{old}\sqsubseteq(\mathtt{new}\cup\mathtt{old})(R;H)$$ (13.6c)

gluing the transaction onto a better schedule for the rest gets no further than gluing it on, or calling the van, and bettering the whole schedule after,

circuit — the `old` branch of each union	**Hinze–Marsden**
	 the `old` operand of `new ∪ old`, in every row
=  `(𝟙×|R|)old ∪ (𝟙×(R∩H))old` `R;H=|R| ∪ (R∩H)` — ≙ `secure`, `∪` distributes,	
⊑  `new (R∩H) ∪ old (R∩H)` (7.19) and (7.20) on `|R|`, (7.21) on `R∩H`	
⊑  `(new ∪ old)(R;H)` `X∩Y⊑X;Y`, converses	

§14. Thinning Algorithms

§14.1. Thinning

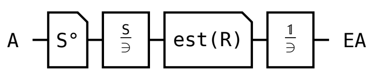
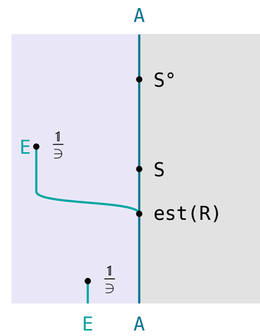
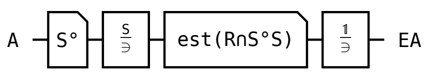
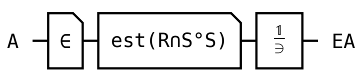
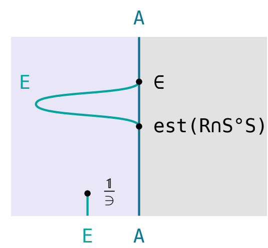
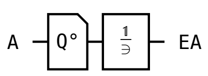
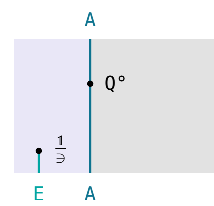
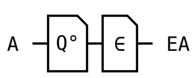
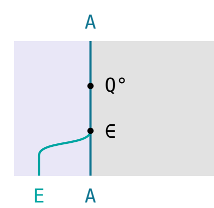
(14.1a)

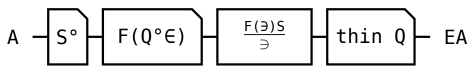
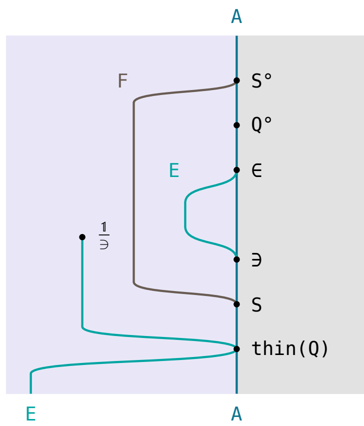
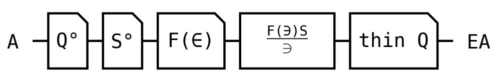
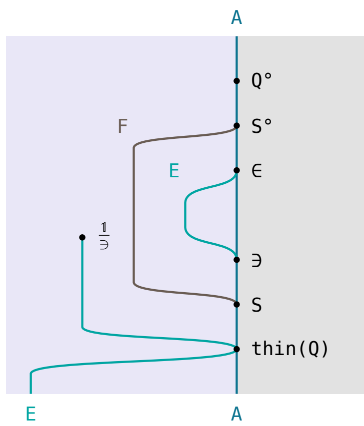
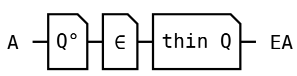
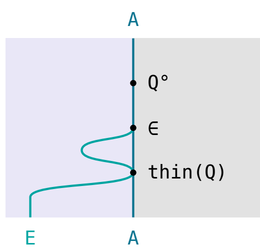
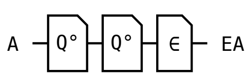
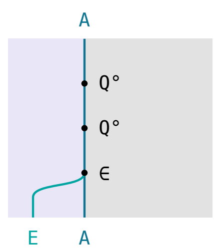
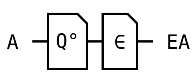
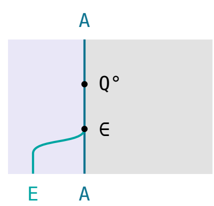
For `Q : A→A`, `thin Q≙(∋/∋)∩(∈\(Q°∈)) : EA→EA` (8.1).

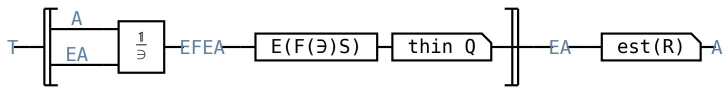
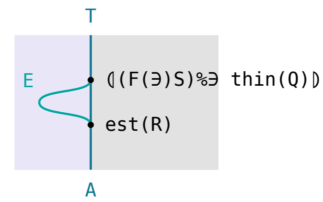
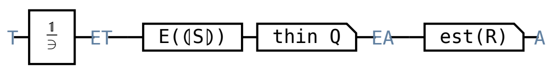
`ys (thin Q) xs⟺xs⊆ys∧(∀a∈ys. ∃b∈xs. b Q a)`

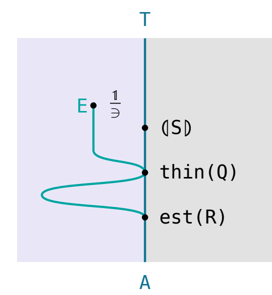
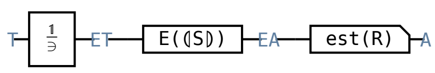
(14.1b)

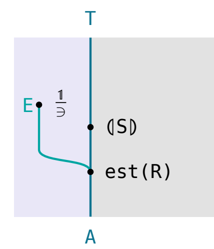
the law	what it says
X⊑ $\frac{S}{\ni}$ thin Q⟺X∋⊑S and S°X⊑Q°∈	everything kept is an S-value, and every S-value has a Q-lower bound among the kept ones
Q⊑R⟹thin Q⊑thin R	the fewer pairs Q relates, the fewer subsets count as thinnings
𝟙⊑thin Q, and thin Q is a preorder if Q is	keeping everything is always a legal thinning
est(R)=thin Q est(R) Q⊑R, both preorders	**thin-introduction**: thinning first cannot lose an R-minimum
thin Q⊒est(Q) $\frac{\mathbb{1}}{\ni}$ (8.2)	**thin-elimination**: keeping one element is a thinning, but its domain is the sets est(Q) is defined on
$\frac{S}{\ni}$ thin Q⊒ $\frac{S}{\ni}$ est(R) $\frac{\mathbb{1}}{\ni}$ (8.3), R∩(S°S)⊑Q — (14.1c)	the usable variant: R need only refine Q between values S gives one argument
union thin Q⊒P(thin Q) union (8.4)	thinning each member set is a thinning of the union

$\frac{S}{\ni}$ **thin Q⊒**$\frac{S}{\ni}$ **est(R)** $\frac{1}{\ni}$ keeping one **R**-least of the **S**-values is a thinning, once **R** refines **Q** between the values **S** gives one argument — **R∩(S°S)⊑Q**, **Q** a preorder		(14.1c)
circuit — one wire, A to EA	**Hinze–Marsden**	
$\frac{S}{\ni}$ est(R) $\frac{1}{\ni}$∋⊑S and S°$\frac{S}{\ni}$ est(R) $\frac{1}{\ni}$⊑Q°∈ (14.1b) at X≜$\frac{S}{\ni}$ est(R) $\frac{1}{\ni}$		
⇐  the first is $\frac{1}{\ni}$∋=𝟙 — (10b) — then $\frac{S}{\ni}$ est(R)=S∩(S°\R°)⊑S — (13.1b); the second is this chain, −⊑Q°∈		
=  $\frac{S}{\ni}$ est(R)=$\frac{S}{\ni}$ est(R∩S°S) — (13.1b)		
⊑  S°$\frac{S}{\ni}$⊑∈, since $\frac{S}{\ni}$∋=S with $\frac{S}{\ni}$ a map — (10b)		
⊑  ∈ est(R∩S°S)⊑(R∩S°S)° — (13.1.1a) at X≜est(R∩S°S), conversed — then R∩(S°S)⊑Q		
⊑  $\frac{1}{\ni}$⊑∈, since $\frac{1}{\ni}$∋=𝟙 with $\frac{1}{\ni}$ a map — (10b)		

$(\!\mid\frac{F(\ni)S}{\ni}\ \mathtt{thin}\ Q\mid\!)\sqsubseteq\frac{(\!\mid S\mid\!)}{\ni}\ \mathtt{thin}\ Q$ thinning at every step of the reduce is a thinning of the whole candidate set — S monotonic on Q, Q a preorder	(14.1d)
circuit — one wire, A to EA	**Hinze–Marsden**
$(\!\mid\frac{F(\ni)S}{\ni}\ \mathtt{thin}\ Q\mid\!)\ni\sqsubseteq(\!\mid S\mid\!)$ and $(\!\mid S\mid\!)^\circ(\!\mid\frac{F(\ni)S}{\ni}\ \mathtt{thin}\ Q\mid\!)\sqsubseteq Q^\circ\in$ (14.1b) at $X\triangleq(\!\mid\frac{F(\ni)S}{\ni}\ \mathtt{thin}\ Q\mid\!)$, $(\!\mid S\mid\!)$ for its S	
$\Leftarrow$  the first by fusion; (11.6.4b) at the bound $Q^\circ\in$ reduces the second to $-\sqsubseteq Q^\circ\in$	
$\sqsubseteq$  $S^\circ F(Q^\circ)\sqsubseteq Q^\circ S^\circ$ — (13.3b) at S, conversed; $F(R)^\circ=F(R^\circ)$ — (11b)	
$\sqsubseteq$  $S^\circ F(\in)\frac{F(\ni)S}{\ni}\sqsubseteq\in$, since $\frac{F(\ni)S}{\ni}\ni=F(\ni)S$ with $\frac{F(\ni)S}{\ni}$ a map — (10b)	
$\sqsubseteq$  $\in\ \mathtt{thin}\ Q\sqsubseteq Q^\circ\in$, the $\in\backslash(Q^\circ\in)$ half of $\triangleq$ $\mathtt{thin}$ — (1.1a)	
$=$  $Q^\circ Q^\circ=Q^\circ$, Q a preorder	

<table>
<tr><th colspan="2">$\llparenthesis \frac{\mathtt{F(\ni)S}}{\ni}\ \mathtt{thin\ Q} \rrparenthesis\ \mathtt{est(R)} \sqsubseteq \frac{\llparenthesis \mathtt{S} \rrparenthesis}{\ni}\ \mathtt{est(R)}$
the thinning fold refines the optimisation problem itself — S monotonic on Q, Q⊑R, both preorders</th></tr>
<tr><th>circuit — one wire, T to A</th><th>Hinze–Marsden</th></tr>
<tr><td>
</td><td>
</td></tr>
<tr><td>⊑ 

thinning theorem</td><td>
</td></tr>
<tr><td>= 

est(R)=thin Q est(R) — (14.1b), Q⊑R</td><td>
</td></tr>
</table>

(14.1e)

§14.2. Paths in a layered network

(14.2a)

$\mathtt{F(A,X)=A+A{\times}X}$, $\mathtt{L=list^{+}}$ with initial algebra $\mathtt{\alpha \triangleq [wrap,cons]\ :\ F(A,LA){\to}LA}$.

$\mathtt{wrapz \triangleq \langle wrap,zero\rangle}$, $\mathtt{consw\ (a,(xs,n))=(cons\ (a,xs),wt\ (a,head\ xs)+n)}$.

$\mathtt{cost \triangleq \llparenthesis [wrapz,consw] \rrparenthesis \pi_2}$, $\mathtt{\llparenthesis [wrapz,consw] \rrparenthesis = \langle \mathbb{1},cost\rangle}$, $\mathtt{R \triangleq cost \le cost^\circ}$.

$\mathtt{Q \triangleq R \cap (head\ head^\circ)}$, $\mathtt{S \triangleq F(\mathbb{1},\ni)\alpha}$, $\frac{\mathtt{F(\ni,\mathbb{1})}}{\ni}\mathtt{=\mathbb{1}+cpl}$, $\frac{\mathtt{F(\mathbb{1},\ni)}}{\ni}\mathtt{=\mathbb{1}+cpr}$, $\mathtt{step \triangleq cpr\ P(cons)\ est(R)}$.

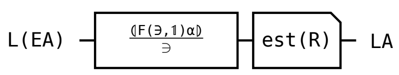
(14.2b)

the law	**what it says**
$\mathtt{F(\ni,Q)\alpha \sqsubseteq F(\ni,\mathbb{1})\alpha Q}$	$\mathtt{F(\ni,\mathbb{1})\alpha}$ is monotonic on Q; on R it is not, since the next edge can cost arbitrarily much
$\mathtt{S\ head \sqsubseteq [\mathbb{1},\pi_1]}$ $\mathtt{S \triangleq F(\mathbb{1},\ni)\alpha}$	S head is simple, which gives $\mathtt{R \cap (S^\circ S) \sqsubseteq Q}$: between two paths S builds from one argument, equal cost and equal head already means Q

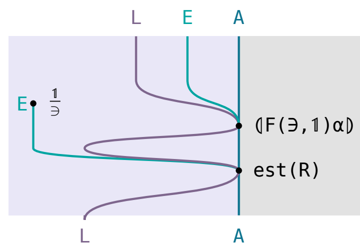
$\frac{(\!|F(\ni,\mathbb{1})\alpha|\!)}{\ni}$ est(R)⊒⦇[P(wrap),cpl P(step)]⦈ est(R) (14.2c)

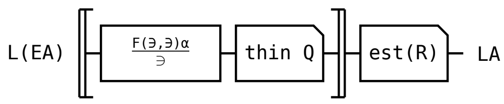
a least-cost path in a layered network, as a fold over the layers

circuit — one wire, L(EA) to LA; the algebra inside the functorial box	**Hinze–Marsden**

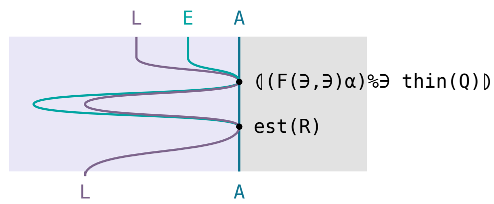


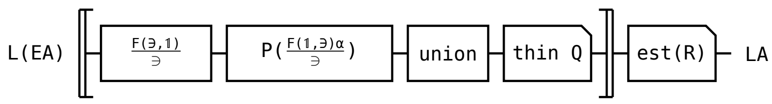
= $\frac{L(\ni)}{\ni}$ est(R)



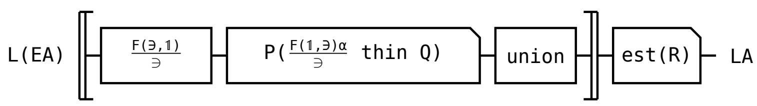


⊒ (14.1e), at F(∋,𝟙)α monotonic on Q — (14.2b).

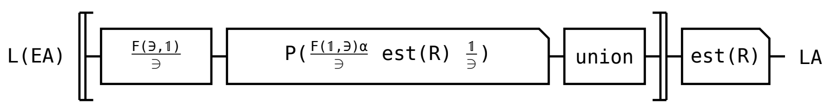




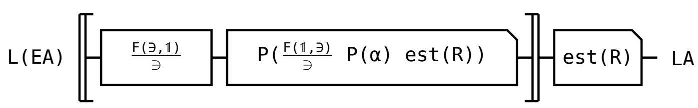
= F(∋,∋)=F(∋,𝟙)F(𝟙,∋); $\frac{F(\ni,\mathbb{1})F(\mathbb{1},\ni)\alpha}{\ni}$=$\frac{F(\ni,\mathbb{1})}{\ni}$ P($\frac{F(\mathbb{1},\ni)\alpha}{\ni}$) union.



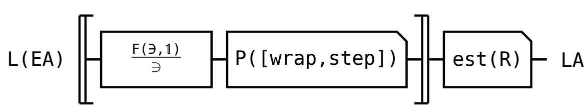
⊒ union thin Q⊒P(thin Q) union — (14.1b).



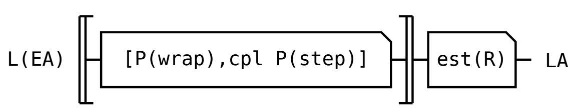
⊒ $\frac{S}{\ni}$ thin Q⊒$\frac{S}{\ni}$ est(R) $\frac{1}{\ni}$ — (14.1b) at S≙F(𝟙,∋)α, Rn(S°S)⊑Q — (14.2b).



= P($\frac{1}{\ni}$) union=𝟙; α a map, so $\frac{F(\mathbb{1},\ni)\alpha}{\ni}$=$\frac{F(\mathbb{1},\ni)}{\ni}$ P(α).



= $\frac{F(\mathbb{1},\ni)}{\ni}$ P(α) est(R)=[wrap,step] — ≙ cost



= $\frac{F(\ni,\mathbb{1})}{\ni}$ =𝟙+cpl — ≙ cost

§14.3. Implementing thin

`setify : [A]→EA`, `cup : EA×EA→EA`, `cp(F)≜` $\frac{\mathtt{F(\ni)}}{\ni}$, `listcp(F) : F([A])→[FA]`, `sort P≜setify° ordered P` for `P` a connected preorder. (14.3a)

`thinlist Q` is any `thinlist Q⊑subseq` with `thinlist Q setify⊑setify thin Q`; one is `⦇[nil,bump Q]⦈`, `bump Q (a,[])=[a]`, `bump Q (a,[b]⧺xs)=(b Q a→[a]⧺xs,a Q b→[b]⧺xs,[a]⧺[b]⧺xs)`.

Binary thinning data: $S=(f_1p_1)\cup(f_2p_2)$ with p_1, p_2 coreflexive; `Q` a preorder with `Q⊑R` and both f_1p_1, f_2p_2 monotonic on `Q`; `P` a connected preorder with both f_1, f_2 monotonic on `P`; g_i`≜list(`f_i`) filter(`p_i`)`.

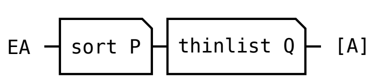
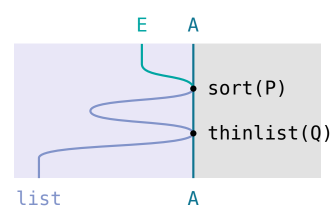
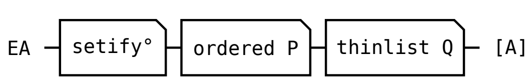
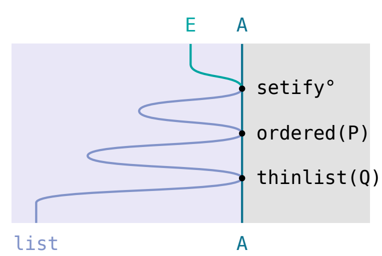
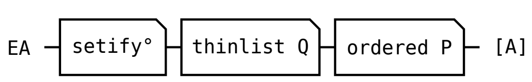
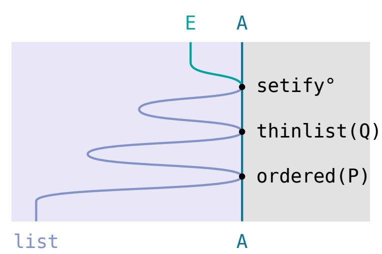
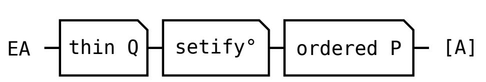
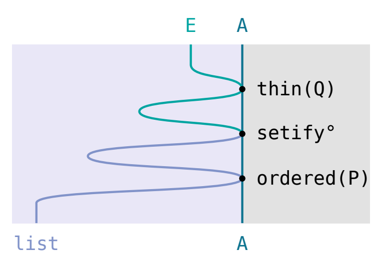
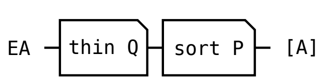
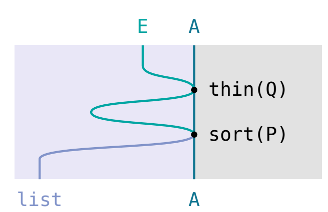
(14.3b)

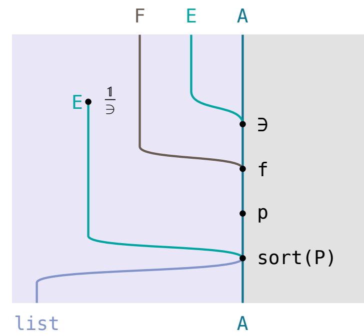
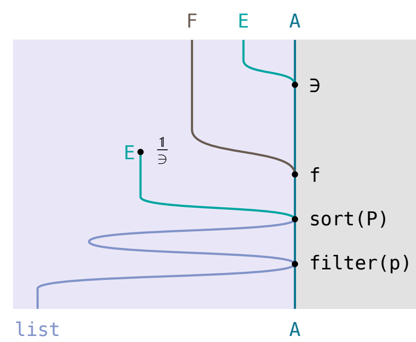
the law	what it says
`thinlist Q xs=[minlist Q xs]` (8.5), `Q` connected, `xs` non-empty	what thinning should come to when it can: one element
`sort P thinlist Q⊑thin Q sort P` (8.6) — (14.3c)	thinning a sorted list is a thinning of the set — this is what `thinlist Q⊑subseq` buys
`sort P minlist Q⊑est(Q)` (8.7)	a minimum of the sorted list is a minimum of the set
`sort(fPf°) list(f)⊑P(f) sort P` (8.8)	shunt a function through a sort
`sort P filter(p)⊑E(p) sort P` (8.9), `p` coreflexive	filtering a sorted list sorts the restricted set
`(sort P×sort P) merge P⊑cup sort P` (8.10)	merging two sorted lists sorts their union
`F(sort P) listcp(F)⊑cp(F) sort(FP)` (8.11), `F` linear	`listcp(F)` is the list implementation of the cartesian product `cp(F)`

(14.3c)

`sort P thinlist Q⊑thin Q sort P`

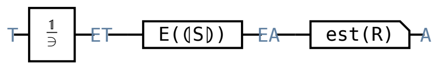
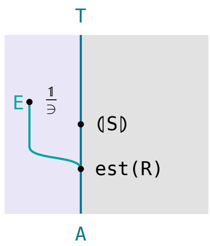
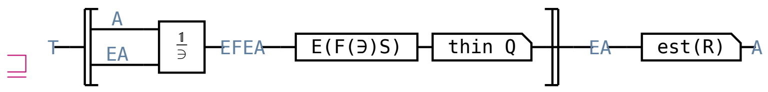
a thinning of the sorted list lists a thinning of the set — `P` a connected preorder, `thinlist Q⊑subseq`.

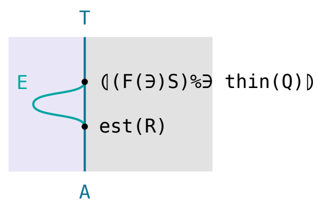
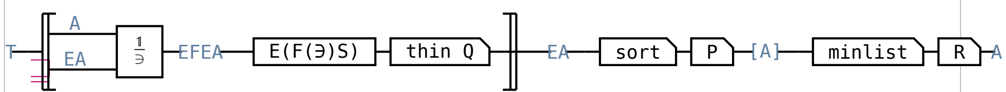
circuit — one wire, `EA` to `[A]`	**Hinze–Marsden**
	
=  `sort P≜setify° ordered P` — ≜ `thinlist`	
⊑  `ordered P thinlist Q⊑thinlist Q ordered P`, since `thinlist Q⊑subseq` — ≜ `thinlist` — and a subsequence of a `P`-ordered list is `P`-ordered	
⊑  ≜ `thinlist`'s `thinlist Q setify⊑setify thin Q` after `setify°`, at `setify°setify⊑𝟙` for `setify` simple — (5c) — then `·setify⊣·setify°` — (1.3a)	
=  `sort P≜setify° ordered P` — ≜ `thinlist`	

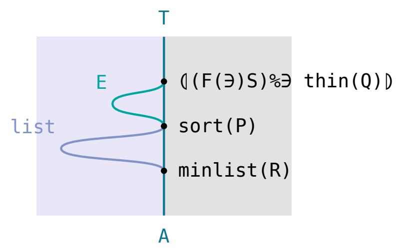
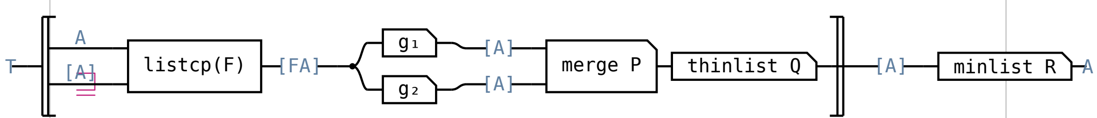
$\frac{\texttt{F(∋)fp}}{\texttt{∋}}$ **sort P⊒F(sort P) listcp(F) list(f) filter(p)** one sorted list built from sorted arguments, instead of a set built and then sorted — **f : FA→A** monotonic on **P**, **p** coreflexive, **F** linear.		(14.3d)
circuit — one wire, F(EA) to [A]	**Hinze–Marsden**	
F(EA) ─ [$\frac{\texttt{F(∋)fp}}{\texttt{∋}}$] ─ [sort P] ─ [A]		
= F(EA) ─ [cp(F)] ─ [E(fp)] ─ [sort P] ─ [A] $\frac{\texttt{F(∋)fp}}{\texttt{∋}}$ = $\frac{\texttt{F(∋)}}{\texttt{∋}}$ E(fp) — (10b); cp(F)≜$\frac{\texttt{F(∋)}}{\texttt{∋}}$ — ≜ thinlist		
= F(EA) ─ [cp(F)] ─ [P(f)] ─ [E(p)] ─ [sort P] ─ [A] E(fp)=E(f)E(p); E(f)=P(f) for f a map — (11.3d)		
⊒ F(EA) ─ [cp(F)] ─ [P(f)] ─ [sort P] ─ [filter(p)] ─ [A] sort P filter(p)⊑E(p) sort P — (14.3b)		
⊒ F(EA) ─ [cp(F)] ─ [sort(fPf°)] ─ [list(f)] ─ [filter(p)] ─ [A] sort(fPf°) list(f)⊑P(f) sort P — (14.3b)		
⊒ F(EA) ─ [cp(F)] ─ [sort(FP)] ─ [list(f)] ─ [filter(p)] ─ [A] FP⊑fPf° — (13.3b) at f a map; sort P≜setify° ordered P grows with P — ≜ thinlist		
⊒ F(EA) ─ [F(sort P)] ─ [listcp(F)] ─ [list(f)] ─ [filter(p)] ─ [A] F(sort P) listcp(F)⊑cp(F) sort(FP) — (14.3b), F linear		

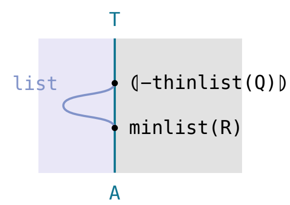
$$\tfrac{⦇S⦈}{\ni}\ \texttt{est(R)}\sqsupseteq⦇\texttt{listcp(F)}\ \langle g_1,g_2\rangle\ \texttt{merge P thinlist Q}⦈\ \texttt{minlist R} \qquad (14.3e)$$

a fold on sorted lists of partial solutions, thinned at every step — at ≙ **thinlist**'s binary thinning data.

<table>
<tr><th>circuit — one wire, T to A</th><th>Hinze–Marsden</th></tr>
<tr><td>
</td><td>
</td></tr>
<tr><td>

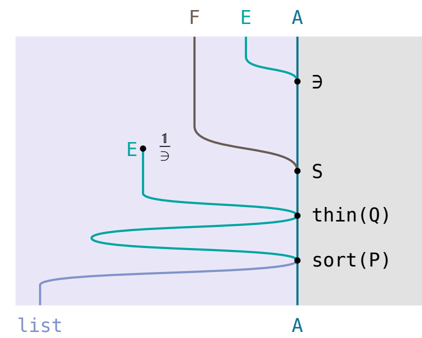
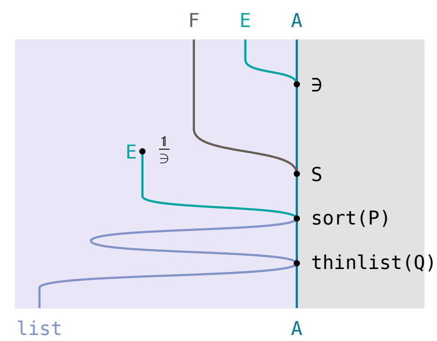
(14.1e) at f_1p_1 and f_2p_2 monotonic on Q — ≙ thinlist</td><td>
</td></tr>
<tr><td>

sort P minlist R⊑est(R) — (14.3b) at its Q≙R</td><td>
</td></tr>
<tr><td>

fusion at (14.3f)</td><td>
</td></tr>
</table>

$$\frac{F(\ni)S}{\ni}\ \mathtt{thin}\ Q\ \mathtt{sort}\ P \sqsupseteq F(\mathtt{sort}\ P)\ \mathtt{listcp}(F)\ \langle g_1,g_2\rangle\ \mathtt{merge}\ P\ \mathtt{thinlist}\ Q \qquad (14.3f)$$

sorting the candidate set is what turns the thinning algebra into an algebra on lists — the side condition of binary thinning theorem's last step.

circuit — one wire, $F(EA)$ to $[A]$	**Hinze–Marsden**
$F(EA)$ — $\boxed{\frac{F(\ni)S}{\ni}}$ — $\boxed{\mathtt{thin}\ Q}$ — $\boxed{\mathtt{sort}\ P}$ — $[A]$	
$\sqsupseteq$ $F(EA)$ — $\boxed{\frac{F(\ni)S}{\ni}}$ — $\boxed{\mathtt{sort}\ P}$ — $\boxed{\mathtt{thinlist}\ Q}$ — $[A]$ $\mathtt{sort}\ P\ \mathtt{thinlist}\ Q \sqsubseteq \mathtt{thin}\ Q\ \mathtt{sort}\ P$ — (14.3b)	
$=$ $F(EA)$ — $\boxed{\langle \frac{F(\ni)f_1p_1}{\ni}, \frac{F(\ni)f_2p_2}{\ni}\rangle}$ — $\boxed{\mathtt{cup}}$ — $\boxed{\mathtt{sort}\ P}$ — $\boxed{\mathtt{thinlist}\ Q}$ — $[A]$ $\langle \frac{F(\ni)f_1p_1}{\ni}, \frac{F(\ni)f_2p_2}{\ni}\rangle\ \mathtt{cup}\ \mathtt{sort}\ P\ \mathtt{thinlist}\ Q$ $S=(f_1p_1)\cup(f_2p_2)$ — ≙ thinlist, then ≙ cup	
$\sqsupseteq$ $F(EA)$ — $\boxed{\langle \frac{F(\ni)f_1p_1}{\ni}, \frac{F(\ni)f_2p_2}{\ni}\rangle}$ — $\boxed{\mathtt{sort}\ P\times\mathtt{sort}\ P}$ — $\boxed{\mathtt{merge}\ P}$ — $\boxed{\mathtt{thinlist}\ Q}$ — $[A]$ $\langle \frac{F(\ni)f_1p_1}{\ni}, \frac{F(\ni)f_2p_2}{\ni}\rangle(\mathtt{sort}\ P\times\mathtt{sort}\ P)\ \mathtt{merge}\ P\ \mathtt{thinlist}\ Q$ $(\mathtt{sort}\ P\times\mathtt{sort}\ P)\ \mathtt{merge}\ P\sqsubseteq \mathtt{cup}\ \mathtt{sort}\ P$ — (14.3b)	
$=$ $F(EA)$ — $\boxed{\langle \frac{F(\ni)f_1p_1}{\ni}\ \mathtt{sort}\ P, \frac{F(\ni)f_2p_2}{\ni}\ \mathtt{sort}\ P\rangle}$ — $\boxed{\mathtt{merge}\ P}$ — $\boxed{\mathtt{thinlist}\ Q}$ — $[A]$ $\langle \frac{F(\ni)f_1p_1}{\ni}\ \mathtt{sort}\ P, \frac{F(\ni)f_2p_2}{\ni}\ \mathtt{sort}\ P\rangle\ \mathtt{merge}\ P\ \mathtt{thinlist}\ Q$ $\langle X,Y\rangle(\mathtt{sort}\ P\times\mathtt{sort}\ P)=\langle X\ \mathtt{sort}\ P, Y\ \mathtt{sort}\ P\rangle$ — (11.1.2b)	
$\sqsupseteq$ $F(EA)$ — $\boxed{F(\mathtt{sort}\ P)}$ — $\boxed{\mathtt{listcp}(F)}$ — $\boxed{\langle g_1,g_2\rangle}$ — $\boxed{\mathtt{merge}\ P}$ — $\boxed{\mathtt{thinlist}\ Q}$ — $[A]$ $F(\mathtt{sort}\ P)\ \mathtt{listcp}(F)\ \langle g_1,g_2\rangle\ \mathtt{merge}\ P\ \mathtt{thinlist}\ Q$ (14.3d) at f_1, p_1 and at f_2, p_2, then $X\langle g_1,g_2\rangle\sqsubseteq\langle Xg_1,Xg_2\rangle$ — (11.1.2b); $g_i ≙ \mathtt{list}(f_i)\ \mathtt{filter}(p_i)$ — ≙ thinlist	

§14.4. The knapsack problem

(14.4a)

$\mathtt{vol},\mathtt{wt} : \mathtt{Item}\to\mathtt{Real}$, $\mathtt{value} ≙ \mathtt{list}(\mathtt{vol})\ \mathtt{sum}$, $\mathtt{weight} ≙ \mathtt{list}(\mathtt{wt})\ \mathtt{sum}$.

$\mathtt{subseq}=⦇[\mathtt{nil},\mathtt{cons}]\cup[\mathtt{nil},\pi_2]⦈$, , within w the coreflexive on xs with $\mathtt{weight}\ xs\le w$, $0\le w$.

$R ≙ \mathtt{value}\ge\mathtt{value}^\circ$, $Q ≙ R\cap(\mathtt{weight}\le\mathtt{weight}^\circ)$, $P ≙ R$, .

$FA=1+\mathtt{Item}\times A$, $\mathtt{listcp}(F)=\mathtt{wrap}+\mathtt{cpr}$, $g_1 ≙ \mathtt{list}([\mathtt{nil},\mathtt{cons}])\ \mathtt{filter}(\mathtt{within}\ w)=[\mathtt{list}(\mathtt{nil}),h_1]$, $g_2 ≙ \mathtt{list}([\mathtt{nil},\pi_2])=[\mathtt{list}(\mathtt{nil}),h_2]$.

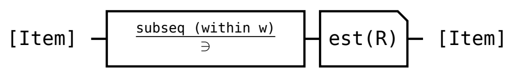
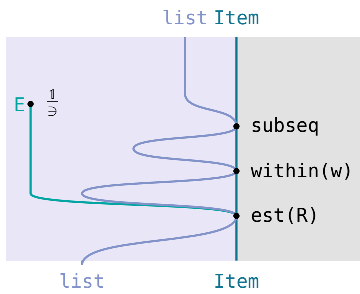
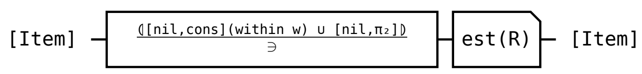
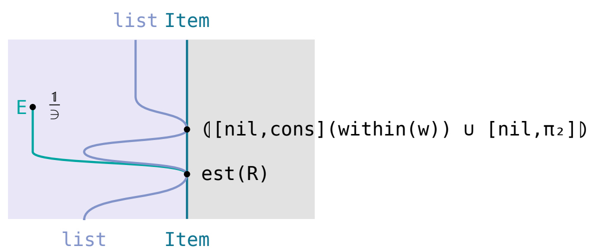
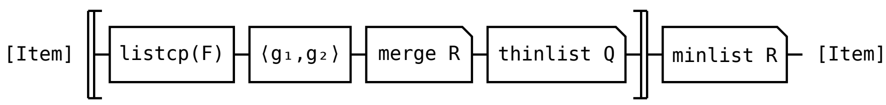
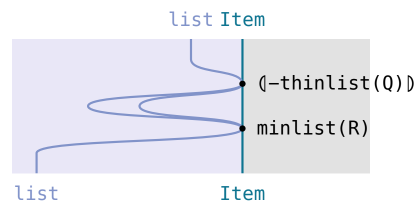
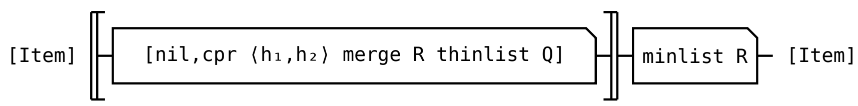
$h_1 ≙ \mathtt{list}(\mathtt{cons})\ \mathtt{filter}(\mathtt{within}\ w)$, $h_2 ≙ \mathtt{list}(\pi_2)$.

(14.4b)

the law	**what it says**
$(\mathbb{1}\times R)\ (\mathtt{cons}\ (\mathtt{within}\ w))\sqsubseteq\mathtt{cons}\ (\mathtt{within}\ w)R$ FALSE	not monotonic on R: a selection of greater value need not still fit once one more item goes in
$(\mathbb{1}\times Q)\ (\mathtt{cons}\ (\mathtt{within}\ w))\sqsubseteq\mathtt{cons}\ (\mathtt{within}\ w)Q$ $(\mathbb{1}\times Q)\pi_2\sqsubseteq\pi_2 Q$,	both halves are monotonic on Q once ties in value are broken by weight

$\frac{\texttt{subseq (within w)}}{\ni}\ \texttt{est(R)} \sqsupseteq (\!|\texttt{[nil,cpr}\ \langle h_1,h_2\rangle\ \texttt{merge R thinlist Q]}|\!)\ \texttt{minlist R}$ (14.4c)

the knapsack problem, as a fold that thins the packings kept at each item

circuit — one wire, `[Item]` to `[Item]`; the algebra inside the functorial box	**Hinze–Marsden**
 $\frac{\texttt{subseq (within w)}}{\ni}\ \texttt{est(R)}$	
 = $\frac{(\!|\texttt{[nil,cons](within w)}\ \cup\ \texttt{[nil,}\pi_2\texttt{]}|\!)}{\ni}\ \texttt{est(R)}$ fusion, weights non-negative.	
 $\sqsupseteq$ $(\!|\texttt{listcp(F)}\ \langle g_1,g_2\rangle\ \texttt{merge R thinlist Q}|\!)\ \texttt{minlist R}$ binary thinning theorem, at P≙R, F linear, Q from (14.4b)	
 = $(\!|\texttt{[nil,cpr}\ \langle h_1,h_2\rangle\ \texttt{merge R thinlist Q]}|\!)\ \texttt{minlist R}$ `listcp(F)=wrap+cpr`, g_i`=[list(nil),`h_i`]` — ≙ `within`; `minlist R` is `head`, packings coming out in descending value	

§14.5. The paragraph problem

(14.5a)

> $\mathtt{Line{=}list^{+}\ Word}$, $\mathtt{Para{=}list^{+}\ Line}$, $\mathtt{FA{=}Word{+}Word{\times}A}$, $\mathtt{listcp(F){=}wrap{+}cpr}$.
>
> $\mathtt{new\ (a,xs){=}[[a]]{+\!\!+}xs}$, $\mathtt{glue\ (a,xs){=}[[a]{+\!\!+}head\ xs]{+\!\!+}tail\ xs}$, $\mathtt{partition}{\triangleq}(\![\mathtt{[wrap\ wrap,new\ \cup\ glue]}]\!) : \mathtt{list^{+}\ Word{\to}Para}$.
>
> $\mathtt{width}{\triangleq}(\![\mathtt{[length,(length{\times}1\!\!1)\ plus\ succ]}]\!)$, $\mathtt{0{\le}length\ a}$, $\mathtt{fits\ w}$ the coreflexive on a line $\mathtt{x}$ with $\mathtt{width\ x{\le}w}$, $\mathtt{ok\ w}$ the coreflexive on $\mathtt{[x]{+\!\!+}xs}$ with $\mathtt{width\ x{\le}w}$.
>
> $\mathtt{white\ w\ x{=}w{-}width\ x}$, $\mathtt{collect}{\triangleq}\mathtt{list(sqr)\ sum}$, $\mathtt{waste\ w}{\triangleq}\mathtt{init\ list(white\ w)\ collect}$.
>
> $\mathtt{R}{\triangleq}\mathtt{(waste\ w){\le}(waste\ w)^{\circ}}$, $\mathtt{Q}{\triangleq}\mathtt{Rn(head\ head^{\circ})}$, $\mathtt{P}{\triangleq}\top$, .
>
> $\mathtt{g_1}{\triangleq}\mathtt{list([wrap\ wrap,new])}$, $\mathtt{g_2}{\triangleq}\mathtt{list([wrap\ wrap,glue])\ filter(ok\ w)}$, $\mathtt{start}{\triangleq}\mathtt{wrap\ wrap\ wrap}$, $\mathtt{h_1}{\triangleq}\mathtt{list(new)}$, $\mathtt{h_2}{\triangleq}\mathtt{list(glue)\ filter(ok\ w)}$.

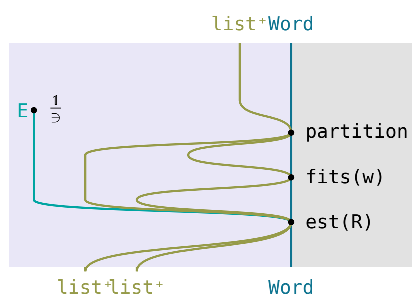
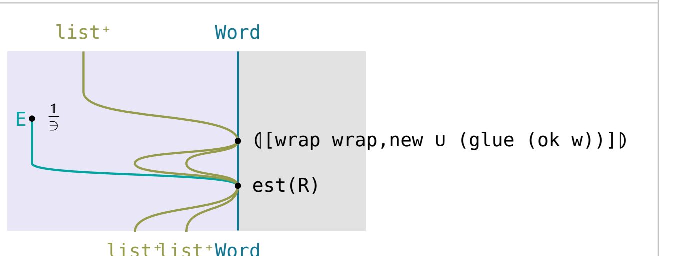
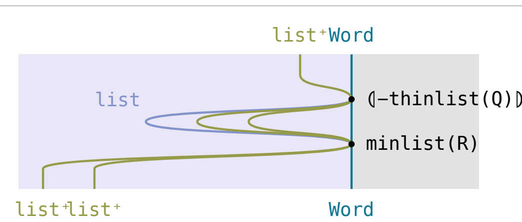
(14.5b)

the law	what it says
$\mathtt{(1\!\!1{\times}R)\ glue{\sqsubseteq}glue\ R}$ FALSE	$\mathtt{glue}$ is not monotonic on $\mathtt{R}$: the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this
$\mathtt{(1\!\!1{\times}Q)\ new{\sqsubseteq}new\ Q}$ $\mathtt{(1\!\!1{\times}Q)\ (glue\ (ok\ w)){\sqsubseteq}glue\ (ok\ w)Q}$ $\mathtt{cons}$ monotonic on $\mathtt{collect{\le}collect^{\circ}}$. ,	both halves are monotonic on $\mathtt{Q}$ once ties in waste are broken by the first line
$\mathtt{merge\ \top{=}cat}$; $\mathtt{P}{\triangleq}\mathtt{head\ prefix\ head^{\circ}}$ also serves	$\top$ needs no sorting at all, and $\mathtt{prefix}$ is a linear order on first lines of paragraphs of one input

(14.5c)

$$\underline{\mathtt{partition\ list^{+}(fits\ w)}}_{\ni}\ \mathtt{est(R)}\sqsupseteq(\![\mathtt{[start,cpr\ \langle h_1,h_2\rangle\ cat\ thinlist\ Q]}]\!)\ \mathtt{minlist\ R}$$

a paragraph laid out as a fold that thins the layouts kept at each word

circuit — one wire, $\mathtt{list^{+}\ Word}$ to $\mathtt{Para}$; the algebra inside the functorial box	**Hinze–Marsden**
list⁺ Word —[partition list⁺(fits w) ∋]—[est(R)]— Para $\underline{\mathtt{partition\ list^{+}(fits\ w)}}_{\ni}\ \mathtt{est(R)}$	
= list⁺ Word —[(\[[wrap wrap,new ∪ (glue (ok w))]\]) ∋]—[est(R)]— Para $\underline{(\![\mathtt{[wrap\ wrap,new\ \cup\ (glue\ (ok\ w))]}]\!)}_{\ni}\ \mathtt{est(R)}$ fusion, every word fits on a line by itself.	
= list⁺ Word —[(\[[wrap wrap,new] ∪ ([wrap wrap,glue] (ok w))\]) ∋]—[est(R)]— Para $\underline{(\![\mathtt{[wrap\ wrap,new]\ \cup\ ([wrap\ wrap,glue]\ (ok\ w))}]\!)}_{\ni}\ \mathtt{est(R)}$ the algebra as $\mathtt{(f_1p_1)\ \cup\ (f_2p_2)}$, $\mathtt{p_1}{\triangleq}1\!\!1$ — $\triangleq$ $\mathtt{thinlist}$.	
$\sqsupseteq$ list⁺ Word —⟦ [listcp(F)]—[⟨g₁,g₂⟩]—[cat]—[thinlist Q] ⟧—[minlist R]— Para $(\![\mathtt{listcp(F)\ \langle g_1,g_2\rangle\ cat\ thinlist\ Q}]\!)\ \mathtt{minlist\ R}$ binary thinning theorem, at $\mathtt{P}{\triangleq}\top$ with $\mathtt{merge\ \top{=}cat}$, $\mathtt{Q}$ from (14.5b)	
= list⁺ Word —⟦ [[start,cpr ⟨h₁,h₂⟩ cat thinlist Q]] ⟧—[minlist R]— Para $(\![\mathtt{[start,cpr\ \langle h_1,h_2\rangle\ cat\ thinlist\ Q]}]\!)\ \mathtt{minlist\ R}$ $\mathtt{listcp(F){=}wrap{+}cpr}$, $\mathtt{g_i}$ along the coproduct — $\triangleq$ $\mathtt{partition}$	

§14.6. Bitonic tours

(14.6a)

`FA=(City×City)+(City×A)`, the base functor of cons-lists of length at least two; `listcp(F)=wrap+cpr`; `tc : City×City→Real`, neither positive nor symmetric.

`start (a,b)=([a,b],[a,b])`, `dropl (a,([b]++xs,ys))=([a]++xs,[a]++ys)`, `dropr (a,(xs,[b]++ys))=([a]++xs,[a]++ys)`, `tour≜⦇[start,dropl ∪ dropr]⦈`.

`cost (xs,ys)=outcost xs+incost ys`, `outcost` $[a_0,\dots,a_n]$`=tc` $(a_0,a_1)+\cdots+$`tc` (a_{n-1},a_n), `incost` $[a_0,\dots,a_n]$`=tc` $(a_1,a_0)+\cdots+$`tc` (a_n,a_{n-1}).

`next≜tail head`, `next2≜next×next`, `head2≜head×head`, `R≜cost≤cost°` , `Q≜R∩(next2 next2°)∩(head2 head2°)`, `P≜T` , , g_1`≜list([start,dropl])`, g_2`≜list([start,dropr])`.

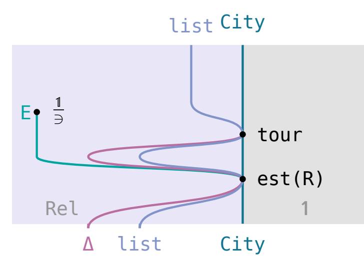
(14.6b)

the law	what it says
`(𝟙×R) dropl⊑dropl R` FALSE `(𝟙×R) dropr⊑dropr R` FALSE	neither drop is monotonic on `R`: the two edges it adds and removes depend on `head` and `next` of both lists
`(𝟙×Q) dropl⊑dropl Q` `(𝟙×Q) dropr⊑dropr Q` ,	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input

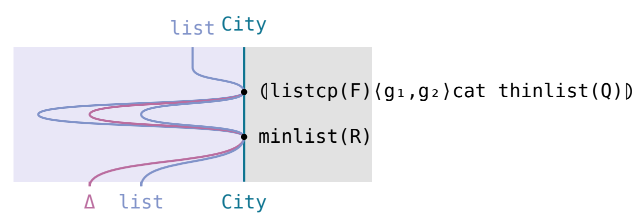
(14.6c)

$\frac{\texttt{tour}}{\ni}$ `est(R)⊒⦇[start wrap,cpr ⟨list(dropl),list(dropr)⟩ cat thinlist Q]⦈ minlist R`

a least-cost bitonic tour, as a fold that thins the tours kept at each city

circuit — one wire, `[City]` to `[City]×[City]`; the algebra inside the functorial box	**Hinze–Marsden**
`[City]` ─[$\frac{\texttt{tour}}{\ni}$]─[`est(R)`]─ `[City]×[City]` $\frac{\texttt{tour}}{\ni}$ `est(R)`	

`[City]` ─[$\frac{\texttt{⦇[start,dropl ∪ dropr]⦈}}{\ni}$]─[`est(R)`]─ `[City]×[City]`
 	
= $\frac{\texttt{⦇[start,dropl ∪ dropr]⦈}}{\ni}$ `est(R)`
`tour≜⦇[start,dropl ∪ dropr]⦈` — ≜ `tour`	

`[City]` ⟦─[`listcp(F)`]─[(g_1,g_2)]─[`cat`]─[`thinlist Q`]─⟧─[`minlist R`]─ `[City]×[City]` ⊒ `⦇listcp(F)` $\langle g_1,g_2\rangle$ `cat thinlist Q⦈ minlist R` binary thinning theorem, at `P≜T` with `merge T=cat`, `Q` from (14.6b)	
`[City]` ⟦─[`[start wrap,cpr ⟨list(dropl),list(dropr)⟩ cat thinlist Q]`]─⟧─[`minlist R`]─ `[City]×[City]` = `⦇[start wrap,cpr ⟨list(dropl),list(dropr)⟩ cat thinlist Q]⦈ minlist R` `listcp(F)=wrap+cpr`, g_i`=[list(start),list(`drop_i`)]` — ≜ `tour`; quadratic, two tours added per step	

§15. Dynamic Programming

§15.1. Theory

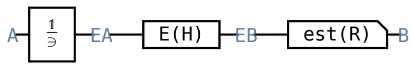
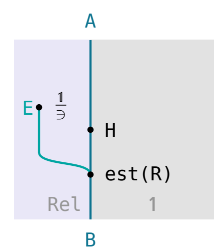
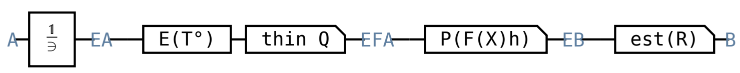
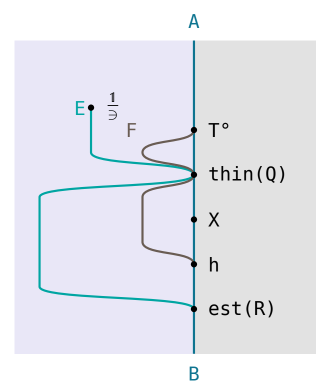
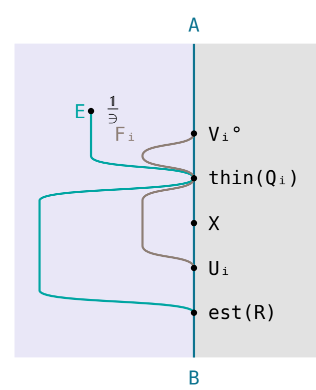
(15.1a)

> $h : FB \to B$ a map, $T : FA \to A$ an F-algebra, $R : B \to B$.
>
> $H \triangleq (\!|T|\!)^\circ (\!|h|\!) : A \to B$, $M \triangleq \frac{H}{\ni}$ $est(R)$ the problem to be solved, $(\mu X : G(X))$ as in $\triangleq$ μ.

(15.1b)

$$\frac{H}{\ni}\ est(R) \sqsupseteq (\mu X : \frac{T^\circ}{\ni}\ thin\ Q\ P(F(X)h)\ est(R))$$

an optimum over everything H returns is reached by taking the input apart every way T allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

circuit — one wire, A to B	**Hinze–Marsden**
 the problem to be solved, $H \triangleq (\!|T|\!)^\circ (\!|h|\!)$ — $\triangleq$ H	
 $\sqsupseteq$ h monotonic on R and Q a preorder with $QF(H)h \sqsubseteq F(H)hR$; $thin\ Q$ as in (14.1b). This is the same with the thinning step dropped — $\mathbb{1} \sqsubseteq thin\ Q$, so the body and with it the fixed point only shrink. Knaster–Tarski leaves (9.1) $\frac{T^\circ}{\ni}$ $P(F(M)h)$ $est(R) \sqsubseteq M$; $M = H \cap (H^\circ \backslash R^\circ)$ splits that into (9.2) $\frac{T^\circ}{\ni}$ $P(F(M)h)$ $est(R) \sqsubseteq H$ and (9.3) $H^\circ \frac{T^\circ}{\ni}$ $P(F(M)h)$ $est(R) \sqsubseteq R^\circ$, and both use only (9.4) = (7.10) $P(X)est(R) \sqsubseteq (\ni X) \cap (\in \backslash (XR^\circ))$ — (13.1.8a)	
 = Proposition 9.1 at $T=[V_1,V_2]$, $h=[U_1,U_2]$, $Q=Q_1+Q_2$, $V_2V_1^\circ=\perp$: FA is usually a coproduct, and disjoint ranges split the fixed point into one branch per summand. The fixed point is unique and entire — T° followed by F's membership relation inductive, $\frac{T^\circ}{\ni}$ finite and non-empty, R connected	

(15.1c)

the condition	**how it is discharged**
$F(R)h \sqsubseteq hR$ Proposition 9.2, $R \triangleq cost {\le} cost^\circ$, $h\ cost = F(cost)k$, $F(\le)k \sqsubseteq k{\le}$	monotonicity when the cost is itself a fold with a step k monotonic on $\le$
$F(R \cap (H^\circ H))h \sqsubseteq hR$ Proposition 9.3, $R \triangleq cost {\le} cost^\circ$, $h\ cost = F(\langle cost, H^\circ \rangle)k$, $F(\le \times \mathbb{1})k \sqsubseteq k{\le}$, H° simple;	monotonicity **in context**: k may also read the input the part was built from
$QF(H)h \sqsubseteq F(H)hR$ at $Q \triangleq F(U,V)$ Proposition 9.4, U, V preorders, $F(U,R)h \sqsubseteq hR$, $VH \sqsubseteq HR$	both conditions at once, split along the two arguments of a bifunctor

§15.2. The string edit problem

(15.2a)

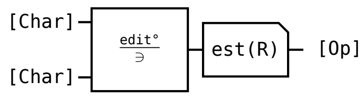
> $Op ::= cpy\ Char \mid del\ Char \mid ins\ Char$, $F(A,B) = 1 + (A \times B)$, $\alpha \triangleq [nil, cons]$.
>
> $edit \triangleq (\!|[base, step]|\!) : [Op] \to [Char] \times [Char]$, $base$ returning $([],[])$, $step\ (cpy\ a,(xs,ys)) = ([a] \mathbin{+\!\!+} xs, [a] \mathbin{+\!\!+} ys)$, $step\ (del\ a,(xs,ys)) = ([a] \mathbin{+\!\!+} xs, ys)$, $step\ (ins\ a,(xs,ys)) = (xs, [a] \mathbin{+\!\!+} ys)$.
>
> $length \triangleq (\!|[zero, \pi_2\ succ]|\!)$, $R \triangleq length {\le} length^\circ$, $U \triangleq \top$, $V \triangleq suffix^\circ \times suffix^\circ$, $Q \triangleq \mathbb{1} + (U \times V)$, $empty$ the coreflexive on (xs,ys) with both lists empty.

> `unstep` implements $\frac{\texttt{step}^\circ}{\ni}$ `thin(U×V)` : `unstep ([a]#xs,[])=[(del a,(xs,[]))]`, `unstep ([],[b]#ys)=[(ins b,([],ys))]`, `unstep ([a]#xs,[b]#ys)=(a=b→[(cpy a,(xs,ys))],[(del a,(xs,[b]#ys)),(ins b,([a]#xs,ys))])`.

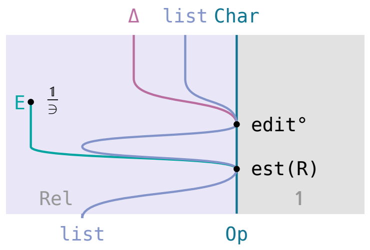
$$\tfrac{\mathtt{edit}^\circ}{\ni}\ \mathtt{est(R)} \sqsupseteq \mathtt{mle},\quad \mathtt{mle=(empty{\to}nil,unstep\ list((𝟙{\times}mle)cons)\ minlist\ R)} \tag{15.2b}$$

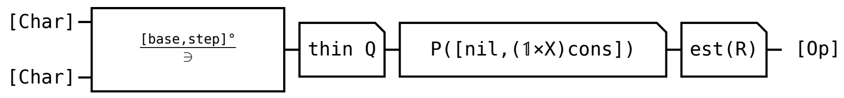
a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

circuit — two wires, one per string | **Hinze–Marsden**

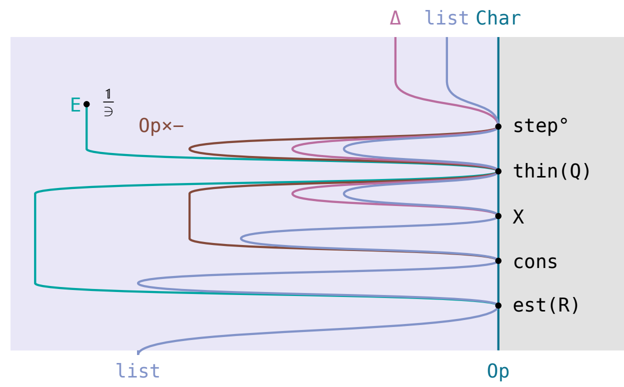


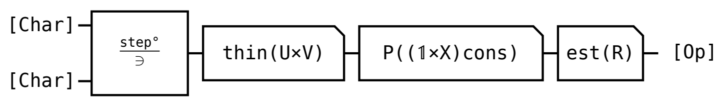
the specification — ≜ edit



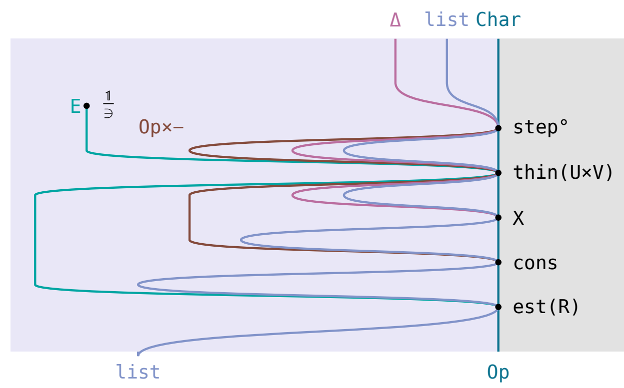


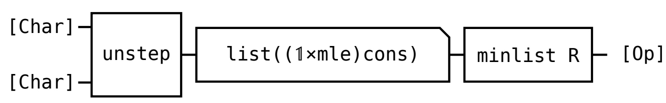
⊒ at $Q≜𝟙+(U×V)$. Monotonicity $F(R)α⊑αR$ is Proposition 9.2 at $length≜⦇[zero,π_2\ succ]⦈$ with $succ$ monotonic on $≤$, so $cons$ is monotonic on R. The thinning condition $QF(𝟙,edit°)α⊑F(𝟙,edit°)αR$ over $F(Op,[Char]×[Char])$ is Proposition 9.4 at $U≜⊤$ — $F(⊤,R)α⊑αR$, left as an exercise, so any two operations may be compared — and $V≜suffix°×suffix°$, — $edit\ \ (𝟙×suffix)⊑R°\ \ edit$, $edit\ \ (suffix×𝟙)⊑R°\ \ edit$. $suffix=tail*$ and $BA⊑CB⟹BA*⊑C*B$ cut those to one step each: drop the operation that produced the head, or weaken its `cpy` to a `del`, never lengthening the sequence



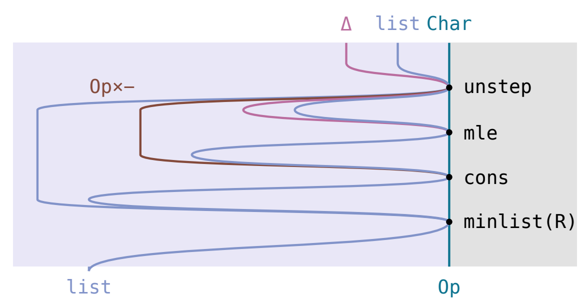


= Proposition 9.1: `base` and `step` have disjoint ranges, so the fixed point splits into one branch per summand — `empty`, the coreflexive on `(xs,ys)` with both lists empty, is where `base` returns





⊒ `unstep` implements $\tfrac{step°}{\ni}$ `thin(U×V)` — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and `minlist R` implements `est(R)`. The same subproblem is solved many times over, so the running time is exponential in the two lengths



= `mle (xs,ys)=head (column xs ys), column xs ys=[mle (u,ys)|u←tails xs]`
`column xs=⦇[fstcol xs,nextcol xs]⦈, fstcol=list(del) tails`
the tabulation: `mle (xs,ys)` needs `mle (u,v)` for every tail `u` of `xs` and `v` of `ys`, so the columns are built right to left

= `column xs ([b]++ys)=nextcol xs (b,column xs ys)`
`nextcol xs (b,us)=⦇[base (b,last us),step b]⦈ xus, xus=zip (xs,zip (init us,tail us))`
each column is a fold built bottom to top, over `xs` zipped with the adjacent pairs of the column to its right

= `base (b,u)=[[ins b]++u]`
`step b ((a,(u,v)),ws)=(a=b→[[cpy a]++v]++ws, [bmin(R) ([del a]++w,[ins b]++u)]++ws)`
`w=head ws`; these `base`, `step` are not `edit`'s. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§15.3. Optimal bracketing

(15.3a)

tree A::=tip A|bin (tree A,tree A), FX=A+X², so F(R)=𝟙+R²; h≙[tip,bin], flatten≙⦇[wrap,cat]⦈ : tree A→list⁺ A , H=flatten°.

⟨cost,size⟩≙⦇[opt,opb]⦈, opt≙⟨zero,st⟩, opb ((cx,sx),(cy,sy))=(cb (sx,sy)+cx+cy,sb (sx,sy)).

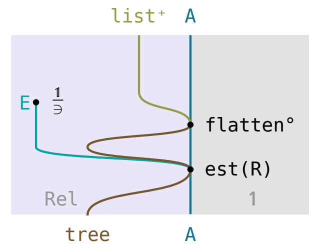
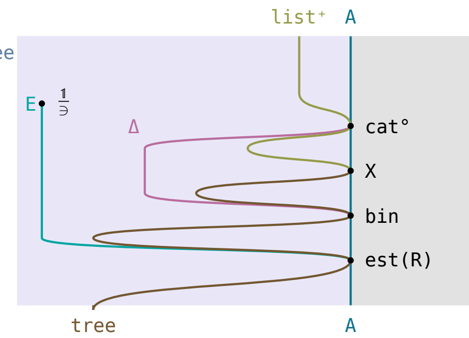
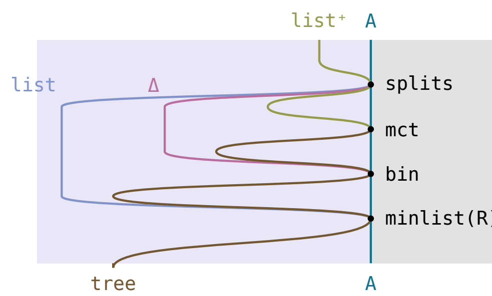
sb associative, so size=flatten sz for a map sz R≙cost≤cost° , g≙[zero,(𝟙×sz)² opb π₁], single the coreflexive on singleton lists.

splits≙⟨inits⁺,tails⁺⟩ zip, an implementation of $\frac{\texttt{cat}^\circ}{\ni}$; array≙inits list(row), row≙tails list(mct), col≙inits list(mct).

mix≙zip list(bin) minlist R, next≙⟨π₁,mix⟩ snoc, process≙((tip wrap)×𝟙) loop(next).

$$\frac{\texttt{flatten}^\circ}{\ni}\ \texttt{est(R)⊒mct,}\quad \texttt{mct=(single→head tip,⟨init col,tail row⟩ mix)}$$ (15.3b)

a least-cost bracketing of $a_1\oplus\cdots\oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit — one wire, `list⁺ A` to `tree A`	**Hinze–Marsden**
`list⁺ A` —[$\frac{1}{\ni}$]— `list⁺ A` —[`E(flatten°)`]— `tree A` —[`est(R)`]— `tree A` the specification — ≜ `flatten`	
`list⁺ A` —[$\frac{1}{\ni}$]— `list⁺ A` —[`E([wrap,cat]°)`]— `EF list⁺ A` —[`P([tip,(X×X)bin])`]— `tree A` —[`est(R)`]— `tree` ⊒ split the list in every way, bracket both halves, join. The condition is monotonicity **in context**, `F(R∩(flatten flatten°))h⊑hR` — only trees with the same flattening are compared — which is Proposition 9.3 at `H°=flatten`, a map, with (9.5) `[tip,bin] cost=(𝟙+⟨cost,flatten⟩²)g` (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) `(𝟙+(≤×𝟙)²)g⊑g≤` (`g` monotonic on `≤` in its two cost arguments)	
`list⁺ A` —[$\frac{1}{\ni}$]— `list⁺ A` —[`E(cat°)`]— `E (list⁺ A)²` —[`P((X×X)bin)`]— `tree A` —[`est(R)`]— `tree A` = Proposition 9.1: `wrap` and `cat` have disjoint ranges, and `single` is the coreflexive on singleton lists, where `wrap` returns	
`list⁺ A` —[`splits`]— `[(list⁺ A)²]` —[`list((mct×mct)bin)`]— `[tree A]` —[`minlist R`]— `tree A` ⊒ `splits≜⟨inits⁺,tails⁺⟩ zip` implements $\frac{\texttt{cat}^\circ}{\ni}$ and `minlist R` implements `est(R)`. Exponential, since the segments of one list overlap	
= `mct=(single→head tip,⟨init col,tail row⟩ mix)` (9.7) the tabulation: `mct xs` is needed for every non-empty segment `xs`, so the values are held as an array of rows, `array≜inits list(row)`, `row≜tails list(mct)`, `col≜inits list(mct)`, `mix≜zip list(bin) minlist R`	
= `col=(single→head tip wrap,⟨init col,tail row⟩ next)` (9.8) `cons col=(𝟙×array) process` (9.9) `row=(single→head tip wrap,⟨mct,tail row⟩ cons)` (9.10) a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, `next≜⟨π₁,mix⟩ snoc`, `process≜((tip wrap)×𝟙) loop(next)`	
= `array=⦇[fstcol,addcol]⦈,  fstcol≜tip wrap wrap` `addcol≜⟨π₁ tip wrap,step⟩ cons,  step≜(process tail,π₂) zip list(cons)` the program: one fold building the array column by column, cubic in the length of the input	

§15.4. Data compression

(15.4a)

`[A]::=nil|snoc ([A],A)`, `list⁺ A::=wrap A|snoc (list⁺ A,A)`, `String=[Char]`, `Code::=sym Char|ptr (String,String⁺)`, `F(Code,String)=1+(String×Code)`, `α≜[nil,snoc]`.

`decode≜⦇[nil,extend]⦈ : [Code]→String`, `extend (xs,sym a)=xs⧺[a]`, `extend (xs,ptr (ys,zs))=xs⧺zs` when `ys⧺zs` is a proper prefix of `xs⧺zs`; `H=decode°`.

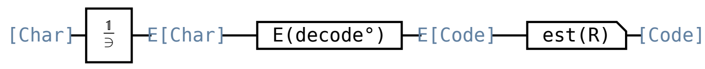
`size≜⦇[zero,distr [𝟙×c,𝟙×p] plus]⦈` with `c`, `p` the constant costs of a symbol and a pointer; `R≜size≤size°`, `Q≜F(⊤+⊤,prefix°)=𝟙+(prefix°×(⊤+⊤))`, the two `⊤` on symbols and on pointers.

`lrt ws=est(prefix°×(T+T)) {(xs,(ys,zs))|xs#zs=ws, ys#zs` a proper prefix of `ws}`, the longest repeated tail; `reduce (ws#[a])=(zs≠[]→[(ws,sym a),(xs,ptr (ys,zs))],[(ws,sym a)])` with `(xs,(ys,zs))=lrt (ws#[a])`.

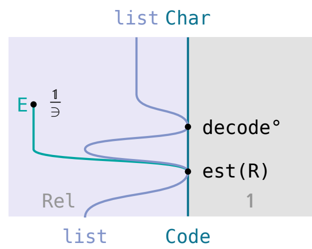
$\frac{\texttt{decode°}}{\ni}$ **`est(R)⊒encode,  encode=(null→nil,reduce list((encode×𝟙)snoc) minlist R)`** (15.4b)

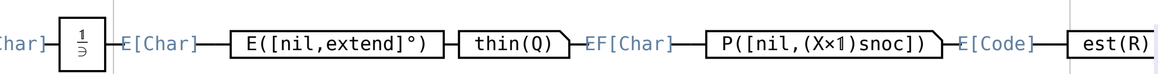
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit — one wire, `String` to `[Code]`	**Hinze–Marsden**

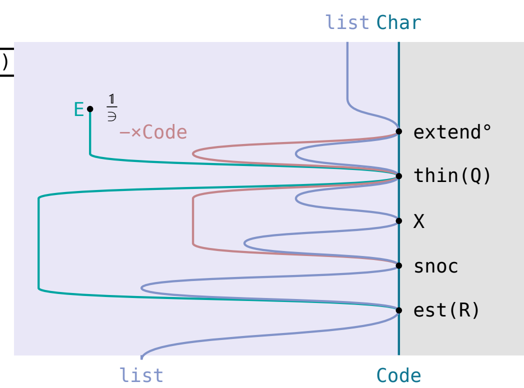


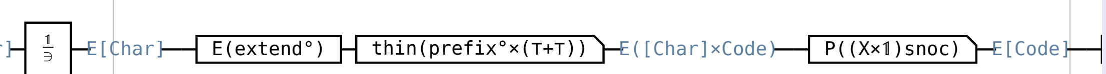
the specification — ≜ `decode`



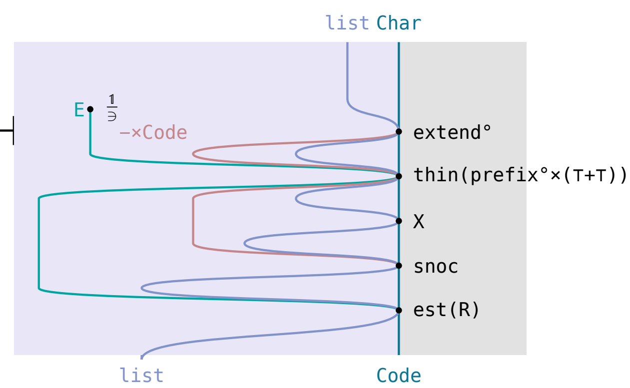


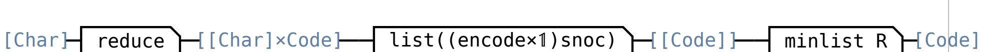
⊒ at `Q≜F(T+T,prefix°)=𝟙+(prefix°×(T+T))`, the two `T` on symbols and on pointers. Monotonicity `F(R)α⊑αR` is routine, the two costs being constants: `(T+T)[c,p]=[c,p]`. Proposition 9.4 splits the thinning condition in two — `F(T+T,R)α⊑αR`, and `decode prefix⊑R° decode`, for which `decode init⊑R° decode` is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



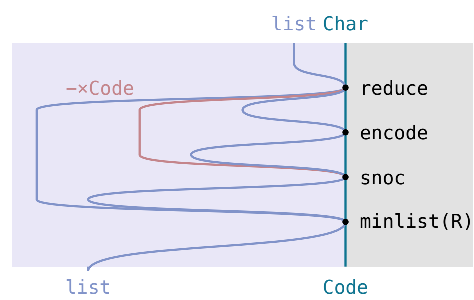


= Proposition 9.1: `nil` and `extend` have disjoint ranges. The decompositions of one string are $\frac{\texttt{extend°}}{\ni}$ `(ws#[a])={(ws,sym a)} ∪ {(xs,ptr (ys,zs))|xs#zs=ws#[a], ys#zs` a proper prefix of `ws}` — take the last character as a symbol, or end with a pointer





⊒ `reduce` implements $\frac{\texttt{extend°}}{\ni}$ `thin(prefix°×(T+T))`: thinning leaves at most two, the symbol and the pointer of the longest repeated tail `lrt`. Again exponential; the book gives no tabulation for it



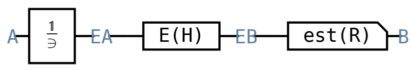
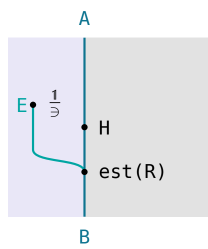
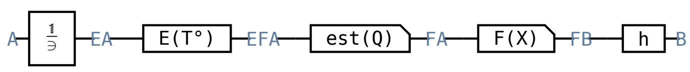
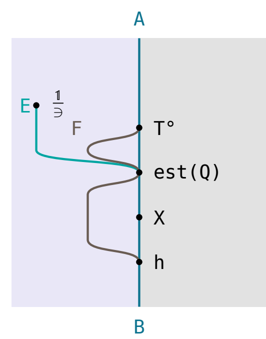
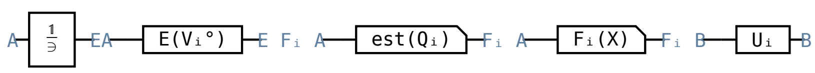
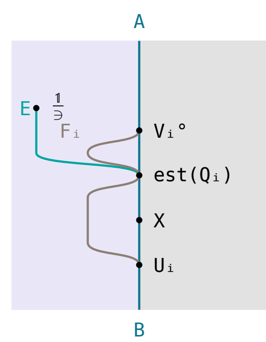
§16. Greedy Algorithms

§16.1. Theory

h, T, R, H, M as in ≜ H; additionally Q a **connected** preorder on the sets $\frac{T°}{\ni}$ returns, so that $\frac{T°}{\ni}$ est(Q) is entire. (16.1a)

$\frac{H}{\ni}$ est(R)⊒(μX : $\frac{T°}{\ni}$ est(Q) F(X)h) (16.1b)

the same optimum reached by keeping ONE decomposition at each step, so that no set is ever carried and the recursion runs on values alone

circuit — one wire, A to B	**Hinze–Marsden**
 the problem to be solved, H≜⦇T⦈°⦇h⦈ — ≜ H, Q	
⊒ 	
=  Proposition 10.1 at T=[V_1,V_2], h=[U_1,U_2], Q=Q_1+Q_2, V_2V_1°=⊥	

§16.2. The detab-entab problem

detab≜⦇[nil,expand]⦈ : String→String over snoc-lists, α≜[nil,snoc], H=detab°; expand (xs,a)=(a=TB→fill xs,xs#[a]), fill xs=xs#blanks (n−(col xs) mod n). (16.2a)

col≜⦇[zero,count]⦈, count (c,a)=(a=NL→0,c+1); TB the tab, BL the blank, NL the newline, tab stops every n columns.

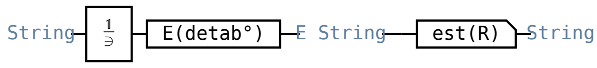
R≜length≤length° , U the preorder with a U b⟺a=TB∨a=b, V≜prefix°∩(fill fill°) , Q≜𝟙+(V×U) .

unfill xs the shortest prefix of xs with fill (unfill xs)=fill xs; tbc the trailing blank count, triple≜⟨unfill entab,⟨tbc,col⟩⟩.

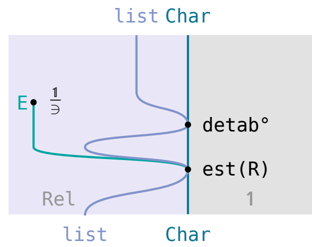
`detab° ∋ est(R)⊒entab,  entab=triple assocl π₁ (𝟙×blanks) cat` (16.2b)

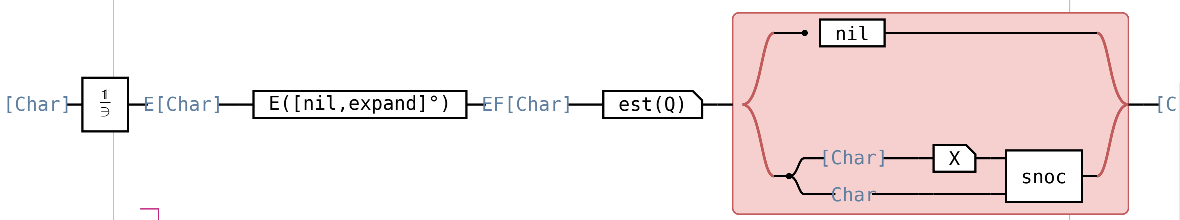
the shortest input **detab** expands to the given output is one pass along that output, holding each blank back and cashing the held blanks in for a tab wherever the column reaches a tab stop

circuit — one wire, `String` to `String` | **Hinze–Marsden**



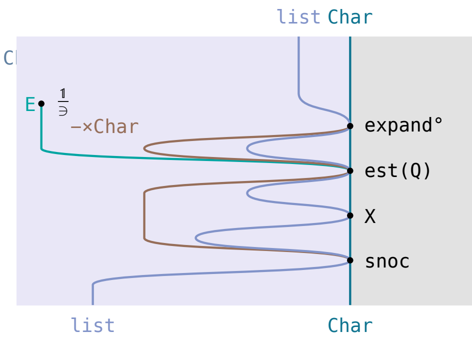
the specification — ≙ `detab`; `detab entab=𝟙` and nothing shorter does





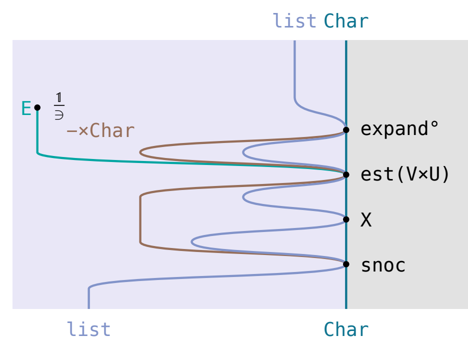
⊒

at `Q≙𝟙+(V×U)` : one character of input is decided at each step. `F(⊤,R)α⊑αR`. `detab prefix⊑R° detab` is FALSE , — at `n=8`, `detab [a,b,c,d,e,TB]=[a,b,c,d,e,BL,BL,BL]`, whose prefix `[a,b,c,d,e,BL,BL]` is longer than any input giving it, and `detab V°⊑R° detab` holds. `expand V°⊑expand ∪ (π₁V°)` — shortening the output either leaves the last step alone or discards it



=

Proposition 10.1: `nil` and `expand` have disjoint ranges. The greedy step is to emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop



=

`entab xs=entab (unfill xs)⧺blanks (tbc xs)` (10.1)

what makes `triple≙⟨unfill entab,⟨tbc,col⟩⟩` a snoc-list reduce: the output splits at the last tab stop

=

```
triple=⦇[base,op]⦈, entab=triple assocl π₁ (𝟙×blanks)
cat
```
base returns `([],(0,0))`, `op ((xs,(t,c)),a)=`
```
  (a=BL∧(c+1) mod n≠0→(xs,(t+1,c+1)),  a=BL→(xs⧺[TB],
(0,c+1)),
  a=NL→(xs⧺blanks t⧺[NL],(0,0)),  (xs⧺blanks t⧺[a],
(0,c+1)))
```
the program: one pass carrying the column and the count of pending blanks

§16.3. The minimum tardiness problem

(16.3a)

`FX=1+(X×Job)`, `α≙[nil,snoc]` on schedules, `β≙[nil,snag]` on bags, `snag` putting a job into a bag; `bagify≙⦇β⦈ : [Job]→Bag Job`, `H=bagify°`.

`ct`, `dt`, `wt : Job→Real` the completion, due and weighting quantities of a job; `penalty (xs,j)=(sum (list(ct) xs)+ct j-dt j)×wt j`.

`cost≙ prefix∋ P(α° [zero,penalty]) est(≥)`, `cost []=0`, `cost (xs⧺[j])=bmax (cost xs,penalty (xs,j))`, `R≙cost≤cost°`.

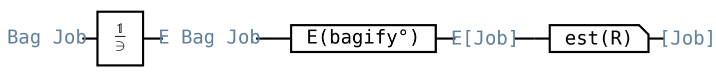
`perm≙bagify bagify°=⦇[nil,add]⦈`, `add (xs,j)=ys⧺[j]⧺zs` for some `xs=ys⧺zs`.

`k≙[zero,assocr (𝟙×((bagify°×𝟙) penalty)) bmax]`, `f≙[zero,(bagify°×𝟙) penalty]`, `Q≙f≤f°`, `Q'≙(bagify°×𝟙) penalty≤penalty°(bagify×𝟙)`.

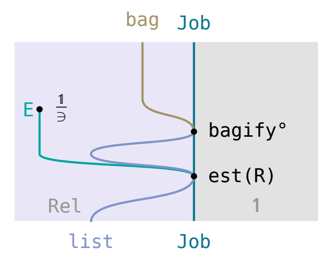
$\frac{\texttt{bagify°}}{\ni}$ `est(R)⊒schedule`, `schedule=(null→nil,pick (schedule×𝟙) snoc)` (16.3b)

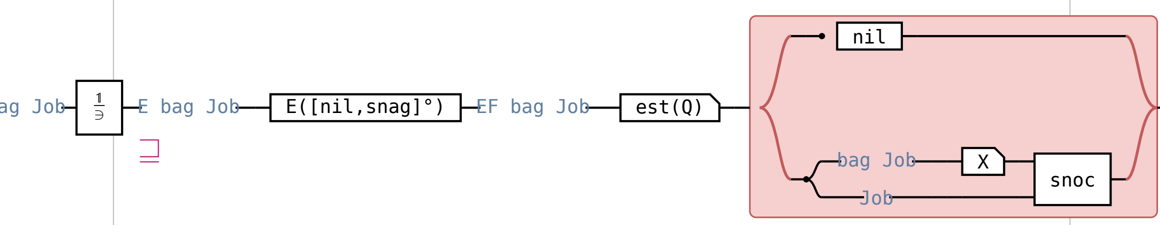
an ordering of the given bag with least maximum penalty is got by taking a job of least penalty out of the bag, putting it last, and scheduling what is left the same way

circuit — one wire, `Bag Job` to `[Job]` | **Hinze–Marsden**

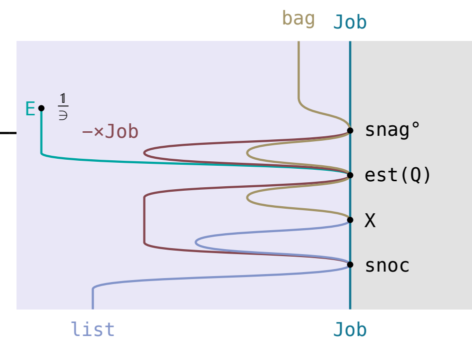


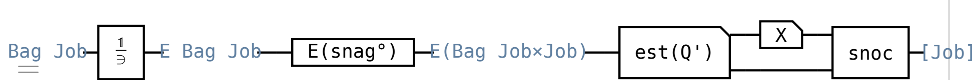
the specification — ≜ `bagify`



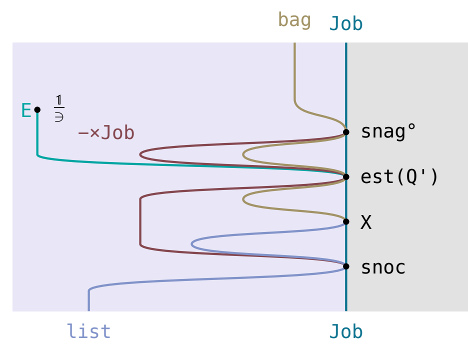


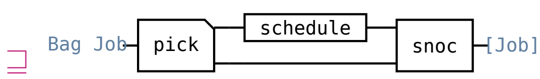
No greedy **reduce** exists — one would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole



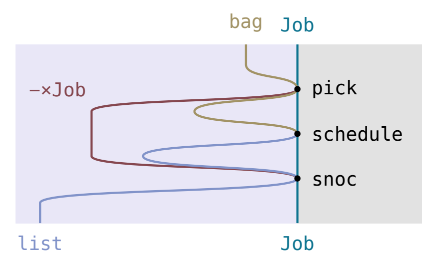


Proposition 10.1: `nil` and `snag` have disjoint ranges





`pick⊑`$\frac{\texttt{snag°}}{\ni}$ `est(Q')`, a partial function, quadratic in the number of jobs



§16.4. The TeX problem

(16.4a)

intern≜val round : Decimal→$[0,2^{16})$, val≜⦇[zero,shift]⦈, shift (d,r)=(d+r)/10, round r rounds 2^{16}r to the nearest integer: round r=n⟺$2n−1<2^{17}r<2n+1$.

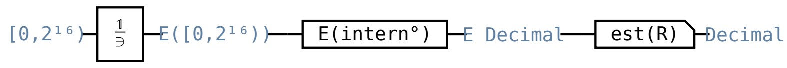
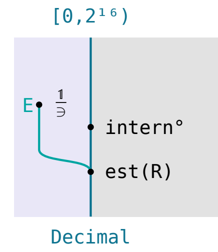
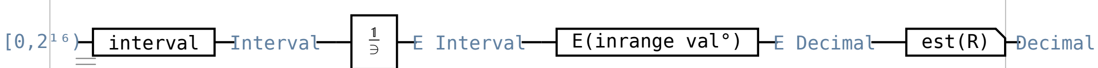
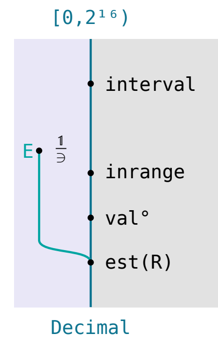
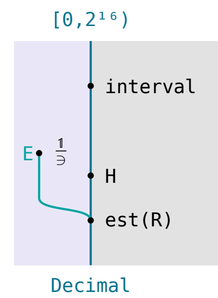
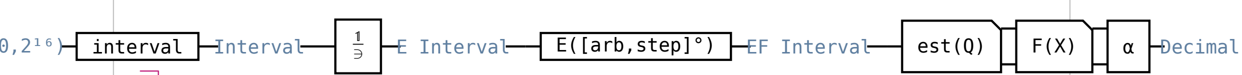
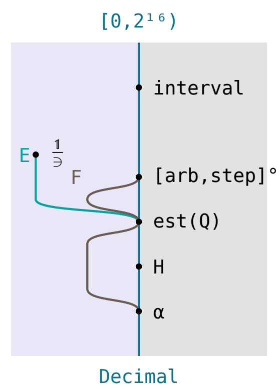
interval n=$((2n−1)/2^{17},(2n+1)/2^{17})$, r inrange (a,b)⟺a<r<b, round°=interval inrange, R≜length≤length°.

Interval the pairs (a,b) with 0<b<1 and a<b (10.9) [arb,step] : 1+(Digit×Interval)→Interval, step (d,(a,b))=((d+a)/10,(d+b)/10).

FX=1+(Digit×X), α≜[nil,cons], H≜⦇[arb,step]⦈°, ! : Digit×Interval→1, Q≜(l°!°r) ∪ 𝟙 l, r are (11.2a)'s injections into FX=1+(Digit×X), so l : 1→F(Interval) and r : Digit×Interval→F(Interval), w≜2^{17}.

$$\frac{\mathtt{intern}^\circ}{\ni}\ \mathtt{est(R)}\sqsupseteq\mathtt{extern},\quad \mathtt{extern\ n=f\ (2n-1,2n+1)}$$ (16.4b)

a shortest decimal whose internal representation is the given multiple of 2^{-16} is got by emitting the one digit the interval of admissible reals allows, until that interval contains zero and the empty decimal will do

circuit — one wire, $[0,2^{16})$ to Decimal	**Hinze–Marsden**
 the specification — ≜ intern	
 = round° is not a map, but interval is, so it comes out of the transpose	
 = fusion: val inrange°=⦇[arb,step]⦈ — the converse of val, cut down to intervals, is a reduce on cons-lists	
 ⊒ $\frac{\mathtt{[arb,step]^\circ}}{\ni}$ returns at most two elements — stop, or take one more digit — and ! nil⊑cons R° makes it stop whenever stopping is legal	
= extern=interval f, f (a,b)=(a<0→[],[d]#f (10a−d,10b−d)) the program, with d the digit above	
= extern n=f (2n−1,2n+1), f (p,q)=(p≤0→[],[d]#f (10p−w d,10q−w d)) d=(10q) div w: the same in integer arithmetic only, as chapter 3 required of intern — every interval reached is (p/w,q/w)	

§17. Appendix

§17.1. P(S) est(R)=(∋S)∩(∈\(SR°))

(p,q) tabulates $W \triangleq (\ni S)\cap(\in\backslash(SR^\circ))$, and $y \triangleq \frac{(p\ni S)\cap(qR)}{\ni}$, a map. (17.1a)

$$\boxed{\frac{W}{P(S)\ est(R)}} \xLeftarrow[p^\circ q=W,\ \mathbb{1}\sqsubseteq yy^\circ]{} \boxed{\frac{p^\circ y}{P(S)}}\ \boxed{\frac{y^\circ q}{est(R)}} \tag{17.1b}$$

$$\boxed{\frac{y^\circ q}{\ni}} \xLeftarrow[f^\circ\cdot\dashv f\cdot,\ \cdot\ni\dashv_{\ni}]{} \boxed{\frac{q}{(p\ni S)\cap(qR)}} \xLeftarrow[\Delta\dashv\cap,\ f^\circ\cdot\dashv f\cdot]{} \boxed{\frac{p^\circ q}{\ni S}}\ \underset{\text{R reflexive}}{\boxed{\frac{\mathbb{1}}{R}}} \tag{17.1c}$$

$$\boxed{\frac{\in y^\circ q}{R^\circ}} \xLeftarrow[\cdot\ni\dashv_{\ni},\ ^\circ,\ \text{meet}]{} \underset{\text{q simple}}{\boxed{\frac{R^\circ q^\circ q}{R^\circ}}}$$

$$\boxed{\frac{p^\circ y}{P(S)}} \xLeftarrow[\Delta\dashv\cap,\ \cdot T\dashv/T,\ ^\circ,\ f^\circ\cdot\dashv f\cdot]{} \boxed{\frac{p^\circ y\ni}{\ni S}}\ \boxed{\frac{p\ni}{y\ni S^\circ}} \tag{17.1d}$$

$$\boxed{\frac{p^\circ y\ni}{\ni S}} \xLeftarrow[\cdot\ni\dashv_{\ni},\ \text{meet}]{} \underset{\text{p simple}}{\boxed{\frac{p^\circ p\ni S}{\ni S}}}$$

$$\boxed{\frac{p\ni}{y\ni S^\circ}} \xLeftarrow[\text{modular law}]{} \boxed{\frac{p\ni}{(p\ni)\cap(qRS^\circ)}} \xLeftarrow[\mathbb{1}\sqsubseteq qq^\circ]{} \boxed{\frac{q^\circ p\ni}{RS^\circ}} \xLeftarrow[^\circ,\ T\cdot\dashv T\backslash]{} \underset{\text{W's right half}}{\boxed{\frac{W}{\in\backslash(SR^\circ)}}}$$

§17.2. P(est(R)) est(R)=P(Dom(est(R))) union est(R)

$$\boxed{P(est(R))\ est(R)} \overline{\overline{\ Dom(est(R))\ est(R)=est(R)\ }} \boxed{P(Dom(est(R))\ est(R))\ est(R)} \tag{17.2a}$$

$$\overline{\overline{\ P\ \text{a relator}\ }} \boxed{P(Dom(est(R)))P(est(R))\ est(R)} \underset{(13.1.9a)}{\sqsubseteq} \boxed{P(Dom(est(R)))\ union\ est(R)}$$

$$\boxed{\frac{P(Dom(est(R)))\ union\ est(R)}{P(est(R))\ est(R)}} \xLeftarrow[\text{(17.1b) at } S:=est(R),\ \Delta\dashv\cap,\ T\cdot\dashv T\backslash]{} \boxed{\frac{P(Dom(est(R)))\ union\ est(R)}{\ni est(R)}}\ \boxed{\frac{\in P(Dom(est(R)))\ union\ est(R)}{est(R)R^\circ}} \tag{17.2b}$$

$$\boxed{\frac{P(Dom(est(R)))\ union\ est(R)}{\ni est(R)}} \xLeftarrow[P(Dom(est(R)))\sqsubseteq\mathbb{1}]{} \boxed{\frac{union\ est(R)}{\ni est(R)}}$$

$$\xLeftarrow[T(U\cap V)\sqsubseteq TU\cap TV,\ \cdot\ni\dashv_{\ni},\ (13.1.4a)]{} \boxed{\frac{(\ni\ni)\cap(\in\backslash(\in\backslash R^\circ))}{\ni est(R)}} \xLeftarrow[\text{modular law}]{} \underset{\text{counit of } T\cdot\dashv T\backslash}{\boxed{\frac{\ni(\ni\cap(\in(\in\backslash(\in\backslash R^\circ))))}{\ni(\ni\cap(\in\backslash R^\circ))}}}$$

$$\boxed{\frac{\in P(Dom(est(R)))\ union\ est(R)}{est(R)R^\circ}} \xLeftarrow[\in\ \text{lax natural}]{} \boxed{\frac{Dom(est(R))\in union\ est(R)}{est(R)R^\circ}}$$

$$\xLeftarrow[\texttt{Dom(est(R))⊑est(R) est(R)°}]{} \boxed{\dfrac{\texttt{est(R) est(R)°∈union est(R)}}{\texttt{est(R)R°}}} \xLeftarrow[\texttt{est(R)°⊑∈},\ \texttt{∈∈union⊑∈}]{} \underset{\text{UP of est}}{\boxed{\dfrac{\texttt{est(R)∈est(R)}}{\texttt{est(R)R°}}}}$$